



Deusto

Facultad de Ingeniería
Ingeniaritza Fakultatea

Grado en Ingeniería Informática

Informatikako Ingeniaritzako gradua

Proyecto fin de grado

Gradu amaierako proiektua

Desarrollo de un videojuego 3D de mazmorras con
generación procedural basada en Unity

(Cortesía de JAPG)

Director: Pablo García Bringas

Bilbao, julio de 2023



Deusto

Facultad de Ingeniería
Ingeniaritza Fakultatea

Grado en Ingeniería Informática

Informatikako Ingeniaritzako gradua

Proyecto fin de grado

Gradu amaierako proiektua

Desarrollo de un videojuego 3D de mazmorras con
generación procedural basada en Unity

(Cortesía de JAPG)

Director: Pablo García Bringas

Bilbao, julio de 2023

Firmado digitalmente por GARCIA
BRINGAS, PABLO (FIRMA)
Nombre de reconocimiento (DN):
c=ES, serialNumber=22754794C,
sn=GARCIA, givenName=PABLO,
cn=GARCIA BRINGAS, PABLO (FIRMA)
Fecha: 2023.06.19 16:31:03 +02'00'

Resumen

Este proyecto, perteneciente al área de conocimiento Informática Gráfica y Experiencia de Usuario, se centra en el diseño y desarrollo de un videojuego de un solo jugador tridimensional en primera persona de mazmorras generadas procedimentalmente usando Unity. Este videojuego será jugable en cualquier ordenador mediante un archivo ejecutable.

En este videojuego el jugador tendrá que escapar de una mazmorra que se genera de manera procedimental y que está compuesta por pisos llenos de salas, por lo que el jugador tendrá que avanzar por las salas hasta encontrar el portal, custodiado por el enemigo final del piso, que llevará al jugador al siguiente piso. En este trayecto, el jugador luchará contra varios enemigos y obtendrá y/o comprará mejoras temporales para volverse más poderoso usando el oro que darán los enemigos al derrotarlos. El método para derrotar a los enemigos es arrojando cartas mágicas (como si fuesen proyectiles), las cuales tienen diferentes elementos y efectos, lo cual obligará al jugador a elegir la mejor carta al enfrentarse a un enemigo.

Con el fin de que la experiencia del jugador sea la mejor posible, se ha decidido implementar una interfaz gráfica agradable y efectos visuales, los cuales darán cierta profundidad al proyecto. Además de esto, se ha decidido implementar un sistema de mejoras permanentes, mediante el cual, al terminar una partida, el jugador retendrá el oro que tuviese al finalizar esa partida, el cual podrá usar para obtener esas mejoras en el menú del inicio del videojuego.

Descriptores

Videojuego, Blender, Unity, Videojuego de mazmorras, Generación procedural.

Índice

1.	Introducción	1
2.	Justificación y alcance	3
2.1	Justificación	3
2.2	Antecedentes y predecesores	3
2.2.1	The Binding of Isaac.....	4
2.2.2	Moonlighter.....	5
2.2.3	Hades.....	6
2.3	Objetivos y alcance.....	7
3.	Solución propuesta.....	11
4.	Plan de trabajo	19
4.1	Planificación temporal.....	19
4.2	Perfiles.....	25
4.3	Presupuesto.....	25
5.	Desarrollo de la solución	28
5.1	Especificación de requisitos	28
5.1.1	Requisitos funcionales.....	28
5.1.2	Requisitos no funcionales.....	29
5.2	Tecnologías utilizadas.....	30
5.2.1	Unity	30
5.2.2	Blender	31
5.2.3	GitHub	31
5.2.4	TeamGantt.....	31
5.2.5	Google Drive	31
5.2.6	Microsoft Word	32

5.3	Terminología de Unity	32
5.4	Diseño del videojuego	34
5.4.1	Diagrama de los GameObjects de la escena StartMenu	35
5.4.2	Diagrama de los GameObjects de la escena VideoGameScene	38
5.5	Desarrollo del videojuego	48
5.5.1	Modelos y animaciones.....	48
5.5.2	Materiales.....	62
5.5.3	Efectos de partículas	62
5.5.4	Sonido y volumen	63
5.5.5	Personaje principal	63
5.5.6	Enemigos	67
5.5.7	Cartas mágicas.....	70
5.5.8	Mejoras.....	71
5.5.9	Salas.....	72
5.5.10	Creador de la mazmorra.....	74
5.5.11	Sistema de guardado.....	79
5.5.12	Interfaz del usuario.....	79
6.	Manual de usuario.....	82
6.1	Iniciar el juego	82
6.2	Interfaz del juego.....	84
6.3	Controles	88
6.4	Enemigos	89
6.5	Entorno.....	92
7.	Valoración ética del proyecto.....	95
8.	Conclusiones y trabajo futuro	98

8.1	Discusión de los resultados obtenidos	98
8.2	Trabajo futuro	98
9.	Bibliografía.....	100

Índice de figuras

Figura 1: Gameplay del videojuego The Binding of Isaac.....	5
Figura 2: Gameplay del videojuego MoonLighter en la mazmorra.....	6
Figura 3: Gameplay del videojuego Moonlighter en la tienda.....	6
Figura 4: Gameplay del videojuego Hades.....	7
Figura 5: Relación entre elementos.....	16
Figura 6: Cronograma de la planificación del proyecto.....	24
Figura 7: Interfaz de Unity.....	30
Figura 8: Interfaz de Blender.....	31
Figura 9: Ejemplo de un GameObject del diagrama.....	34
Figura 10: Diagrama del Canvas StartMenuCanvas.....	36
Figura 11: Diagrama de los GameObjects encargados del efecto del pasillo.....	36
Figura 12: Diagrama del GameObject BackgroundMusic.....	37
Figura 13: Diagrama del GameObject SpawnSphere/DespawnSphere.....	38
Figura 14: Diagrama del GameObject InGameUpgrade.....	39
Figura 15: Diagrama del GameObject Portal.....	39
Figura 16: Diagrama del GameObject MagicCard.....	40
Figura 17: Diagrama del GameObject SphereAttack.....	41
Figura 18: Diagrama del GameObject Player.....	42
Figura 19: Diagrama del GameObject TotemEnemy/TotemBossEnemy.....	44
Figura 20: Diagrama del GameObject Room.....	46
Figura 21: Diagrama del GameObject DungeonCreator.....	47
Figura 22: Diagrama del GameplayCanvas.....	47
Figura 23: Cubo partido a la mitad.....	48
Figura 24: Aplicación del mirror al cubo.....	49

Figura 25: Realización de los cortes al cubo usando la herramienta de corte	49
Figura 26: Caras del cubo estiradas usando la herramienta de extrusión	50
Figura 27: Modelo de personaje con el componente subdivisión	50
Figura 28: Modelo del personaje con máscara	51
Figura 29: Material agregado al modelo	52
Figura 30: Marcas agregadas al modelo y realizado el unwrap	53
Figura 31: Guantes del personaje seleccionados para pintarlos.....	53
Figura 32: Agregado el esqueleto al modelo.....	54
Figura 33: Ventana Weight Panel.....	55
Figura 34: Postura del personaje modificada.....	56
Figura 35: Timeline con los fotogramas del personaje	56
Figura 36: Modelo del enemigo	57
Figura 37: Separación de los clips de los brazos del personaje.....	58
Figura 38: Separación de los clips del cuerpo del personaje	59
Figura 39: Separación de los clips del modelo del enemigo	60
Figura 40: Material usado para los modelos de los brazos y cuerpo del personaje	61
Figura 41: Efecto de partículas de la explosión de un ataque de fuego	62
Figura 42: Ejemplo de AudioSource y el Audio Mixer	63
Figura 43: Componentes del personaje principal	64
Figura 44: Animator de ambas partes del cuerpo y sus clips	64
Figura 45: Relación entre animaciones de los brazos del personaje	65
Figura 46: Relación entre animaciones del cuerpo del personaje	65
Figura 47: Character Controller del personaje	65
Figura 48: GameObject del personaje y sus componentes.....	66
Figura 49: La cámara y sus componentes	67

Figura 50: Enemigo tótem de fuego y sus componentes	68
Figura 51: Enemigo tótem de fuego y los componentes de cada parte del cubo.....	69
Figura 52: NavMeshAgent del enemigo	69
Figura 53: Relación entre animaciones del enemigo	70
Figura 54: Carta de fuego con todos sus componentes	71
Figura 55: Ejemplo de una mejora con todos sus componentes	72
Figura 56: Materiales de la sala.....	73
Figura 57: Ejemplo de una sala.....	73
Figura 58: Portal del videojuego y sus componentes.....	74
Figura 59: GameObject DungeonCreator con el script DungeonGenerator	74
Figura 60: Matriz de ejemplo después del primer paso.....	75
Figura 61: Matriz de ejemplo después del segundo paso	76
Figura 62: Matriz de ejemplo después de finalizar el último paso	76
Figura 63: Sala final de la mazmorra	77
Figura 64: Primera imagen de una mazmorra generada de manera procedimental.....	78
Figura 65: Segunda imagen de una mazmorra generada de manera procedimental.....	78
Figura 66: Tercera imagen de una mazmorra generada de manera procedimental	79
Figura 67: Diagrama de flujo del menú inicial.....	80
Figura 68: Diagrama de flujo del menú de pausa.....	81
Figura 69: Menú inicial	82
Figura 70: Menú de opciones del menú inicial	83
Figura 71: Menú de mejoras permanentes.....	83
Figura 72: Interfaz del personaje del jugador	84
Figura 73: Interfaz del usuario - Cantidad de vida	84
Figura 74: Interfaz del usuario - Cantidad de oro.....	84

Figura 75: Interfaz del usuario - Elementos y elemento agua seleccionado.....	85
Figura 76: Interfaz del usuario - Elementos y elemento fuego seleccionado	85
Figura 77: Interfaz del usuario - Elementos y elemento tierra seleccionado.....	85
Figura 78: Interfaz del usuario - Elementos y elemento rayo seleccionado	86
Figura 79: Menú de pausa	86
Figura 80: Opciones del menú de pausa	87
Figura 81: Mensaje de victoria	87
Figura 82: Interfaz de derrota	88
Figura 83: Esfera de aparición de enemigos y mejoras.....	89
Figura 84: Mejora del videojuego de la sala del mercader	90
Figura 85: Mejora del videojuego de la sala de mejora	90
Figura 86: Enemigo eléctrico en el centro y enemigo de tierra a la izquierda.....	91
Figura 87: Enemigo de fuego lanzando una esfera de fuego	91
Figura 88: Enemigo de agua	92
Figura 89: Jefe del piso	92
Figura 90: Ejemplo del interior de una sala.....	93
Figura 91: Mejora por derrotar al jefe del piso y portal de teletransporte	93
Figura 92: Sala final con la salida.....	94

Índice de tablas

Tabla 1: Tabla con las tareas a realizar y su duración	23
Tabla 2: Tabla de presupuestos para la empresa.....	27

1. INTRODUCCIÓN

Esta memoria representa todo el trabajo realizado durante estos últimos meses para el proyecto fin de grado, trabajo en el cual he aplicado todo el conocimiento que he adquirido tras estos cuatro años de aprendizaje en esta Universidad. En este documento se recogerá toda la información relacionada con el videojuego creado, desde los objetivos a cumplir con este trabajo hasta las conclusiones y resultados obtenidos.

Para facilitar la comprensión de este documento, se ha decidido dividirlo en una serie de capítulos para una mayor facilidad en la lectura. Los capítulos creados corresponden a los definidos en la normativa de los trabajos fin de grado proporcionada por la propia universidad, aunque a algunos de ellos se les han modificado los títulos y agregado varios subapartados adicionales, para proporcionar de esta manera una mejor explicación. Estos capítulos, descartando en el cual estamos actualmente, serían los correspondientes:

- **Justificación y alcance:** En este capítulo se redactará el motivo por el cual se ha elegido desarrollar este proyecto. Adicionalmente, se dará una serie de antecedentes para poder comprender el pasado de este tipo de videojuegos y que videojuegos han inspirado al que ha sido creado para este trabajo. Se finalizará este capítulo redactando que objetivos se pretenden alcanzar con la realización de este proyecto y que queda fuera del alcance.
- **Solución propuesta:** En este capítulo, se dará una definición más amplia del videojuego creado para que los lectores de esta memoria puedan visualizar el proyecto de manera más precisa y detallada, explicando las mecánicas principales y el sistema de juego.
- **Plan de trabajo:** A la hora de desarrollar cualquier proyecto informático, es importante contar con una planificación de las tareas a realizar para garantizar la correcta finalización de dicho proyecto. En la planificación adjuntada en este apartado, se detallará todas las tareas realizadas en el desarrollo del proyecto, indicando la duración y fecha de inicio de estas. También se hablará de que perfiles de trabajadores han sido necesarios para la realización de este proyecto. Para finalizar, se explicará el presupuesto obtenido para la creación del producto y cuál sería el presupuesto necesario en caso de que se quiera lanzar al mercado este videojuego.
- **Desarrollo de la solución:** Este capítulo contendrá toda la información recabada en el transcurso del desarrollo del proyecto. Se dará una explicación detallada de cuáles son los requisitos que ha de cumplir este proyecto para poder llegar a considerarse exitoso. Adicionalmente, se explicarán que herramientas han sido utilizadas en este trabajo fin de grado, tanto para el desarrollo del proyecto como para la creación de esta memoria. También se detallará la terminología a usar para comprender de mejor manera el desarrollo del proyecto. Para finalizar, se hablará del diseño y desarrollo del videojuego, donde estará redactado de manera detallada todo lo creado en el proceso de este trabajo fin de grado.
- **Manual de usuario:** En este apartado se proporcionará, a modo de guía para el usuario, todo lo que se puede hacer en el videojuego creado para este trabajo, que controles tiene y que función cumple cada uno de ellos.
- **Valoración ética del proyecto:** Cualquier proyecto o trabajo informático realizado tiene una serie de consecuencias arraigadas al desarrollo y publicación de este. Por ello es imprescindible

evaluar esas consecuencias desde un punto de vista no solo legal, sino también desde un punto de vista ético.

- Conclusiones y trabajo a futuro: En este penúltimo capítulo se expondrán las conclusiones y resultados obtenidos una vez el desarrollo del proyecto ha concluido. Adicionalmente, habrá un apartado en el que se redactará el futuro que tendrá este proyecto.
- Bibliografía: Finalizando con los capítulos, este recogerá todas las fuentes de información usadas a lo largo de este trabajo fin de grado, ya hayan sido usadas para la planificación del proyecto, para el desarrollo de este o si se ha usado para la creación de esta memoria.

2. JUSTIFICACIÓN Y ALCANCE

En este capítulo se abordará la justificación de por qué se ha optado por el desarrollo de un videojuego 3D de mazmorras con generación procedural basada en Unity como trabajo a realizar en el proyecto fin de grado. También se desarrollarán cuál ha sido la historia de este género de videojuegos y que videojuegos han servido de inspiración para crear el mío propio. Adicionalmente, se redactará los objetivos que se planean cumplir mediante el desarrollo de este proyecto y cuáles, debido a diferentes factores, quedan fuera del alcance de este trabajo.

2.1 JUSTIFICACIÓN

Este proyecto nació de mi pasión por la informática y, sobre todo, por los videojuegos. A lo largo de mi vida, la industria de los videojuegos me ha ayudado mucho en varios momentos importantes de la misma y ha definido como soy yo ahora. Es por ello por lo que he decidido realizar como trabajo de fin de grado el desarrollo de un videojuego como homenaje y agradecimiento a esta industria.

Además de esto, el videojuego a desarrollar pertenece a mi género favorito de videojuegos, no solo debido a preferencias personales, sino también debido a que los videojuegos de este género me han acompañado en todo tipo de momentos, tanto buenos como malos, ayudándome a superarlos.

El motivo de porque se ha decidido desarrollarlo en tres dimensiones es debido a que a día de la entrega de esta memoria no hay apenas videojuegos de esta temática que sean tridimensionales, por lo que considero que la realización de este proyecto puede llegar a motivar a otros desarrolladores a crear nuevos productos de este tipo.

Adicionalmente, considero que este proyecto es una buena manera de demostrar todo el conocimiento adquirido a lo largo de la carrera, poniéndolo en práctica en un tipo de proyecto el cual no ha sido comúnmente realizado en la carrera. No solo se usará el conocimiento más relacionado con la programación, sino también se usará el conocimiento relacionado con la planificación y el desarrollo de proyectos software.

2.2 ANTECEDENTES Y PREDECESORES

Hoy en día los videojuegos no son solo un simple pasatiempo como cualquier otro, ya que estos son capaces de generar un impacto significativo en la sociedad. Algunos videojuegos son capaces de hacer que los jugadores sientan todo tipo de emociones y otros videojuegos son capaces de dar mensajes muy profundos realizando críticas constructivas a problemas que suceden en la vida real. Un ejemplo podría ser el videojuego Persona 5 Royal [1], un JRPG por turnos el cual, entre otras muchas cosas, trata de manera profunda temas como pueden ser la corrupción y el abuso del poder. Este videojuego no solo plantea problemas sociales, sino también dilemas éticos. En un momento en la historia del videojuego, el antagonista principal le da dos opciones al jugador para elegir: aceptar el trato del antagonista, consistente en manipular la realidad misma y hacer que todas las personas del mundo vean instantáneamente sus deseos cumplidos y otorgarles, por ende, la mayor felicidad a cambio de

que ellos vivan en una mentira sin saberlo, o negarse haciendo que la realidad siga siendo la misma y que la humanidad viva de manera libre a cambio de no conseguir esa felicidad.

Al igual que pasa con los libros y con las películas, los videojuegos también tienen numerosos géneros, desde los videojuegos de acción hasta los videojuegos de miedo y terror. De todos los géneros de videojuegos que existen hasta la fecha, el videojuego a desarrollar pertenece al género de videojuegos de mazmorras. Este género tuvo sus orígenes con el videojuego *Rogue* en 1980 [2], cuyo objetivo era el de avanzar el máximo número de pisos posibles en una mazmorra infinita llena de enemigos. Este videojuego se le considera el padre de todos los videojuegos de mazmorras, también conocidos como *Roguelike*. Por ello, todos los videojuegos pertenecientes a ese género cuentan con características que el predecesor estableció como obligatorias. Esas características son las siguientes:

- Generación procedimental de mapas: La característica principal de estos videojuegos reside en la generación procedural de niveles del videojuego, y algunos de ellos también generan de manera procedural otros elementos como pueden ser los enemigos o las estadísticas de estos. Mediante este tipo de generación de mapas se consigue que cada partida que juegue el jugador se sienta única y diferente.
- Llegar lo más lejos posible: El objetivo de estos videojuegos es llegar lo más lejos que se pueda sin perder la partida o llegar a la salida de la mazmorra.
- Muerte permanente: Otra de las características que poseen los *roguelike* es el sistema de muerte permanente. Mediante este sistema, si un jugador pierde la partida, todo el progreso conseguido se pierde y tendrá que empezar desde el inicio.

A lo largo de los años los videojuegos de mazmorras poseían ambas características heredadas del padre, sin embargo, con el paso del tiempo surgió un nuevo género de videojuegos el cual heredaba todas las características de los videojuegos *roguelike* menos el sistema de muerte permanente y en su lugar tenía un sistema de progresión mediante el cual por cada partida realizada el jugador podía reducir la dificultad de sus siguientes partidas. A estos videojuegos se les conocen por el término *roguelite*. Al igual que pasa en la industria de los videojuegos, en esta memoria se referirá tanto a los videojuegos *roguelike* como a los *roguelite* como videojuegos de mazmorras.

En el mercado se pueden encontrar videojuegos de mazmorras de todo tipo, como pueden ser videojuegos *roguelite* de acción, videojuegos *roguelike* por turnos, videojuegos *roguelite* de ritmo o videojuegos *roguelike* de cartas. Se ha demostrado que la compatibilidad de este género con otros es muy amplia y que esas combinaciones dan resultados realmente sorprendentes e innovadores.

Para la realización de este proyecto se ha tenido en cuenta como inspiración varios videojuegos de este género. Estos videojuegos son los siguientes:

2.2.1 The Binding of Isaac

El videojuego 2D *The Binding of Isaac* [3], creado por dos personas Edmund McMillen y Florian Himsl, es el videojuego que más ha inspirado al videojuego creado para este trabajo, sobre todo por su

estructura de generación de niveles procedurales. Es uno de los videojuegos de mazmorras más famosos de todos, el cual ocasiono una gran revolución en la industria, creando una era en la que la mayoría de los videojuegos que salían al mercado eran de su mismo género.

En este videojuego el jugador controlará a Isaac, un niño el cual deberá de escapar de su madre bajando a través de una trampilla que hay en el sótano. En el camino hasta la salida, Isaac se tendrá que enfrentar a enemigos nacidos de sus mayores miedos y temores. En este trayecto Isaac irá obteniendo nuevos objetos y mejoras que lo ayudarán a escapar de ese sótano.



Figura 1: Gameplay del videojuego The Binding of Isaac

2.2.2 Moonlighter

Este videojuego 2D fue el primer videojuego desarrollado por una empresa española de videojuegos llamada Digital Sun y fue publicado el 5 de noviembre de 2018 [4]. Este videojuego, pese a no ser tan famoso como el resto de los videojuegos de este apartado, ha vendido más de dos millones de copias.

En este videojuego el jugador controlará a Will, un joven que trabaja como mercader por el día en su pequeña tienda y que por la noche explora mazmorras con el objetivo de hacerse con los tesoros que habitan en ellas y así poder venderlos en su tienda.

Este videojuego mezcla dos géneros: por el día el género principal será el de gestión de recursos, ya que el jugador tendrá que gestionar su tienda y sus productos, teniendo que vender sus productos y evitando que algún ladrón decida irse de la tienda con un producto sin pagarlo, y por la noche el género principal será de mazmorras, donde el jugador tendrá que derrotar a los enemigos de la mazmorra y conseguir el máximo número de tesoros que pueda sin caer derrotado.



Figura 2: Gameplay del videojuego Moonlighter en la mazmorra

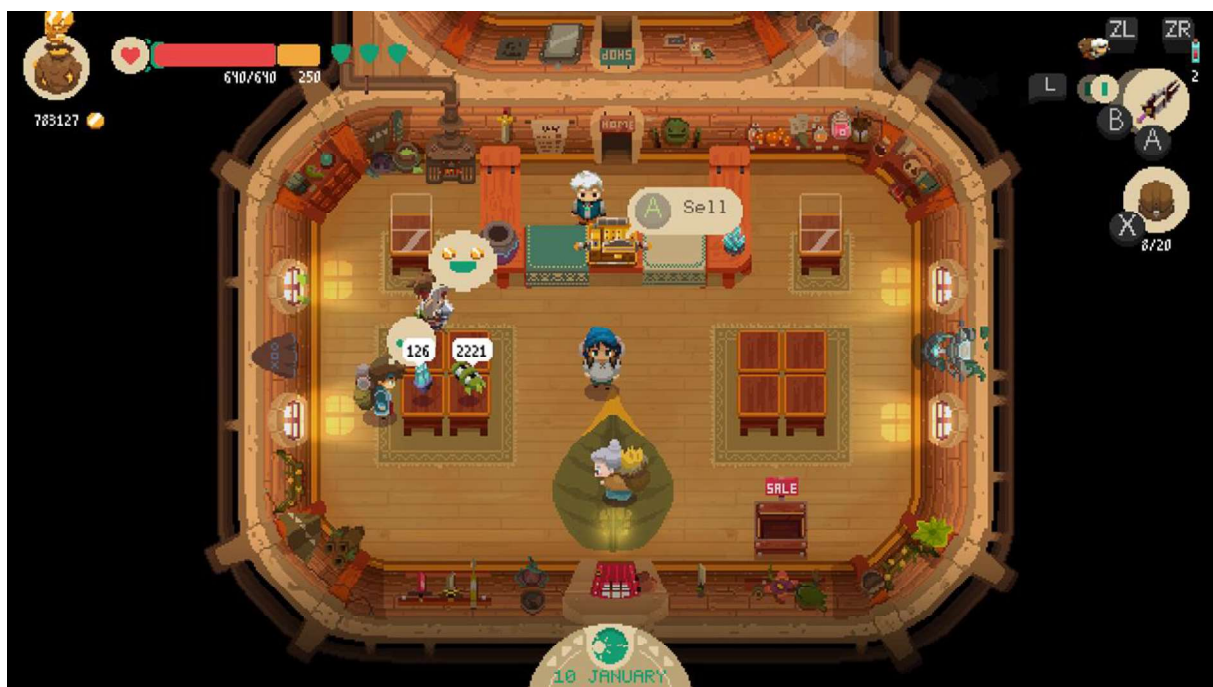


Figura 3: Gameplay del videojuego Moonlighter en la tienda

2.2.3 Hades

Hades [5] es un videojuego de mazmorras 2D desarrollado por Supergiant Games el cual gana varios premios de varias galas de videojuegos, entre los cuales destaca el premio al mejor videojuego independiente del año en la gala de los Game Awards 2020, gala la cual se considera como la gala de los Oscar de los videojuegos. Además, este juego cuenta con una gran historia, la cual ha sido nominada en varias galas a mejor historia, y en algunas de ellas ganando el premio.

En este videojuego el jugador controla a Zagreo, el hijo del dios Hades y por ende el príncipe del Inframundo, el cual quiere escapar de ese lugar. A lo largo de la aventura, otros dioses del olimpo irán ayudando a Zagreo, otorgándole bendiciones, las cuales agregarán efectos adicionales a sus ataques, los cuales usará para eliminar a los súbditos de su padre. Para evitar que Zagreo escape, el dios Hades modificará la estructura del inframundo para evitar que Zagreo pueda aprenderse el mapa de la mazmorra y escapar fácilmente.

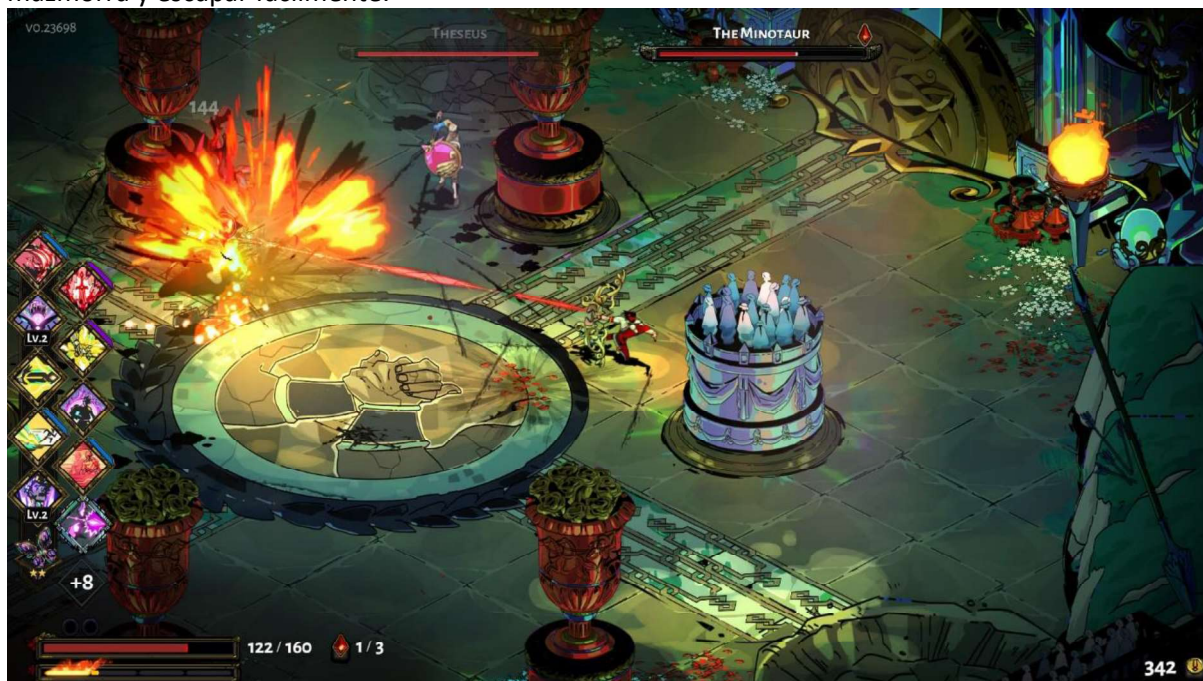


Figura 4: Gameplay del videojuego Hades

Este videojuego, con la mecánica de las mejoras de los dioses que dan diferentes efectos a los ataques de Zagreo, fue la que me dio la idea de crear diferentes cartas elementales con efectos diferentes entre ellas para el videojuego creado para este trabajo.

2.3 OBJETIVOS Y ALCANCE

El objetivo principal de este proyecto es desarrollar un videojuego de mazmorras generadas de manera procedimental, que además el videojuego sea tridimensional y que el jugador juegue en primera persona. Cabe resaltar que este proyecto será de un solo jugador, es decir que no podrá jugar con o contra otras personas. Para lograr el objetivo principal, se han creado varios objetivos basándose en el principal. La creación de estos objetivos nos servirá como métricas para poder identificar si el proyecto realizado ha sido un éxito o un fracaso. Los objetivos por cumplir serían los redactados a continuación:

- Objetivo 1. Desarrollar un videojuego lo suficientemente completo como para poder entrar en el mercado de los videojuegos.
- Objetivo 2. Implementar un sistema de combate de acción a tiempo real de disparos en primera persona.

- Objetivo 3. Implementar diferentes tipos de proyectiles, los cuales al impactar causen diferentes efectos donde hayan impactado.
- Objetivo 4. Diseñar variedad de enemigos con diferentes estadísticas y que cada uno dispare un tipo de proyectil diferente.
- Objetivo 5. Implementar variedad en la estructura del mapa de la mazmorra mediante la generación procedimental y que esta siga una serie de normas.
- Objetivo 6. Crear los modelos tridimensionales que serán usados en el videojuego.
- Objetivo 7. Crear animaciones para varios objetos y modelos del videojuego y enlazarlos con el código para que se realicen esas animaciones cuando suceda un evento en el código.
- Objetivo 8. Crear efectos de partículas para generar una mayor impresión en el jugador.
- Objetivo 9. Implementar métodos que consigan que el videojuego tenga rejugabilidad y progreso para incitar al jugador a seguir jugando.
- Objetivo 10. Implementar una IA que controle el movimiento de los enemigos y mediante la cual estos podrán perseguir al jugador de manera autónoma.
- Objetivo 11. Diseñar una interfaz de usuario y una estética del videojuego que sea cómoda para el jugador.
- Objetivo 12. Poder externalizar este proyecto en formato ejecutable a cualquier tipo de ordenador.

Los objetivos definidos anteriormente entran en el alcance de este proyecto, ya que no solo son necesarios para lograr el objetivo principal, sino que además son asequibles en el tiempo que se dispone para realizar este trabajo.

Sin embargo, existen otros objetivos que debido a que no son necesarios para lograr el principal objetivo o por otros motivos quedan fuera del alcance de este proyecto. Los objetivos descartados serían los listados a continuación:

- Realizar un sistema de guardado en la nube: El motivo por el que se ha optado por no realizar un sistema de guardado en la nube se debe a varias razones. Algunas de estas razones son que, en el caso de este videojuego, el tener un sistema de guardado en la nube y un sistema de guardado local no supone ninguna diferencia. Otra razón es el coste que esto supondría, ya que como se verá en el apartado del presupuesto, se ha optado por reducir el dinero necesario para el desarrollo del proyecto lo máximo posible.

- Implementar un modo multijugador: La razón por la cual se ha decidido que este videojuego será de un solo jugador, es que el hecho de hacerlo multijugador supone el alojamiento del videojuego en un servidor, lo cual, tal y como sucedía con el sistema de guardado en la nube, supondría un coste muy grande que afectaría al presupuesto. Además de eso, el modo multijugador de este videojuego sería muy difícil de diseñar, ya que de todos los modos multijugador solo habría dos que encajarían bien en el concepto de este videojuego. Uno de estos modos sería realizado mediante la implementación de un sistema de ranking en el que el jugador que haya tardado menos tiempo para escapar de la mazmorra estará en el como número uno en el ranking. El otro modo sería un modo multijugador cooperativo, en el que dos jugadores jugarían en la misma partida juntos. Aunque esto puede agregar cierta diferenciación respecto a los demás videojuegos de este género, también produce ciertos problemas en su diseño. El modo que usa el sistema de ranking se siente injusto, ya que como las mazmorras se generan de manera procedural, un jugador puede conseguir mejor tiempo que otro, no porque su habilidad en el videojuego sea mejor, sino que se puede deber a que ese jugador tuvo suerte con la generación de la mazmorra. En el caso del modo cooperativo estaría completamente desbalanceado, puesto que al ser dos jugadores podrán causar más daño a los enemigos. Debido a las razones anteriormente mencionadas, este objetivo queda fuera del alcance del proyecto.
- Crear una gran variedad de enemigos, mejoras y salas: Se ha decidido no centrarse en agregar mucha variedad de enemigos, mejoras y salas en el videojuego, debido a que, tal y como se mencionará en posteriores capítulos, todos los modelos han sido creados de manera manual y por un servidor. Si bien el código que contiene cada objeto es el mismo y por ende muy fácil de replicar y ajustar, la creación de los modelos visuales es algo que conlleva una gran cantidad de tiempo. Además de esto, se ha determinado que, aunque en este videojuego haya o bien cinco tipos de enemigos, o bien cien, no va a afectar en la calidad del proyecto. Debido a todo ello, se ha optado por realizar el mínimo número de modelos necesarios, pero que sean suficientes como para no crear un videojuego monótono.
- Tutorial interactivo para nuevos jugadores: Aunque es cierto que al empezar una nueva partida el nuevo jugador no se sabrá los controles del videojuego, se ha decidido a no crear un tutorial interactivo en el propio videojuego. En su lugar, y tal y como se mencionó en el capítulo 1, se ha agregado una guía de usuario en esta memoria, para que cualquier lector comprenda los controles y el funcionamiento del videojuego.
- Lanzamiento del producto al mercado: Si bien uno de los objetivos es conseguir desarrollar un videojuego lo suficientemente completo como para poder ser lanzado al mercado, se ha optado por no realizar la fase del lanzamiento al mercado. Esto es debido a que, para poder publicar este videojuego en el mercado de manera eficiente, ha de ser a través de empresas distribuidoras de videojuegos, las cuales tardan mucho en contestar debido a la gran cantidad de peticiones que les llegan al día, además de que la mayoría de ellas piden que el creador del videojuego le efectúe un pago inicial. Otro método podría ser subir el videojuego a una plataforma de distribución de videojuegos como lo puede ser Steam o EpicGames. En ambos casos, es necesario que el desarrollador realice un pago a la plataforma para que este pueda

publicarlo en la misma. Debido a que en ambos casos el hecho de implementarlo supondría un coste económico, se ha decidido no lanzar el videojuego al mercado.

- Creación de la música, efectos de sonido, las texturas y elementos de la interfaz gráfica: Debido a que este trabajo es un proyecto de fin de grado de la carrera de Ingeniería Informática más Transformación Digital de la Empresa, se ha decidido no crear de manera manual los elementos del videojuego que no tengan ninguna relación con la ingeniería informática. Estos elementos serían la música y efectos de sonido del juego, las texturas usadas en varios objetos del juego y los elementos que componen la interfaz gráfica. En vez de crearlos de manera manual, lo que se ha hecho ha sido usar material externo e implantarlo en el videojuego.

3. SOLUCIÓN PROPUESTA

En este capítulo se detallará de manera precisa la solución propuesta. Para ello se describirá de manera detallada la concepción y el enfoque dado, además de describir todas las mecánicas y características que posee el videojuego desarrollado.

El videojuego tridimensional desarrollado en Unity para ordenadores como parte de este trabajo fin de grado es un videojuego perteneciente al género de videojuegos de mazmorras. Como se comentó en el apartado 2.2, este tipo de videojuegos tiene como objetivo principal el intentar llegar al final de una mazmorra cuya estructura se genera de manera procedimental en cada partida. Debido a esta pseudoaleatoriedad se consigue que cada partida sea diferente a la anterior, debido a que en una nueva partida el mapa no será el mismo, al igual que tampoco lo será el resto de los elementos que conforman la mazmorra.

Las mazmorras que se generan procedimentalmente en este videojuego están compuestas por una serie de pisos. Cada uno de estos pisos tiene una serie de salas cuadradas con hasta cuatro posibles salidas, las cuales interconectan con otras salas adyacentes. Si el jugador ha entrado en una habitación en la que había un enemigo, se elevarán unos pilares los cuales bloquearán las salidas, dejando encerrados al enemigo y al jugador en la misma sala. Solo se volverá a abrir una vez el jugador haya derrotado a los enemigos de esa sala. Todas las salas de cada piso están interconectadas al menos con otra sala, por lo que el jugador podrá acceder a todas las salas de cada piso.

Las salas mencionadas anteriormente, si bien es cierto que tienen una misma apariencia física, cada una de ellas pueden ser de varios tipos, los cuales son:

- Sala inicial: Esta es la sala en la que el jugador reaparecerá al comenzar la partida y al subir a un nuevo piso. Esta sala es una sala segura y vacía donde no habrá ningún enemigo. La sala cuenta con 4 posibles salidas, una en cada lado de la sala para que el jugador tenga cuatro posibles rutas a escoger.
- Salas de combate: Al cruzar una sala de este tipo, las puertas que dan acceso a las salas adyacentes se bloquearán, y aparecerán enemigos, los cuales seguirán y atacarán al jugador con el objetivo de acabar con él. El jugador tendrá que derrotarlos para poder salir de esa sala, debido a que mientras haya un enemigo en esa sala, las puertas seguirán bloqueadas. Al derrotar a un enemigo, el jugador obtendrá oro, el cual tiene varios usos, los cuales se explicarán después. Una vez no haya ningún enemigo, las puertas que bloqueaban la sala se abrirán, permitiendo al jugador seguir buscando el camino hacia la salida. Una sala de combate completada no volverá a cerrar las puertas cuando se vuelva a entrar en ella. Esta sala puede aparecer en cualquier lugar del piso, siempre y cuando haya al menos una sala adyacente a esta.
- Salas de mejora: Al entrar en una de estas salas, aparecerá un objeto al cual se le ha denominado como “mejora”. Este objeto podrá ser recogido por el jugador y al hacerlo proporcionará un aumento en las estadísticas del personaje del jugador. El entrar en esta sala

nunca se cerrará las puertas de la sala, por lo que el jugador tiene total libertad en decidir si quiere recoger la mejora o abandonar la sala. En caso de abandonarla, la mejora no desaparece, por lo que el jugador puede volver para recogerla cuando quiera. Estas mejoras solo serán efectivas en la partida en la que se han recogido, por lo que al jugar una nueva partida las mejoras obtenidas en anteriores partidas no estarán presentes en el personaje del jugador. Este tipo de salas solo pueden aparecer en lugares donde solo haya exactamente una sala adyacente y que esa sala adyacente sea obligatoriamente una sala de combate. De esta manera se dificulta la búsqueda de este tipo de sala, ya que solo se puede acceder por una puerta. En cada piso siempre existirá una única sala de mejora.

- Sala del mercader: En esta sala aparecerá una mejora (como la que aparece en la sala de mejora) que el jugador podrá comprar con el oro obtenido al eliminar a los enemigos. Para obtener una mejora en esta sala es necesario poseer el oro que cuesta ese objeto, y en caso de no tenerlo, no se podrá obtener la mejora. Al igual que sucedía con la sala de mejora, los pilares no bloquearán la salida una vez se entre en esta sala, por lo que el jugador puede abandonarla libremente y volver cuando quiera. Esto da la opción al jugador de explorar más salas en busca de enemigos para conseguir el oro necesario y comprar la mejora. Además, tal y como pasa con la sala de mejora, en cada piso solo puede existir una única sala del mercader y ha de estar solamente conectada por una sala, siendo esta sala adyacente una sala de combate.
- Sala del jefe del piso: En esta sala se halla el jefe final del piso, el cual es un enemigo mucho más poderoso que el resto. Esta sala, al igual que las salas de combate, cerrará las puertas hasta que no se haya derrotado al jefe final del piso. Al derrotar a este enemigo, no solo proporcionará oro como el resto de los enemigos, sino que adicionalmente, aparecerá una mejora (como las que aparecen en la sala de mejora) como recompensa por haber derrotado al jefe del piso y también aparecerá un portal, el cual al cruzarlo teletransportará al jugador a la sala inicial del siguiente piso. Una vez cruzado este portal y habiendo sido teletransportado al siguiente piso, el jugador no podrá volver al piso anterior. Al cruzar el portal del último piso, el jugador será teletransportado a una habitación con una salida al exterior. Al cruzarla el jugador habrá ganado la partida. Las salas del jefe del piso también son únicas por piso y solamente puede estar interconectada por una sola sala de combate.
- Sala final: Esta sala consta es la última sala de la mazmorra, la cual tiene una salida que lleva al exterior y que al cruzarla el jugador habrá ganado la partida. En esta sala no aparecerán ni enemigos ni mejoras.

Habiendo explicado los tipos de salas que tendrá este videojuego, se redactará cuál es método usado para la generación procedural. Como se ha comentado, la mazmorra, que está compuesta por una serie de pisos que contienen una serie de salas interconectadas entre sí, se generará de manera procedimental, lo cual aportará cierto dinamismo y dificultad al videojuego. Cabe resaltar que no es lo mismo procedural que aleatorio, ya que lo que se genera de manera procedimental se crea siguiendo ciertos procedimientos los cuales intentan simular un comportamiento aleatorio. Sin embargo, si los datos de entrada que se le dan a un algoritmo procedural son siempre los mismos, el resultado también

lo será. De la misma manera, si al algoritmo encargado de la generación de la mazmorra se le pasan los mismos parámetros de, siempre se generará la misma mazmorra.

Las mazmorras creadas con este algoritmo pueden tener cuantos pisos se quiera tener (será a elección del desarrollador determinar cuántos pisos se quiere tener), y cada uno de ellos tendrá 12 salas, entre las cuales siempre habrá siempre una sala inicial, una sala de mejora, una sala de mercader y una sala de jefe del piso, siendo las restantes salas de combate. El algoritmo de generación procedural de la mazmorra también determinará que cantidad de enemigos, las estadísticas de estos y que tipo de enemigos aparecerán en cada sala de combate. También determinará que mejoras aparecerán tanto en la sala de mejoras como en la sala del mercader. De esta manera no solo se consigue aleatorizar la estructura de la mazmorra, sino también se aleatoriza el contenido de esta.

El jugador empezará el intento de huida en la sala inicial del piso más bajo de la mazmorra y su objetivo será intentar buscar la sala donde esté el jefe del piso y derrotarlo, consiguiendo así abrir el portal que lo llevará al siguiente piso. El jugador tendrá que repetir este proceso hasta que consiga derrotar al jefe del último piso y cruzar el portal que lo llevará a la sala final donde podrá escapar de la mazmorra. En el proceso para alcanzar la salida, el jugador irá explorando otras salas en las cuales irá obteniendo y/o comprando mejoras, las cuales aumentarán las características del jugador. También tendrá que luchar contra los enemigos que habitan las salas de combate y cuyo objetivo es derrotar al jugador.

A medida que se vaya avanzando pisos aparecerán enemigos con estadísticas cada vez mayores, por lo que el jugador tendrá que mejorar también sus estadísticas para poder enfrentarse a estos enemigos. Para ello, el jugador tendrá que conseguir mejoras en las salas de mejora y en las salas del mercader. Estas mejoras aumentan las estadísticas relacionadas con el combate, como puede ser la cantidad de vida o el poder de ataque del jugador, entre otras muchas más. Una vez se termine la partida, estas mejoras dejarán de tener efecto en el personaje del jugador.

La mayoría de los videojuegos de mazmorras suelen tener como característica principal “muerte permanente”, es decir, una vez que el personaje del jugador haya sido derrotado, el jugador no recibirá ninguna recompensa y tendrá que empezar una nueva partida desde cero. Sin embargo, el videojuego desarrollado en este proyecto tiene implementado un sistema de progresión para que, aunque el jugador haya perdido o ganado la partida, pueda conseguir el oro obtenido en ese intento.

Este oro obtenido al finalizar la partida se irá acumulando y con él se podrá comprar mejoras permanentes, las cuales estarán presentes en todas las partidas que el jugador juegue una vez las haya adquirido. Estas mejoras permanentes se podrán comprar en el menú inicial del videojuego al jugar una partida. Antes de comenzar la partida, el jugador verá varias mejoras que puede comprar por cierta cantidad de oro. En esta parte del menú, también podrá ver la cantidad de oro que tiene en ese momento el jugador. Una vez el jugador haya terminado de comprar mejoras, este puede empezar la partida. El oro que se usa en el menú principal no se pierde, pero este oro acumulado no se puede usar en mitad de la partida, ya que solo se puede usar para comprar mejoras permanentes. La información relacionada con el oro acumulado que tiene el jugador y las mejoras permanentes que posee se guardará en un fichero usando el sistema de autoguardado del videojuego.

Además de ser un videojuego de mazmorras, también es un videojuego de acción, puesto que en este videojuego es importante la coordinación entre el ojo y las manos, además de enfatizar mucho en la capacidad de reacción para esquivar los ataques de los enemigos. También es un videojuego de disparos en primera persona debido a que, aparte de que el videojuego se verá desde una perspectiva de primera persona, el jugador tendrá que realizar ataques a distancia para acabar con los enemigos que encuentre en las salas.

Los proyectiles que lanzará el jugador al realizar sus ataques a distancia serán lo que se ha denominado “cartas mágicas”. El jugador podrá lanzar estas cartas como si fuesen un proyectil. Cuando impactan en el cuerpo de los enemigos, estos recibirán daño en su vitalidad. Cuando la vitalidad de un enemigo sea lo suficientemente baja, este desaparecerá y dará al jugador oro de manera automática como recompensa por derrotarlo. Existen cuatro tipos de cartas: cartas de rayo, cartas de fuego, cartas de agua y cartas de tierra. El jugador contará desde el inicio de la partida con los cuatro tipos de cartas y podrá elegir cuál usar como proyectil principal en medio de la partida. Cada tipo de carta tiene un elemento y efecto especial diferente. Los enemigos a los que el jugador se enfrentará tienen ciertas resistencias y debilidades a cada uno de los cuatro tipos de elementos mencionados anteriormente, por lo que cada carta realizará un daño diferente a un enemigo en función de sus debilidades y resistencias.

A continuación, se detallará de mejor manera que efecto tiene cada tipo de carta:

- Fuego: En comparación con el resto de las cartas mágicas, estas no realizan mucho daño por contacto a los enemigos. Sin embargo, cuentan con un efecto especial llamado “quemadura” el cual se va acumulando en los enemigos aumentando su “nivel de quemadura”. Cuando el nivel de quemadura de un enemigo es lo suficientemente grande, este recibirá una gran cantidad de daño de manera instantánea. Este tipo de cartas están planteadas para aplicar mucha quemadura a los enemigos e infligirles una gran cantidad de daño por su efecto de daño instantáneo por quemadura.
- Agua: Estas cartas tienen un daño medio en comparación con el resto de las cartas y tienen un efecto especial llamado “congelación”. Al igual que pasaba con el efecto de las cartas de fuego llamado quemadura, las cartas de agua al impactar contra un enemigo aplicaran el efecto de congelación a este enemigo, aumentando su “nivel de congelación”. Cuando el nivel de congelación del enemigo sea lo suficientemente grande, la frecuencia a la cual ataca el enemigo aumentará realizando menos ataques por minuto. Este tipo de cartas están pensadas para ayudar al jugador a reducir la velocidad de ataque de los enemigos.
- Rayo: Las cartas de este elemento suelen tener un daño medio y cuentan con un efecto especial llamado “electrificación”. Este efecto sigue las mismas normas que los efectos quemadura y congelación de las cartas de fuego y agua respectivamente. Al acumular cierta cantidad de electrificación en el “nivel de electrificación” de un enemigo, los proyectiles que lance el enemigo se desplazarán más despacio. Este tipo de cartas están planteadas para facilitar al jugador el esquivar proyectiles enemigos.

- Tierra: Las cartas con el elemento tierra son las cartas que mayor daño causan. Sin embargo, estas no poseen ningún efecto especial como los otros tipos de cartas. La funcionalidad de esta carta consiste en realizar el máximo de daño posible a los enemigos.

Si bien es cierto que cada tipo de carta tiene ciertos diferentes efectos y diferentes atributos, hay un atributo que es común para todos, el cual es el daño que las cartas realizan al enemigo que golpean. Cada tipo de carta tiene un atributo que indica el elemento al que pertenecen y el daño elemental que tiene esa carta.

En el diseño y desarrollo se ha puesto mucho énfasis en que el jugador use determinados tipos de cartas en función de contra que enemigo se esté enfrentando. Por ello, se ha agregado algunos atributos tanto al jugador como a los enemigos.

En el caso del jugador, además de contar con sus atributos de poder de ataque, cantidad de vida, velocidad de ataque, tiempo de espera para volver a lanzar una carta y oro que tiene en ese momento, también cuenta con cuatro atributos relacionados con el daño elemental que los jugadores causan. Este atributo servirá para aumentar el daño realizado al utilizar ciertos tipos de cartas elementales. Por ejemplo, si el jugador tiene un daño elemental de fuego de 2, todo el daño que realice con cartas de fuego se multiplicará por 2.

En el caso del enemigo este además de su poder de ataque, la cantidad de vida que este tiene, el tiempo de espera para realizar un ataque y la velocidad de ataque, contará con cuatro atributos relacionados con la resistencia a cada elemento. Este atributo servirá tanto para que el enemigo reciba menos daño al recibir un impacto de una carta de un elemento específico, como para recibir más daño de una carta con un elemento específico. Por ejemplo, si la resistencia elemental al agua es 3, al recibir un ataque elemental este recibirá un tercio del daño que recibiría originalmente. De igual manera, si su resistencia al agua fuera 0.25, el enemigo recibiría cuatro veces más de daño por un ataque de agua.

En el diseño de este juego, se ha decidido que debe de haber una relación entre el elemento al cual pertenecen los enemigos y a que elementos son débiles y resistentes. Esta relación se puede representar gráficamente con la siguiente imagen:

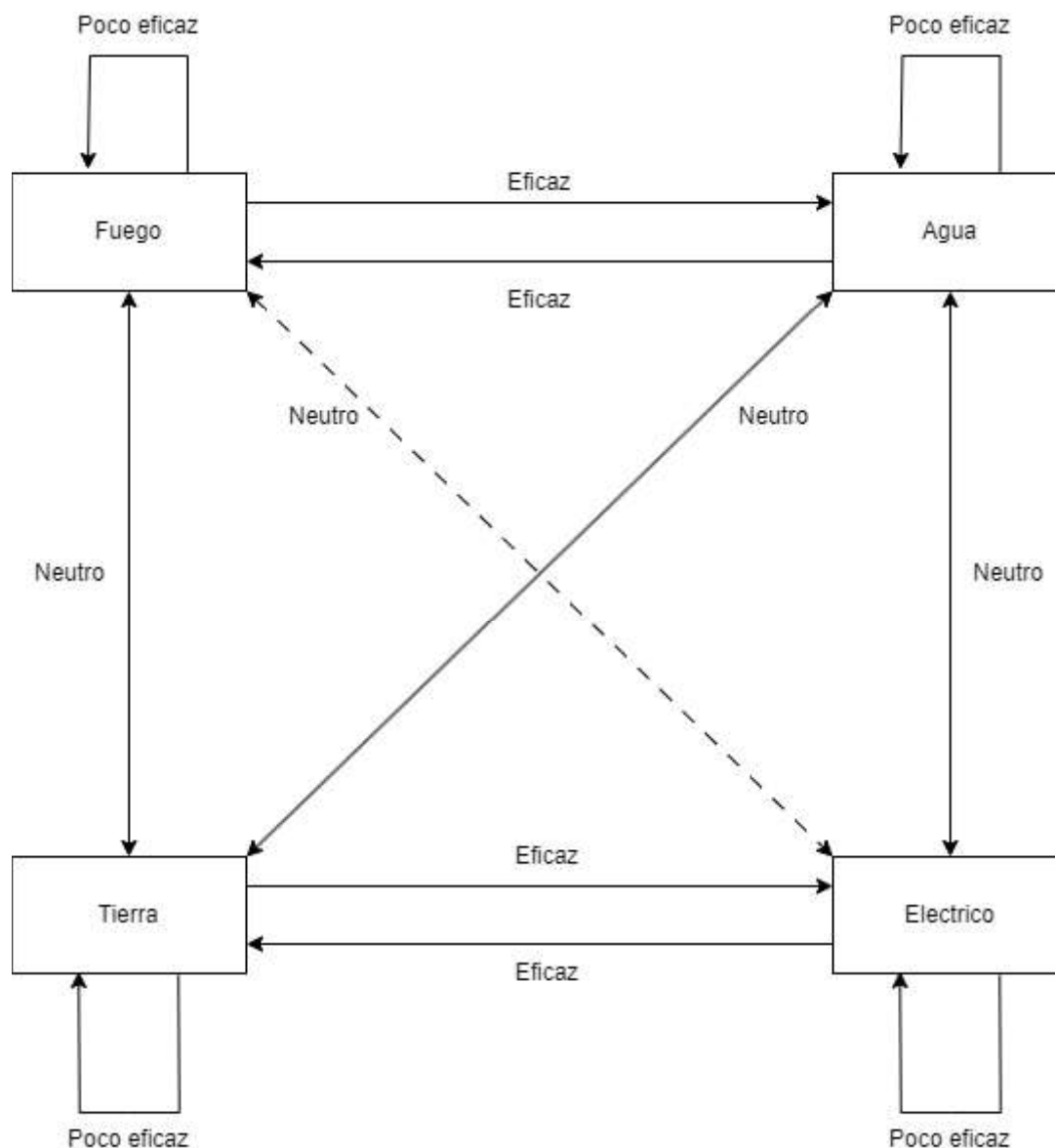


Figura 5: Relación entre elementos

Dada la imagen anterior, podemos observar que al realizar un ataque de agua contra un enemigo de fuego este recibirá más daño que el que recibiría originalmente, mientras que, si al mismo enemigo le realizamos un ataque de fuego, este recibirá menos daño. Y, por último, si realizamos un ataque de tierra o eléctrico a un enemigo de fuego le haremos el mismo daño que recibiría originalmente.

Para hacer que el jugador sea capaz de determinar a qué elemento pertenece el enemigo, se han creado a los enemigos de una manera mediante la cual el jugador pueda deducir a simple vista a qué elemento pertenece. Por ejemplo, uno de los enemigos de este videojuego está continuamente echando llamas a su alrededor, por lo que el jugador puede deducir que ese enemigo es de fuego. Cabe resaltar que los jefes de los pisos no pertenecen a ningún elemento, por lo que estos no tendrán ninguna debilidad elemental.

El cálculo de daño que recibe el enemigo varía en función los siguientes atributos: daño y daño elemental de la carta; daño y daño de elemento del jugador; y resistencia elemental del enemigo. En el apartado 5.4.2 se detalla cual es la fórmula empleada para calcular del daño que reciben los enemigos.

Para terminar con este apartado y a modo de conclusión, se destacará las principales diferencias que el videojuego desarrollado para este proyecto fin de grado tiene en comparación con la mayoría de los videojuegos del mismo género que hay en el mercado, las cuales son las siguientes:

- La primera diferencia es que este videojuego es tridimensional, característica la cual supone una gran ventaja contra la competencia, debido a que la gran mayoría de los videojuegos de mazmorras suelen ser de dos dimensiones. Además de ser tridimensional, la perspectiva del jugador será en primera persona, lo cual le aportará al jugador mayor inmersión en el videojuego.
- La segunda diferencia es que al terminar una partida el jugador recibirá el oro obtenido en esa partida, el cual conservará de forma permanente. Este oro se usará para comprar mejoras permanentes en el menú inicial y así mejorar las estadísticas del personaje del jugador, lo cual será útil para facilitar sus siguientes partidas. Usando este sistema, lo que se consigue es generar una progresión en el videojuego, al contrario que en la mayoría de los videojuegos de mazmorras, en los cuales al terminar una partida y empezar otra, empiezas desde cero, lo cual, pese a que obliga al jugador a mejorar mecánicamente en el juego a base de ensayo y error, puede generarle cierta frustración. Mediante este sistema de progresión no solo se evita el frustrar a los jugadores, sino que también motivamos a los jugadores a seguir jugando partidas aun habiendo ya ganado anteriormente una partida y alargar de esta manera el tiempo de vida del videojuego.
- La tercera característica diferenciadora sería que además de ser un videojuego de mazmorras, también es un videojuego de acción y de disparos en primera persona. Actualmente, existen muy pocos videojuegos que contengan estos géneros, por lo que es un mercado muy bueno para entrar. Aun haber pocos, sí que existen algunos videojuegos famosos que tienen estos géneros, como puede ser el videojuego Gunfire Reborn [6] el cual salió en el 18 de noviembre de 2021 en la tienda online de videojuegos Steam [7] y que a día de hoy su comunidad de jugadores sigue estando bastante activa. Se conoce por fuentes oficiales que este videojuego vendió más de un millón de copias en el 28 de noviembre de 2020 [8] y se sabe que en según unas estimaciones realizadas por servicios externos a Steam que la cantidad de ventas de este videojuego ronda entre los 1.71 millones y 2.57 millones de copias [9].
- La cuarta y última característica que nos diferencia con el resto de los títulos de este género sería el sistema del combate, concretamente todo lo referente con los elementos de las cartas y enemigos. Este es un sistema de combate el cual obliga al jugador a cambiar entre tipos de cartas en medio de la acción para poder realizar una mayor cantidad de daño al enemigo, aprovechándose de sus debilidades elementales. Además, cada tipo de carta es único, teniendo ciertos efectos especiales que ayudan en el combate. Otro de los puntos fuertes del

combate es su movilidad, ya que no solo se cuenta con el movimiento lateral, sino que también se puede realizar saltos y embestidas, las cuales son muy útiles para esquivar los ataques de los enemigos. La existencia de múltiples tipos de enemigos hace que el jugador tenga que cambiar su estrategia en mitad del combate. Todo esto hace que el sistema de combate de este videojuego sea bastante diferente al que tiene el resto de los videojuegos del género de mazmorras.

4. PLAN DE TRABAJO

En este capítulo se detallará la planificación realizada para este trabajo fin de grado, agregando un cronograma en el que se representará gráficamente las horas de trabajo. También se hablará de los perfiles de trabajadores necesarios para este trabajo fin de grado y cuales más serían necesarios para el lanzamiento del videojuego. Por último, se agregará el presupuesto usado para este proyecto y el que debería usarse en caso de querer sacar al mercado este videojuego.

4.1 PLANIFICACIÓN TEMPORAL

En este apartado se describirá las fases y tareas que se han realizado para completar con éxito este trabajo fin de grado. Este proyecto se divide en seis fases, cada una de ellas con tareas a completar. La planificación se ha realizado teniendo en mente que el desarrollo del proyecto seguiría una estructura similar a la que posee el modelo en cascada, es decir, hasta no finalizar todas las tareas de una fase no se avanzará a la siguiente. Una vez finalizadas todas las tareas de una fase, se avanzará a la siguiente, repitiendo este proceso hasta llegar a finalizar la última fase. Las tareas adjuntas a cada fase también siguen esta metodología.

Antes de mostrar la estructura del cronograma del proyecto se redactará cada una de las fases del proyecto con una lista de las tareas a completar en cada fase con una descripción clara y concisa:

Fase 1. Definición y registro:

Tarea 1. Definición y documentación de la idea del proyecto: Definir detalladamente la idea del proyecto y documentar toda la información relevante para su entendimiento.

Tarea 2. Reunión y presentación del proyecto al tutor: Presentar al tutor fin de grado la idea y determinar la viabilidad del proyecto.

Tarea 3. Registro del TFG: Realizar el registro de la idea en el formulario adjunto en la plataforma Alud.

Fase 2. Planificación:

Tarea 4. Definición de los objetivos: Definir detalladamente los objetivos a lograr con este proyecto, tanto el principal como los secundarios.

Tarea 5. Especificación de los requisitos: Definir de manera precisa los requisitos a efectuar para cumplir con los objetivos propuestos.

Tarea 6. Creación del plan de trabajo: Definir las tareas a realizar para cumplir los requisitos propuestos, indicando la duración que debería de tener cada una.

Fase 3. Diseño:

- Tarea 7. Diseño conceptual del videojuego: Realizar un documento en el que se detalle todos los aspectos tanto técnicos como artísticos que ha de tener el videojuego.
- Tarea 8. Investigación y recolección de tecnologías y herramientas: Investigar las mejores tecnologías y herramientas para usar en el proyecto y obtenerlas para el futuro desarrollo del videojuego.

Fase 4. Desarrollo:

- Tarea 9. Crear modelos 3D en Blender, sus animaciones e implementarlos en Unity: Diseñar los modelos 3D de algunos objetos en Blender, crearles animaciones y exportarlos e implementarlos en el proyecto de Unity.
- Tarea 10. Implementar el personaje jugador y programar sus funcionalidades: Implementar todos los componentes necesarios en el personaje del jugador y desarrollar el código que permitirá al personaje del jugador realizar todas sus acciones.
- Tarea 11. Implementar los componentes y programar las funcionalidades de los enemigos: Implementar en los enemigos los componentes necesarios y desarrollar el código que permitirá al enemigo realizar sus acciones.
- Tarea 12. Implementar los componentes y programar el código de las cartas mágicas: Implementar los componentes y programar las funcionalidades de las cartas mágicas.
- Tarea 13. Implementar los componentes y programar el código de las mejoras: Implementar en las mejoras los componentes necesarios y programar sus funcionalidades.
- Tarea 14. Diseñar y programar los tipos de salas: Diseñar la estructura de las salas y los tipos de salas, además de agregarle mediante código sus funcionalidades y los componentes necesarios.
- Tarea 15. Diseñar y programar el algoritmo de generación procedural: Diseñar y programar el funcionamiento del algoritmo de generación procedural de la mazmorra y agregar los componentes necesarios para su correcto funcionamiento.
- Tarea 16. Crear efectos de partículas: Fabricar los efectos de partículas y agregarlos en los objetos correspondientes.

Tarea 17. Crear y programar el sistema de progresión: Realizar el código que proporciona al videojuego una progresión mediante un sistema de mejoras permanentes para el personaje del jugador.

Tarea 18. Programar el sistema de guardado: Programar el sistema de guardado para realizar una copia de los datos del personaje del jugador.

Tarea 19. Crear la interfaz de usuario: Diseñar y crear los menús con los que el usuario va a interactuar.

Tarea 20. Crear diferentes escenas e implementar las tareas anteriores en las escenas correspondientes: Implementar los resultados obtenidos de las anteriores tareas en diferentes escenas para una mejor estructuración.

Tarea 21. Realización de pruebas: Comprobar que el correcto funcionamiento del videojuego mediante pruebas de verificación y validación.

Fase 5. Documentación:

Tarea 22. Realización de la memoria: Documentar en la memoria todo el trabajo anteriormente realizado en las fases anteriores de manera detallada.

Tarea 23. Revisión y corrección de la memoria con el tutor: Mostrar al tutor la memoria realizada y realizar las correcciones necesarias.

Fase 6. Finalización:

Tarea 24. Entrega de la memoria: Entregar este documento en la entrega del TFG en la plataforma de Alud y en la plataforma Deusto+.

Tarea 25. Creación de la demo para la defensa del proyecto: Crear una versión más pequeña del videojuego para poder mostrarla en la defensa del trabajo fin de grado.

Tarea 26. Preparación para la defensa del proyecto: Crear una presentación de soporte para la defensa y prepararse un guion con los puntos importantes a decir en la defensa del proyecto.

En la siguiente tabla se recogen las fases y tareas mencionadas anteriormente y su duración en horas:

Fase	Tarea	Horas
Fase 1	Tarea 1	6h
	Tarea 2	2h
	Tarea 3	2h
Fase 2	Tarea 4	4h
	Tarea 5	4h
	Tarea 6	4h
Fase 3	Tarea 7	6h
	Tarea 8	4h
Fase 4	Tarea 9	20h
	Tarea 10	20h
	Tarea 11	10h
	Tarea 12	16h
	Tarea 13	10h
	Tarea 14	38h
	Tarea 15	14h
	Tarea 16	12h
	Tarea 17	8h
	Tarea 18	12h
	Tarea 19	16h
	Tarea 20	8h
	Tarea 21	10h
Fase 5	Tarea 22	48h
	Tarea 23	16h

Fase	Tarea	Horas
Fase 6	Tarea 24	4h
	Tarea 25	6h
	Tarea 26	16h

Tabla 1: Tabla con las tareas a realizar y su duración

A continuación, en la siguiente figura muestra el cronograma del proyecto:

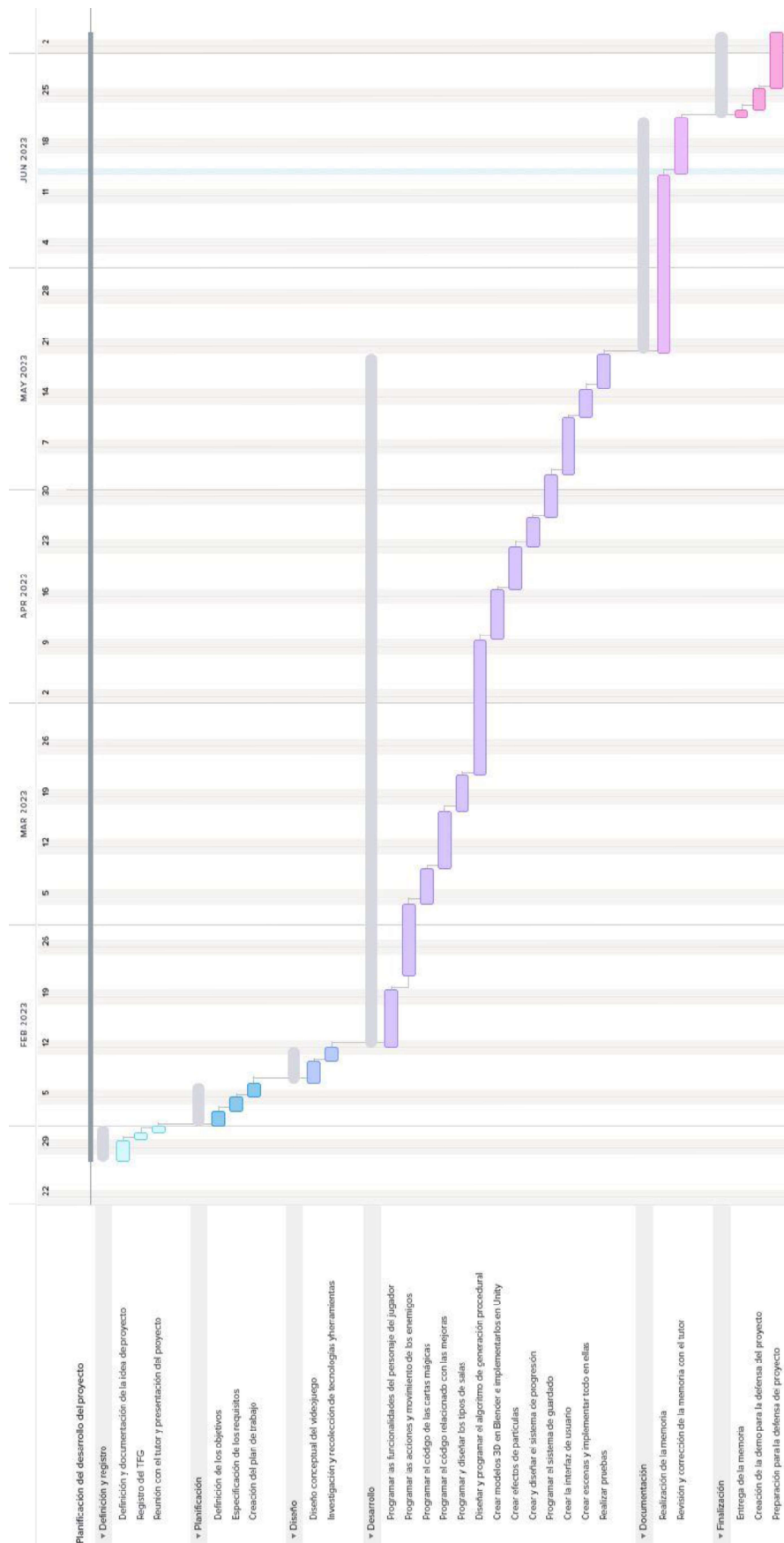


Figura 6: Cronograma de la planificación del proyecto

4.2 PERFILES

Pese a que para este proyecto yo he sido el único desarrollador de este (siendo yo el director del proyecto, el programador y el diseñador), debido al planteamiento que se le ha dado en el apartado 4.3, se ha decidido explicar en este apartado que perfiles se necesitarían para la realización de este videojuego en caso de que se quiera ofrecer posteriormente como producto a los usuarios. Estos perfiles son los siguientes:

- **Director de proyecto:** A la hora de crear un proyecto es importante tener a un director el cual evalúe la situación actual del proyecto y controle el flujo de trabajo.
- **Programador:** Para que el videojuego funcione es imprescindible tener a profesionales encargados de desarrollar la lógica del videojuego y de su correcto funcionamiento.
- **Diseñador:** Para que un videojuego se sienta vivo es necesario contar con modelos de las estructuras y de los personajes, y que estos tengan animaciones y texturas de calidad.
- **Músico:** La música es uno de los componentes que más valor aportan a un videojuego, ya que esta es la que ayuda a que el jugador se sumerja aún más en el videojuego.
- **Encargado del marketing:** Para que el videojuego se dé a conocer al mundo y tenga una buena recepción es necesario organizar una estrategia de marketing y lanzarla.

4.3 PRESUPUESTO

Para la realización de este proyecto no se ha usado ningún presupuesto, ya que todo componente del videojuego se ha creado con herramientas gratuitas o se han usado elementos externos gratuitos. Además de esto, el equipo de trabajo usado para realizar el videojuego es mi ordenador propio, por lo que tampoco se ha comprado nada de hardware. Debido a que se decidió no utilizar nada de dinero, se ha optado por usar música, texturas y efectos de sonido sin Copyright y de uso libre.

Por ello, en este apartado no se hablará del presupuesto empleado para este trabajo fin de grado, sino que se enfocará en el presupuesto que hubiera sido necesario en caso de que este proyecto fuese hecho como un trabajo en una empresa de desarrollo de videojuegos que quisiera sacar el videojuego al mercado. Este presupuesto se evaluará desde la perspectiva de la empresa para comprobar cuánto dinero se tendría que haberse gastado.

Para calcular ese presupuesto se tendría en cuenta varios factores que supondrían un coste para la empresa, los cuales son los listados a continuación:

- **Coste del programador y diseñador de los modelos:** En este trabajo fin de grado un servidor ha sido el encargado tanto de la programación como del desarrollo de los modelos y los efectos visuales. En esta evaluación del presupuesto esto seguirá siendo así, sin embargo, al estar

trabajando para una empresa, yo como trabajador he de recibir un sueldo en función de mis horas, lo que supone un coste para la empresa. Otra cosa que cabe resaltar es que para este proyecto se han invertido aproximadamente 320 horas, sin embargo, para tener un videojuego lo suficientemente pulido como para salir al mercado y que la empresa obtenga mayores beneficios, serían necesarias otras 320 horas, las cuales se usarían para mejorar y crear nuevo contenido, como puede ser el mejorar los modelos de los personajes, agregar nuevos tipos de ataques, crear nuevos modelos de enemigos, crear nuevas cartas con efectos diferentes y diseñar y programar nuevos tipos de salas entre otras cosas. Dicho eso, se realizó una investigación, se ha aproximado que el salario bruto que debería cobrar un desarrollador de videojuegos recién salido de la universidad y que se encargará de las tareas mencionadas anteriormente sería de unos 21.500 euros brutos. Teniendo en mente que se trabajaría a jornada completa, serían cuatro meses de trabajo, lo que equivaldría a 7166 euros que a la empresa le costaría mis servicios.

- Coste de los músicos: Como se mencionó en el apartado 2.3, para este videojuego se usó música externa sin Copyright. Sin embargo, ahora que en este planteamiento se tiene la intención de publicarlo, es necesario crear música propia, y para ello es necesario la contratación de músicos, los cuales tendrán que crear cinco composiciones de un minuto (ya que estas se reproducirán en bucle). Se usaría una canción para el menú de inicio, otra para el combate contra el jefe del piso, otra para cuando se esté explorando el resto del piso, una cuando se haya perdido la partida y otra cuando se haya ganado. Dado que cada compositor de música pide una cantidad diferente en función de sus preferencias y su nivel como músico, se ha realizado un estudio y se ha estimado que el coste medio sería de unos 1200 euros por las 5 canciones.
- Publicación en una plataforma de videojuegos: Se ha decidido que la plataforma de videojuegos a la cual se subirá el videojuego será Steam, ya que es la plataforma para videojuegos en ordenador más grande del mundo, por lo que llegaría a más público. El precio de subir un videojuego a esta plataforma sería de 100 euros, los cuales una vez el videojuego haya obtenido 1000 euros de beneficio se devolverán esos 100 euros a la empresa. Steam adicionalmente se llevará un porcentaje del dinero generado por las ventas.
- Ordenador de trabajo: Para este trabajo se realizó el proyecto en un ordenador MSI GT72 6QD Dominator G, el cual cuesta ahora mismo 1293 [10] euros aproximadamente. Aunque es cierto que la empresa proporcionaría un ordenador y que probablemente no sea un portátil sino un ordenador de sobremesa, debido a que Unity y Blender no tienen unos requisitos para su uso muy exigentes, la empresa puede comprar desde el más barato, el cual podrá usar ambas aplicaciones, o comprar un ordenador más caro para que el rendimiento de estas herramientas sea el mejor posible. Debido a la gran cantidad de opciones que tiene la empresa a la hora de comprar el material, se ha decidido que se tendrá en cuenta el ordenador usado para el trabajo fin de grado, el cual ha dado buenos resultados, aun siendo un ordenador de hace más de siete años.

Para este presupuesto no se ha tenido en cuenta los efectos de sonido, debido a que dependiendo de cómo lo plantee la empresa, estos sonidos se podrían llegar a hacer de manera casera y gratuita. Otra cosa que cabe mencionar, aunque no se tendrá en cuenta a la hora de hacer el presupuesto, son los costes variables. En este caso concreto, la empresa tendría que enfrentarse a un coste variable adicional, el cual es la tarifa que Unity pide a las empresas o autónomos que hayan ganado cierta cantidad alta de dinero de las ventas de videojuegos creados con el motor de Unity.

Dicho esto, en la siguiente tabla se puede observar el presupuesto necesario para llevar a cabo la realización completa del videojuego y su salida al mercado:

Nombre del coste	Dinero
Programados y diseñador	7.166,00 €
Músicos	1.200,00 €
Comisión de Steam	100,00 €
Ordenador	1.293,00 €
TOTAL	9.759,00 €

Tabla 2: Tabla de presupuestos para la empresa

El presupuesto que necesitaría la empresa encargada del desarrollo del videojuego sería de unos 9759 euros. Es importante resaltar que esto es una estimación aproximada de lo que costaría, ya que muchos de estos datos varían en función de lo que la empresa esté dispuesta a ofrecer a los músicos, además de que el ordenador a usar para la realización del videojuego puede ser diferente y por ende su coste puede ser menor o mayor. Además, en este presupuesto no se ha tenido en cuenta la campaña de marketing debido a que esta es variable en función de lo que la empresa decida.

5. DESARROLLO DE LA SOLUCIÓN

En este capítulo, se detallarán los requisitos que ha de cumplir este proyecto para lograr los objetivos mencionados anteriormente, que tecnologías se han usado tanto para la creación del proyecto como la de la memoria y se explicará la terminología más compleja para comprender mejor el diseño y el desarrollo del videojuego, los cuales se explicarán también en este capítulo.

5.1 ESPECIFICACIÓN DE REQUISITOS

En este apartado se definirán los requisitos que ha de tener este proyecto para así cumplir los objetivos mencionados en el apartado 2.3. Estos requisitos se dividen en requisitos funcionales y requisitos no funcionales.

5.1.1 Requisitos funcionales

- RF1. El jugador podrá mover su personaje por el mapa y la cámara de su personaje alrededor de este.
- RF2. El jugador podrá realizar acciones de esquivas como saltos o embestidas.
- RF3. El jugador contará con estadísticas, tanto las relacionadas con el combate como las relacionadas con los recursos del videojuego.
- RF4. El jugador podrá recoger y comprar mejoras que tenga a su alcance y que al hacerlo mejore sus estadísticas.
- RF5. El jugador podrá atacar lanzando cartas mágicas.
- RF6. Los enemigos han de contar con estadísticas de combate.
- RF7. Los enemigos deben de poder atacar a la dirección donde esté el jugador.
- RF8. Los enemigos deben de poder moverse siguiendo al jugador.
- RF9. Los enemigos han de poder aparecer en la sala de combate y desaparecer cuando sean eliminados, dando al jugador cierta cantidad de oro.
- RF10. Al derrotar al jefe del piso, aparecerá una mejora que el jugador podrá recoger y un portal, el cual al tocarlo teletransportará al jugador al siguiente piso (o a la sala final en caso de que ya no haya más pisos a los que subir).
- RF11. Las cartas mágicas deberán de poseer estadísticas para el cálculo de daño al enemigo.

- RF12. Las cartas mágicas han de poseer diferentes efectos en función de su tipo de elemento.
- RF13. Las mejoras han de poder modificar las estadísticas del jugador al obtenerlas.
- RF14. Las mejoras han de tener requisitos para poder permitir al jugador obtenerlas.
- RF15. La mazmorra deberá poseer diferentes tipos de salas.
- RF16. La mazmorra se deberá crear de manera procedimental al igual que los elementos que la componen.
- RF17. El videojuego deberá de poseer la capacidad de replicar continuamente una estructura determinada de una mazmorra mediante una “semilla”.
- RF18. El videojuego deberá tener un sistema de progresión mediante el cual el jugador podrá mejorar las estadísticas de su personaje.
- RF19. El videojuego deberá de tener un sistema de guardado automático de ciertas características del personaje del jugador.
- RF20. El usuario podrá interactuar con los menús del videojuego.
- RF21. El usuario podrá cambiar el audio del videojuego en el menú de opciones.
- RF22. El jugador podrá pausar la partida.
- RF23. El jugador podrá gestionar su partida desde el menú principal borrando la información guardada.
- RF24. El videojuego se dividirá en diferentes escenas para lograr una mayor optimización.
- RF25. El videojuego debe funcionar correctamente sin ningún tipo de error.

5.1.2 Requisitos no funcionales

A continuación, se nombrarán los requisitos no funcionales:

- RnF1. Habrá una interfaz de usuario al jugar una partida, la cual mostrará algunas estadísticas del jugador y todas las cartas que tiene disponible, marcando la carta mágica equipada en ese momento.

- RnF2. Los diseños de los elementos del videojuego han de ser visiblemente bonitos.
- RnF3. Los modelos de los enemigos y del personaje deberán de contar con animaciones al realizar sus acciones.
- RnF4. El diseño artístico de videojuego no debe desentonar en su conjunto.
- RnF5. El videojuego debe de tener efectos de partículas para generar mayor impacto en el jugador.
- RnF6. El videojuego ha de tener música y sonidos para una mayor inmersión.
- RnF7. Los menús que el usuario usará han de ser amigables y fáciles de usar.

5.2 TECNOLOGIAS UTILIZADAS

En este apartado se detallarán todas las tecnologías usadas para este proyecto, tanto para el desarrollo del videojuego, como para la creación de esta memoria.

5.2.1 Unity

Unity es un motor de videojuegos para la creación de videojuegos multiplataforma 2D, 3D y de realidad virtual. Se optó por este motor de videojuegos respecto a los demás existentes debido a varios factores, entre los cuales están la gran comunidad de desarrolladores que existe y la cual brinda ayuda entre sí y el fácil uso de este motor respecto a los demás ya existentes. En el proyecto se ha decidido usar la versión 2022.2.7f1.

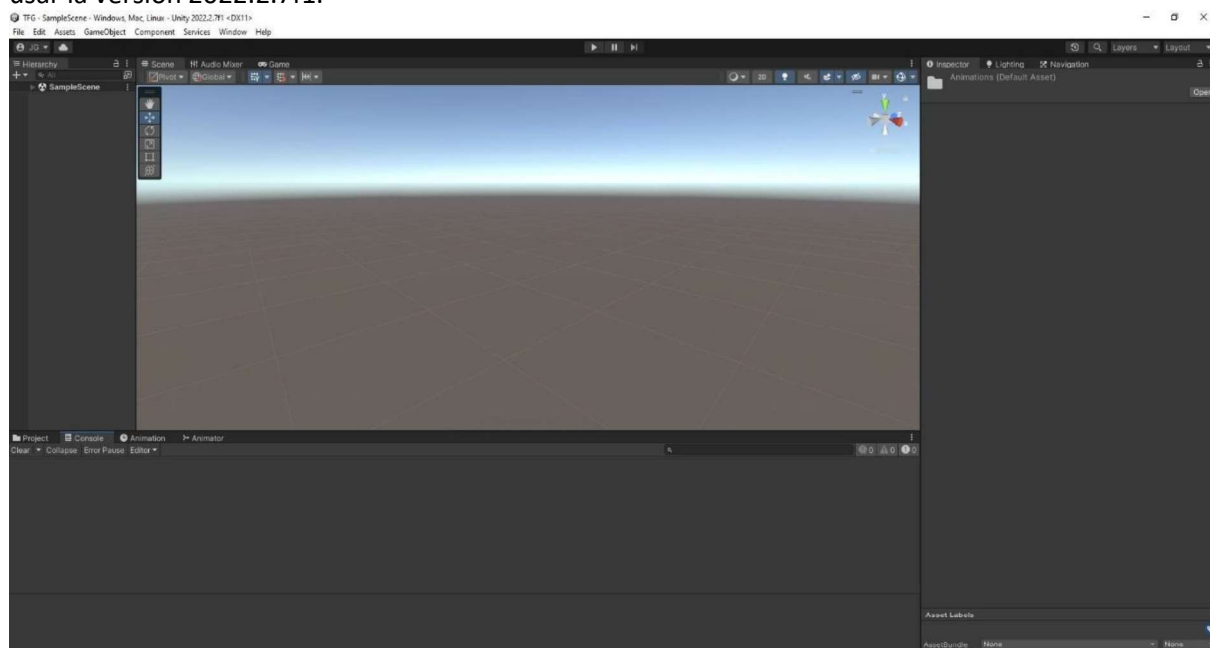


Figura 7: Interfaz de Unity

5.2.2 Blender

Blender es una herramienta software gratuita de modelado, animación y simulación 3D. En este proyecto se usará esta herramienta para la creación de los modelos de los enemigos y del personaje del jugador, agregando también animaciones a estos.

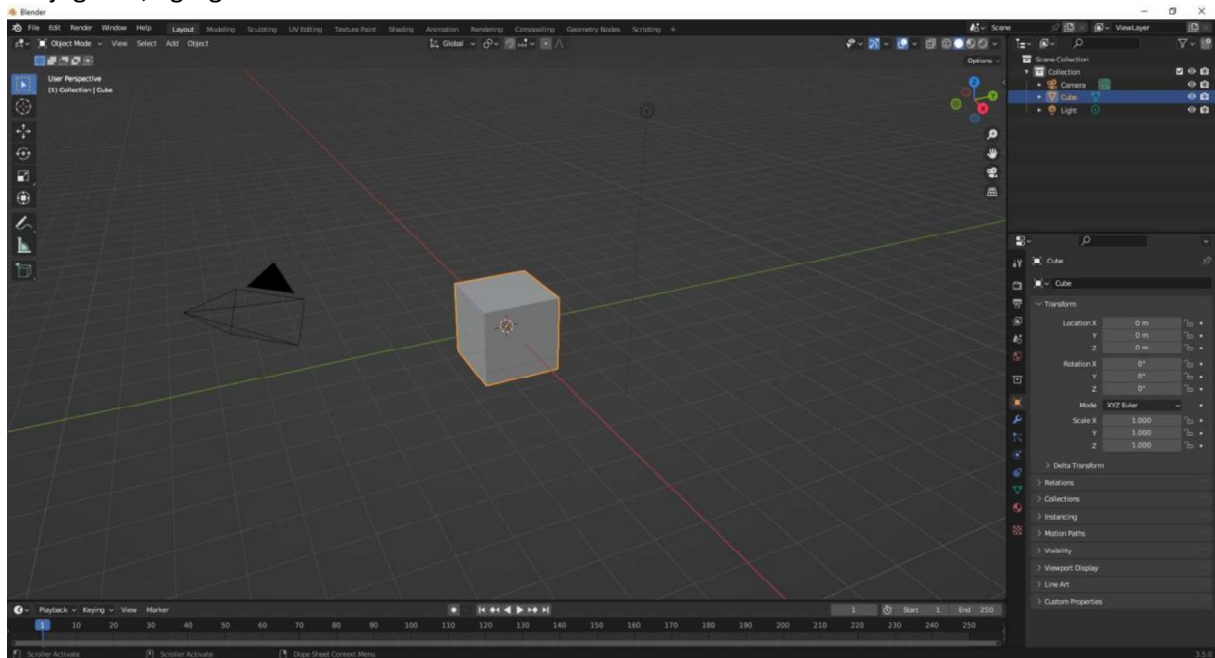


Figura 8: Interfaz de Blender

5.2.3 GitHub

GitHub es un repositorio de código remoto en el que se pueden almacenar multitud de proyectos y así tener una copia en la nube. En el proyecto fin de grado se ha usado GitHub para asegurarse de que en caso de que el proyecto almacenado localmente se corrompa o se borre, se pueda volver a cargar el proyecto desde la nube.

5.2.4 TeamGantt

TeamGantt es una herramienta software gratuita en la que se puede crear de manera simple diagramas Gantt. Esta herramienta ha sido usada para poder crear el cronograma de las actividades realizadas a lo largo de este proyecto, el cual está en el apartado 4.1.

5.2.5 Google Drive

El servicio de almacenamiento en la nube Google Drive ha sido usado principalmente para el almacenamiento de varios documentos y para la gestión de las tareas más técnicas usando las hojas de cálculo.

5.2.6 Microsoft Word

Para la creación de la memoria se ha optado por usar Microsoft Word debido a que ofrece muchas más facilidades para formatear el documento que las que puede ofrecer los documentos de Google.

5.3 TERMINOLOGIA DE UNITY

Con el objetivo de facilitar la comprensión de los apartados 5.4 y 5.5, se ha decidido crear este apartado, en el cual se explicará la terminología y los conceptos más importantes que se usan en Unity. A lo largo de los apartados mencionados anteriormente se irán mencionando en repetidas ocasiones ciertas palabras clave relacionadas con el entorno de Unity. Esas palabras clave son:

- **Escena:** Los proyectos de Unity están compuestos por escenas, las cuales son los espacios virtuales en los que se trabaja colocando los objetos, las luces, la cámara y otros elementos los cuales se representarán en el videojuego final. En Unity se pueden crear múltiples escenas, lo cual es muy útil para organizar el contenido y sus recursos [11]. Este proyecto tendrá dos escenas principales, la relacionada con el menú inicial llamada *StartMenu* y la otra que será la encargada del *gameplay* del videojuego llamada *VideoGameScene*.
- **GameObject:** En el entorno de Unity, todo elemento que este dentro de una escena es considerado un objeto, los cuales se les conocen como *GameObject* [12]. Estos suelen representar, por ejemplo, a personajes y partes del escenario. Para agregar funcionalidad a estos objetos se les suelen agregar componentes. Estos objetos cuentan también con varios atributos como puede ser el nombre y su etiqueta, entre otros muchos más atributos.
- **Prefab:** Los *Prefabs* [13] son *GameObjects* los cuales no están dentro de una escena, sino que están dentro de la carpeta llamada *Assets*. Estos *GameObjects* pueden ser reutilizados múltiples veces en la escena conservando los valores iniciales. Crear *Prefabs* es muy útil, sobre todo si se quiere crear copias de un mismo objeto, pero que esas copias sean independientes entre sí mismas.
- **Componentes:** Los componentes [14] son un conjunto de funcionalidades que permiten a los *GameObjects* la capacidad de realizar acciones. La mayoría de los componentes se pueden modificar no solo desde la interfaz de Unity, sino también mediante código.
- **Camera:** Al agregar este componente a un objeto permite al jugador poder observar el entorno del videojuego por la pantalla [15]. Este componente está siempre predeterminado en los *GameObject* llamados *Main Camera*.
- **Post-Process Layer y Post-Process Volume:** Estos componentes provienen de un paquete de Unity llamado *Post Processing* [16]. Mediante estos dos componentes se puede crear agregar varios efectos visuales, como puede ser el efecto de profundidad, mediante el cual la cámara verá el fondo desenfocado. El *Post-process Layer* es el componente que se le agrega a la

cámara para que cuando detecte alguna colisión con algún otro *GameObject* que tenga el componente *Post-Process Volume*, se aplique el efecto que tenga en ese momento el *Post-Process Volume*.

- **Material:** El material [17] de un objeto es aquel que configura la apariencia visual que tendrá un objeto. Esta configuración va desde cambiar los colores hasta agregarle una gamma metálica al objeto. Si se quiere también se puede usar una imagen (a la cual se la suele referir como textura) y agregársela al material y de esta manera conseguir materiales mucho más realistas.
- **Mesh Filter y Mesh Renderer:** El *Mesh Renderer* [18] es el encargado de dibujar los materiales y las texturas de la mayoría de los objetos 3D de Unity (los modelos de Blender no necesitan estos componentes) y un *Mesh Filter* [19] es el encargado de definir la estructura geométrica del objeto.
- **Transform:** El *Transform* [20] es un componente que, salvo ciertos casos, todo *GameObject* tiene. Este componente se encarga de establecer la posición, la rotación y la escala del objeto al cual está agregado este componente.
- **Rigidbody:** El *Rigidbody* [21] es el componente principal que agrega las leyes físicas a un objeto, como puede ser la gravedad, su masa, etc.
- **Collider:** El componente *Collider* [22] se encarga de determinar la forma del objeto en relación con las colisiones físicas. El *Collider* puede tener una forma completamente diferente a la que tiene el objeto, por ejemplo, un *GameObject* que sea cuadrado puede tener un *Collider* que sea esférico.
- **Character Controller:** Este componente es capaz de mover un objeto y controlar las colisiones que sufre el mismo sin necesidad de un *Collider* ni de un *Rigidbody* [23]. Este objeto es útil porque permite modificar de manera más simple el comportamiento de las físicas del *GameObject*.
- **Animator:** El *Animator* [24] es el componente es el encargado de asignar a nuestro *GameObject* las animaciones. Usando una ventana de Unity llamada *Animator Controller*, se puede crear animaciones almacenando en una franja de tiempo varios fotogramas en los que se van cambiando datos del *GameObject* como puede ser la posición, rotación, etc. Además, si se ha creado animaciones usando Blender, se pueden exportar los fotogramas de esas animaciones de Blender al *Animator Controller*.
- **NavMesh:** *NavMesh* [25] es una estructura de datos que describe la superficie por la cual un objeto con el componente *NavMeshAgent* puede caminar y que obstáculos con el componente *NavMeshObstacle* no puede cruzar. La superficie por la cual un agente puede caminar ha de tener el componente *NavMeshSurface*. Mediante el *NavMesh*, los objetos que tengan el

NavMeshAgent podrán ser controlados por IA e irán al destino que se les fije. Esto es muy útil y usado, por ejemplo, para proporcionar a los enemigos la capacidad de perseguir al jugador de manera eficiente.

- **Unity Particle System:** El *Particle System* [26] de Unity es el encargado de generar pequeñas imágenes un gran número de veces para generar efectos visuales vistosos. Mediante este sistema se puede crear fogatas o humo de manera realista. Este sistema de partículas se le puede agregar a los objetos como si fuese un componente.
- **Audio Source, Listener y Mixer:** El *Audio Source* [27] es el encargado de reproducir el audio que esté adjuntado a este componente. El audio reproducido será escuchado por cualquier objeto que tenga el componente *Audio Listener*. El volumen de los audios reproducidos por los *Audio Source* se pueden controlar desde estos mismos o desde el *Audio Mixer* al cual este adjunto el *Audio Source*. El *Audio Mixer* se suele usar como controlador de volumen del videojuego.
- **Script:** Si bien es cierto que gracias a los componentes proporcionados por Unity se pueden agregar muchas funcionalidades a los *GameObjects*, a veces es necesario crear nuevas funcionalidades, las cuales no existen en ningún otro componente. Para ello, Unity da la capacidad de crear scripts [28]. Los scripts son clases que pueden estar escritas en el lenguaje de programación C# o en UnityScript (en el proyecto se ha escrito usando el lenguaje de programación C#). Mediante estos scripts se puede desde agregar funcionalidades a un *GameObject*, hasta controlar todo el flujo del videojuego.
- **Canvas:** El *Canvas* [29] es el área donde todos los elementos de la interfaz de usuario deben de estar. En este objeto se puede agregar cualquier elemento UI, como puede ser imágenes, botones, textos, etc.

5.4 DISEÑO DEL VIDEOJUEGO

Antes de comenzar con el desarrollo del videojuego, se creó varios diagramas mediante los cuales se explicarían de que están compuestos los *GameObjects*. Cabe resaltar que, si bien el diagrama creado se basa en los diagramas UML, se le han creado unas modificaciones debido a que este proyecto al estar realizado en Unity iba a estar centrado en *GameObjects* más que en clases. Dicho esto, en la siguiente imagen aparecerá un poco los cambios realizados frente a UML para que ahora en vez de ser un diagrama de clases sea un diagrama de *GameObjects*:

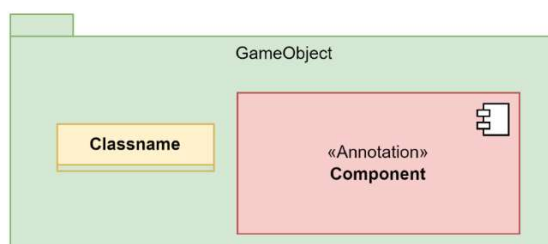


Figura 9: Ejemplo de un *GameObject* del diagrama

En esta imagen aparece como se representarán a lo largo de este documento los *GameObjects*. Los *GameObjects* y *Canvas* de este proyecto se representarán de color verde, los componentes que poseen los *GameObjects*, como pueden ser el *Rigidbody* o el *Collider* (componentes mencionados en el apartado 5.3) estarán de color rojo, y las clases (que en este caso son los scripts), se colocarán de color amarillo y sin poner “.cs”. Otra de las cosas a resaltar es que en algunos casos un *GameObject* puede estar dentro de otro. Esto se ha hecho así para respetar la estructura que se creó en Unity. En caso de que eso pase, el componente que estuviese dentro se colocaría color azul. Si hubiera tres *GameObjects* uno dentro de otro, se colocaría el principal en verde, el hijo en azul y el hijo del hijo en verde.

Explicadas las diferencias de nuestro diagrama respecto al diagrama de clases UML, procederemos a representar el diseño del videojuego presentando las clases y componentes que tendrán nuestros *GameObjects*, que hacen estas clases y cómo estas clases y componentes se relacionan entre sí. En este apartado no se explicará que hacen los componentes, ya que eso está explicado en el apartado 5.3. Como el videojuego se ha hecho en dos escenas, siendo una la del menú inicial y otra la del gameplay del videojuego, se ha decidido que primero se explicará que *GameObjects* componen la escena del inicio (llamada *StartMenu*) y luego que componentes componen la escena del gameplay del videojuego (llamada *VideoGameScene*).

5.4.1 Diagrama de los GameObjects de la escena StartMenu

El primer *GameObject* que se diseñó para esta escena es el *Canvas* llamado *StartMenuCanvas* que tendrá el menú inicial con el script *StartMenuController*, el cual podrá borrar los datos guardados y jugar una nueva partida, mantener los datos y jugar una nueva partida, obtener mejoras permanentes usando oro el cual habrá obtenido de anteriores partidas, abrir el menú de opciones y en el modificar el volumen del juego, y por último cerrar el videojuego. Para que el menú sea más cómodo para el jugador, se ha usado el componente *Animator*, mediante el cual se harán transiciones como fundidos de pantalla en negro cuando se salga del juego o las transiciones del apartado principal del menú al de opciones. También hará uso del script *SaveSystem* para borrar o guardar los datos del personaje del jugador, como pueden ser las mejoras adquiridas en este menú o el oro que tiene acumulado. Para que se pueda modificar el volumen del videojuego se ha usado un *Audio Mixer*, el cual modificara el volumen de todos los sonidos y músicas del videojuego. En la imagen Figura 10 se puede observar cómo quedó su diseño:

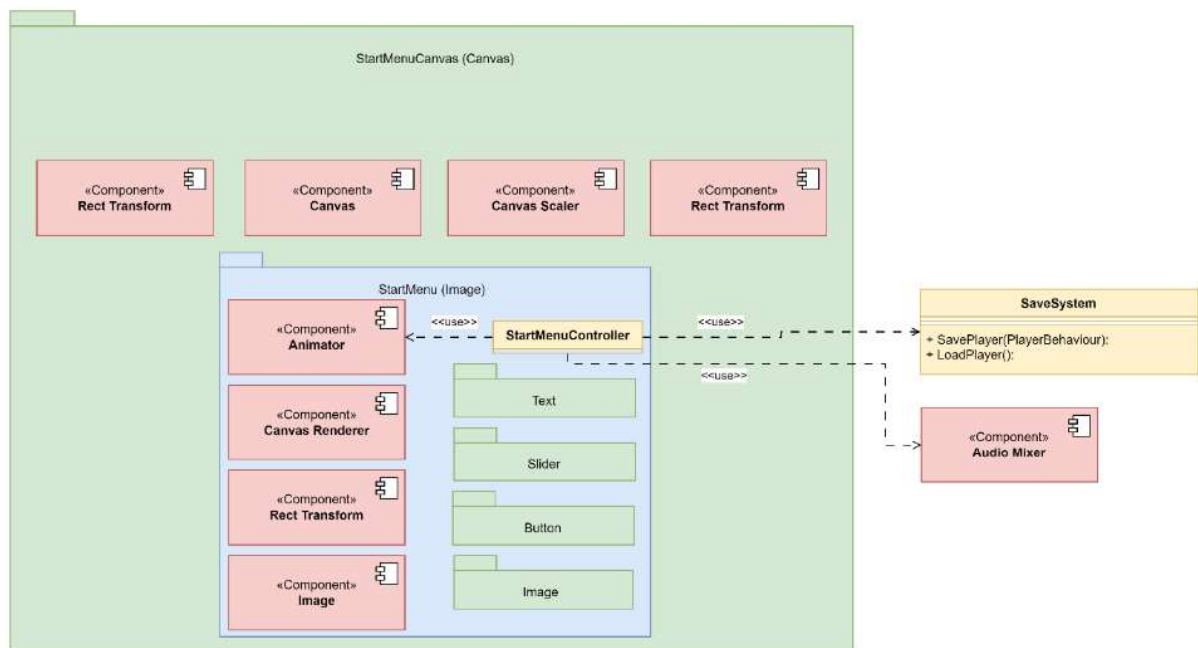


Figura 10: Diagrama del Canvas StartMenuCanvas

Para generar el efecto visual de que en el menú la cámara va avanzando por un pasillo oscuro, lo que se hizo fue crear varios *GameObjects* que ayudarían para crear este efecto, los cuales son los representados en la Figura 11.

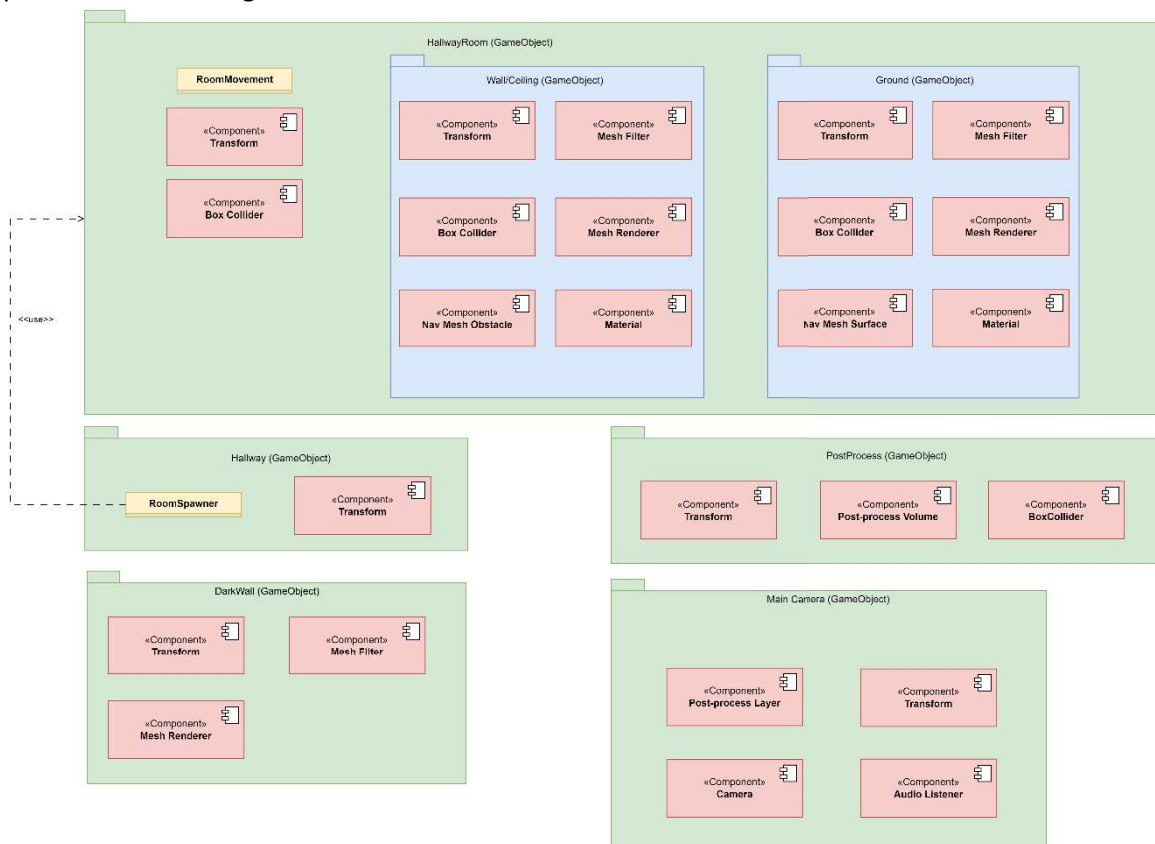


Figura 11: Diagrama de los GameObjects encargados del efecto del pasillo

Mediante los *GameObjects* que aparecen en la Figura 11 se ha podido crear el efecto visual de que se está pasando por un pasillo. A continuación, se describirá el papel que tiene cada *GameObject*:

- **GameObject HallwayRoom:** Este *GameObject* representa las salas que componen el pasillo. Este *GameObject* está compuesto por paredes, techo y suelo (los cuales son los *GameObjects* adjuntos). Además de esos objetos, éste tiene un script llamado *RoomMovement*, el cual será el encargado de mover la sala hacia la cámara. Este script también es el encargado de que en cuanto la sala haya alcanzado cierta distancia se destruirá (en la posición en la que se borra la sala está detrás de la cámara, por lo que no se verá cuando se borra). Se decidió en el diseño que era mucho mejor idea que en vez de mover la cámara se moviese las salas, ya que de la otra manera podría dar problemas si se mantiene mucho tiempo el menú abierto debido a que la cámara no pararía de avanzar. La destrucción de las salas también ayuda al rendimiento, ya que, si se dejase el videojuego mucho tiempo abierto en esta escena, se crearían miles de salas, todas ellas moviéndose y consumiendo recursos.
- **GameObject HallWay:** Este *GameObject* es el encargado de generar al inicio del videojuego varios *HallwayRoom* para generar el pasillo e ir generando continuamente salas para continuar con ese pasillo mediante el script *RoomSpawner*.
- **GameObject DarkWall:** Para que el pasillo tuviera un fondo oscuro, una de las cosas que se hicieron fue colocar una pared de color negro para que pareciese que el fondo estaba oscuro.
- **GameObject PostProcess:** La otra cosa que se hizo para generar la impresión de que la cámara se estaba adentrando por un pasillo oscuro fue crear un objeto vacío llamado *PostProcess* con un *Post-process Volume*, el cual se ha modificado para que agregue un efecto de oscuridad a la cámara. Para que este objeto afecte a la cámara se ha agregado el componente *Box Collider* y que al estar ambos en colisión ese efecto visual se aplicará a la cámara.
- **GameObject Main Camera:** Esta cámara contiene los componentes de cualquier cámara y adicionalmente tiene el componente *Post-process Layer* mediante el cual se podrá aplicar el efecto generado por el *Post-Process Volume* del objeto *PostProcess*.

Para terminar con esta escena hablaremos brevemente del objeto encargado del sonido, el cual es el *BackgroundMusic*. Este objeto es el encargado de reproducir la música del videojuego, y su volumen, al igual que el resto de los objetos que tengan audio de este videojuego, es controlado por el *Audio Mixer* tal y como se ve en su diagrama representa en la siguiente figura:

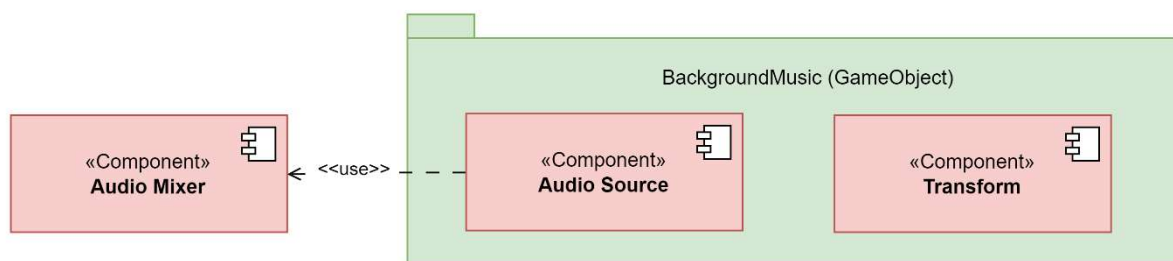


Figura 12: Diagrama del *GameObject BackgroundMusic*

5.4.2 Diagrama de los GameObjects de la escena VideoGameScene

Una vez hemos terminado de explicar los objetos que compondrán la escena del menú de inicio, comenzaremos con la segunda escena, la cual es la encargada del gameplay del videojuego. Como en esta escena los *GameObjects* se relacionan mucho más entre sí en comparación a los de la anterior escena, se empezará desde los objetos más simples y con menos relaciones para una mejora comprensión del apartado. Además, no se volverá a explicar los *GameObjects* que estén repetidos en ambas escenas, aunque sí que se mencionarán. Uno de los objetos que se repiten sería el *GameObject* *BackgroundMusic* que tiene la misma estructura que tenía en la escena *StartMenu*.

Dicho esto, empezaremos con el diagrama del *GameObject* llamado *SpawnSphere/DespawnSphere*, el cual es el encargado de enmascarar la aparición y desaparición espontánea del resto de *GameObjects* como pueden ser los enemigos o las mejoras. Para ello lo que se hace es mediante el componente *Animator*, hace que una esfera con un tamaño minúsculo crezca hasta llegar a un determinado tamaño (al llegar a este tamaño el objeto a enmascarar su aparición aparecerá dentro de la propia esfera, por lo que no se verá que aparece de repente) y una vez haya alcanzado el tamaño máximo establecido volverá a hacerse pequeña poco a poco hasta que su tamaño sea cero y en ese momento se destruirá.

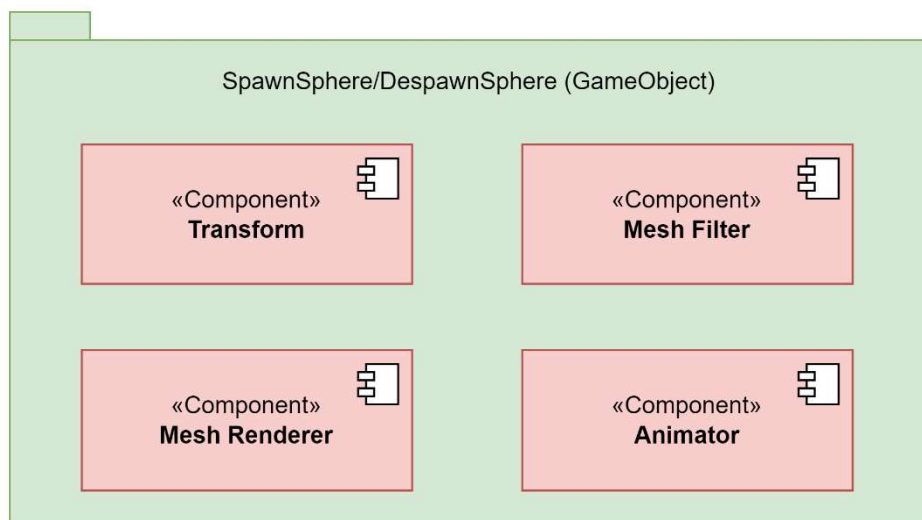


Figura 13: Diagrama del *GameObject* *SpawnSphere/DespawnSphere*

A continuación, se describirá el diagrama del *GameObject* *InGameUpgrade*, el cual tiene la siguiente estructura definida por la imagen Figura 14:

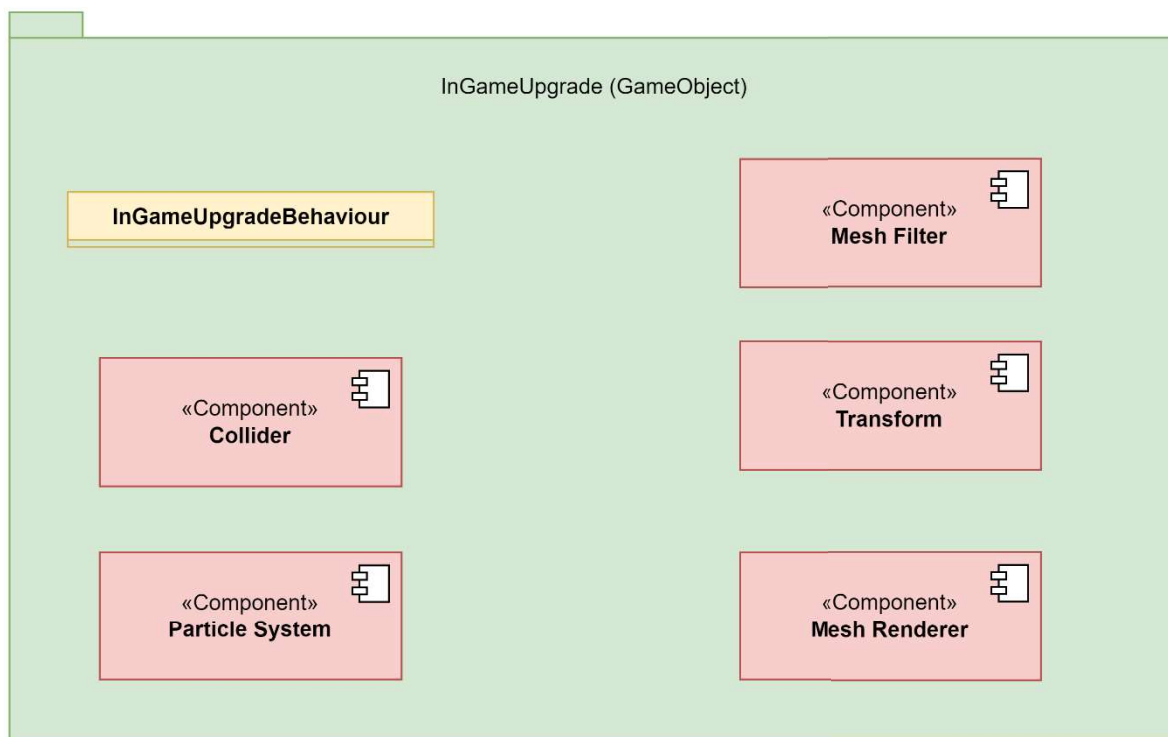


Figura 14: Diagrama del GameObject *InGameUpgrade*

Este *GameObject* el cual representará las mejoras que el jugador encontrará por la mazmorra y las cuales podrá recoger, cuenta con un script que tendrá almacenado todos los atributos que mejorará a las estadísticas del jugador cuando este lo recoja. Además de esos atributos, también cuenta con otro que indica cuánto dinero cuesta la mejora, el cual servirá para diferenciar las mejoras de las salas de mejoras y las mejoras de la sala del mercader.

El siguiente *GameObject* para analizar será el portal que los jefes del piso crearán una vez estos hayan sido derrotados:

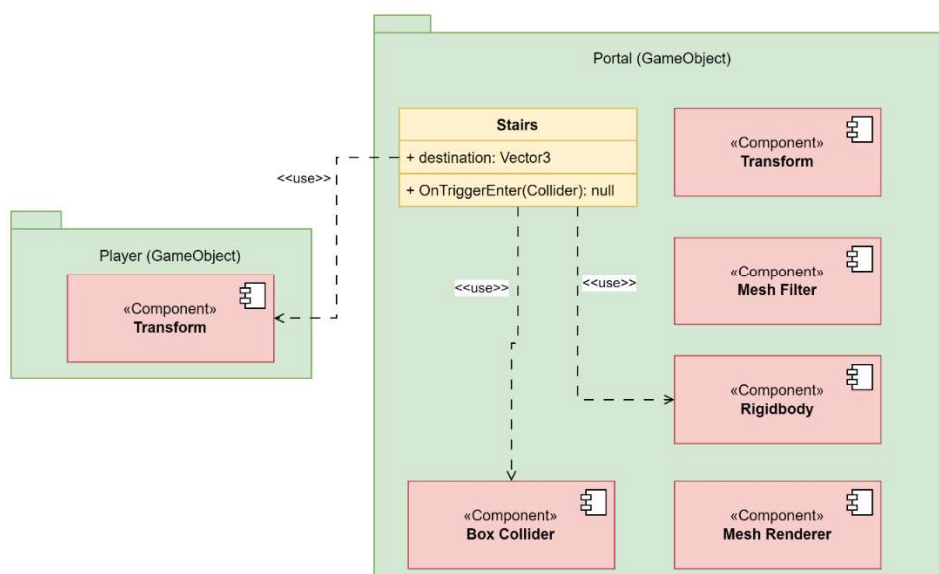


Figura 15: Diagrama del GameObject *Portal*

Este objeto contendrá un vector con los tres ejes, que indicará a donde este portal tendrá que teletransportar al jugador. Mediante la función `OnTriggerEnter()` la cual hará uso del *Rigidbody* y del *Box Collider* hará que cuando este objeto detecte una colisión con el jugador, este cambiará la posición del *Transform* del jugador, haciendo que se teletransporte.

Los siguientes objetos que se diseñaron fueron los dos tipos de proyectiles que habrá en el juego: por un lado, están los ataques del jugador, siendo estos los *GameObject MagicCard*, y, por otro lado, están los ataques de los enemigos, siendo estos los *GameObject SphereAttack*. Primero se empezará con el *GameObject MagicCard*:

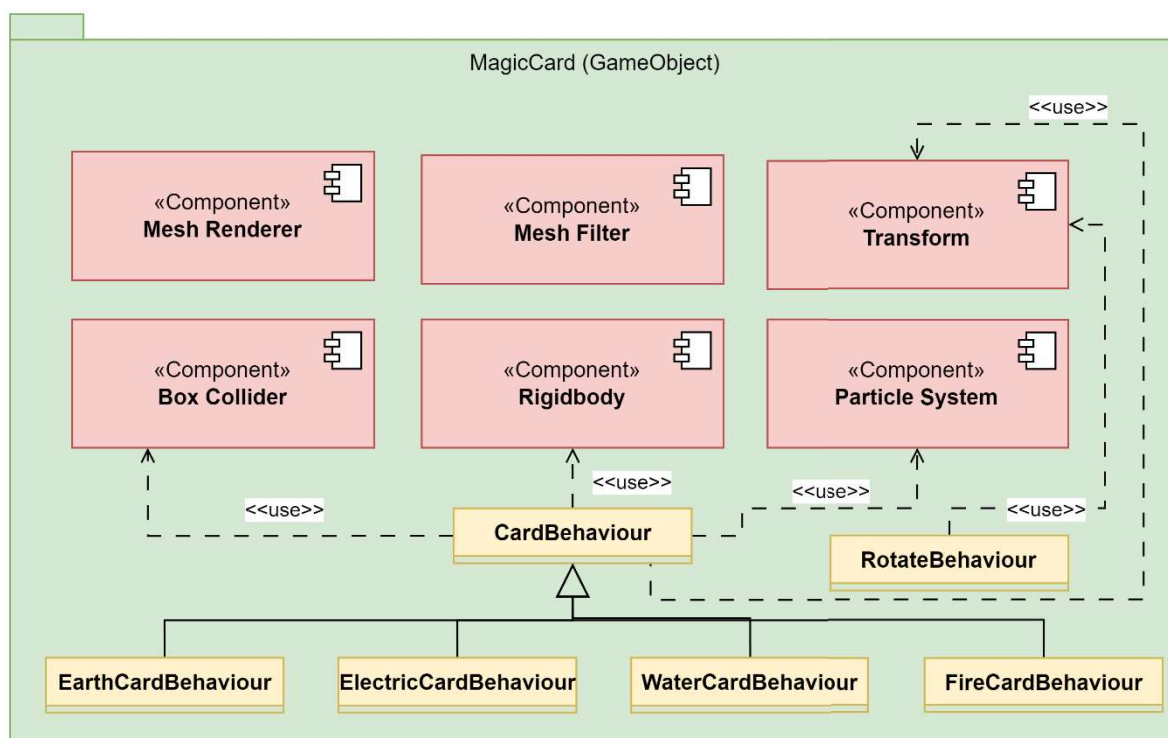


Figura 16: Diagrama del *GameObject MagicCard*

El *GameObject MagicCard*, el cual será el arma a distancia del jugador, tendrá varios scripts, los cuales son:

- **RotateBehaviour:** Este script usará el propio *Transform* de este objeto y hará que la carta rote continuamente sobre sí misma y de esta manera simulará el efecto que tendría una carta al ser lanzada en la vida real.
- **CardBehaviour:** Este script tendrá los atributos y funciones que tendrán en común todas las cartas. En este caso los únicos atributos que tienen en son el daño que la carta realiza y el sistema de partículas que se usará cuando la función `OnTriggerEnter()`, que usa los componentes *Box Collider* y *Rigidbody*, detecte que ha colisionado con algo y en ese caso destruirá la carta y generará, en la posición en el que la carta haya sido destruida, el efecto de partículas simulando la explosión de la carta. De esta clase principal nacen cuatro clases hijas, las cuales tendrán dependiendo de que carta sea, su daño de elemento y su efecto especial

(como los mencionados en el apartado 3). Por ejemplo, el script `FireCardBehaviour` tendrá un atributo llamado `fireDamage` y `burningEffect`.

Finalizando con los tipos de ataques, seguiremos con los ataques de los enemigos llamados `SphereAttackBehaviour`:

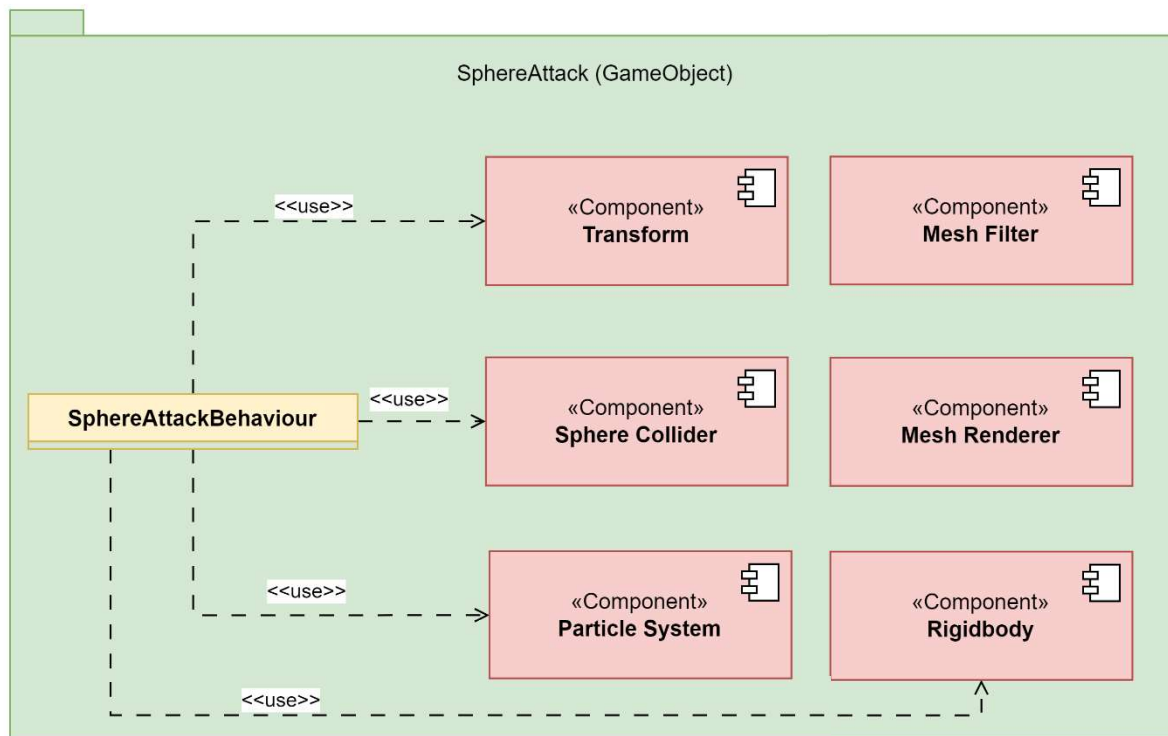


Figura 17: Diagrama del `GameObject SphereAttack`

Este `GameObject` cuenta con un script llamado `SphereAttackBehaviour`, el cual tiene un atributo que indica el daño que causa esa esfera, el efecto de partículas que generará al destruirse y la función `OnTriggerEnter()` anteriormente nombrada, la cual detectará cuando ha colisionado con algo y se destruirá.

A continuación, se explicará el diseño del `GameObject` que controlará el jugador en la partida, el cual se le ha decidido nombrar `Player`. Este está representado en la Figura 18:

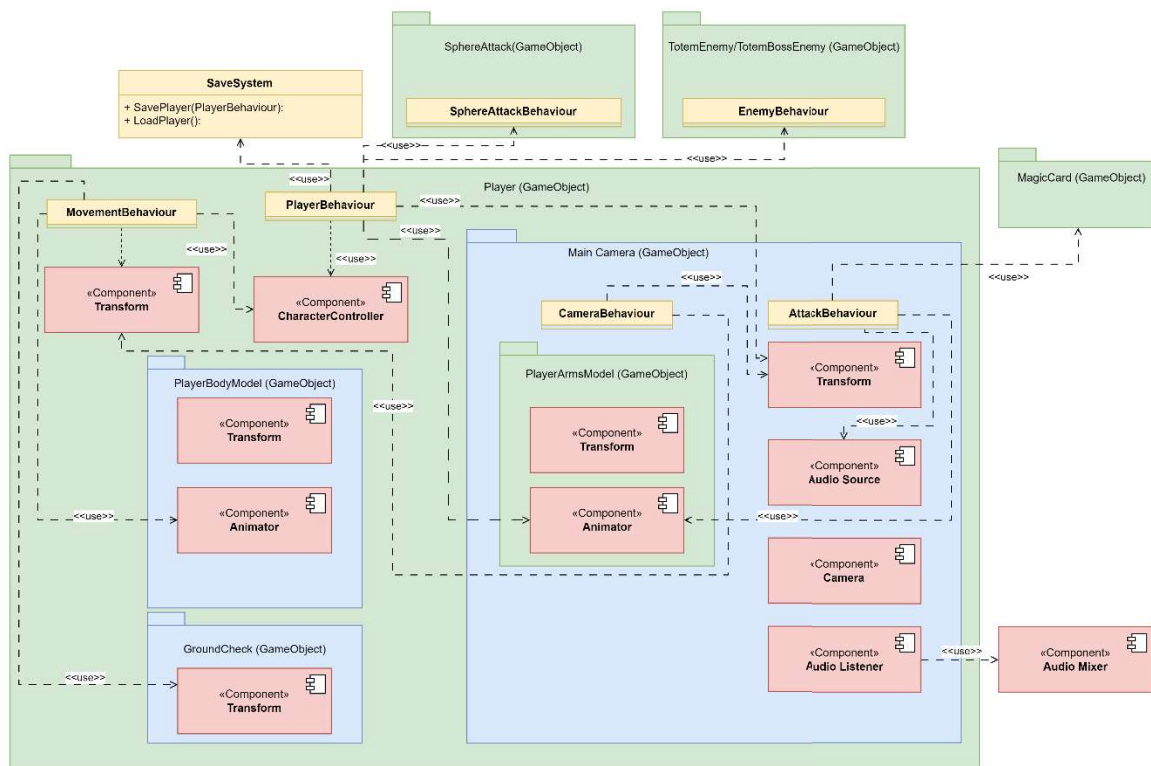


Figura 18: Diagrama del *GameObject* Player

En este diagrama podemos observar cómo se diseñó el *GameObject* Player. A continuación, se explicará la estructura de este objeto:

- **GameObject GroundCheck:** Se decidió en el diseño que en vez de usar el componente *Rigidbody* para calcular las físicas y el funcionamiento del salto se usaría este objeto (a lo largo de este apartado se explicará cómo se consiguió).
- **GameObject PlayerBodyModel:** Este objeto es el modelo del cuerpo sin brazos del personaje, el cual mediante el componente *Animation* puede realizar las animaciones creadas en Blender. Se ha decidido agregar este modelo dentro del *GameObject* Player, debido a que al moverse este, el modelo del cuerpo también lo hará.
- **GameObject Main Camera:** Esta es la cámara principal por la que el jugador verá el juego. Se ha decidido que en vez de dejarla separada como cualquier otro objeto se adjuntará al objeto Player, ya que de esta manera al mover el *GameObject* Player también lo hará la cámara y no sería necesario crear un script que hiciese que la cámara siguiera al jugador. Dentro de la cámara nos encontramos con los siguientes elementos:
 - **GameObject PlayerArmsModel:** Este objeto es el modelo de los brazos del personaje, el cual mediante el componente *Animator* puede realizar las animaciones creadas en Blender. Se ha decidido agregar este modelo dentro del *GameObject* Camera debido a que al girar la cámara los brazos también lo harán.

- **Script CameraBehaviour:** Este script será el encargado de detectar cuando el ratón del jugador se está moviendo y en función de la dirección la cual lo haya movido, la cámara del videojuego también lo hará. Para ello lo que necesita este script sería el *Transform* de la *Main Camera* para cambiar su rotación y así girarla, y el *Transform* del *GameObject Player*, para que, si se gira la cámara hacia la derecha o izquierda, el *GameObject Player* también se gire hacia esos lados.
- **Script AttackBehaviour:** Este script es el encargado de lanzar las *MagicCards* hacia la dirección donde esté mirando el personaje del jugador. Para ello, este script usará los *GameObjects MagicCards* para crear copias y lanzarlos como proyectiles. Con el objetivo de generar un mayor impacto en el jugador, se ha hecho que, al atacar mediante este script, usando el *Audio Source*, se reproducirá un sonido que simulará el sonido de lanzamiento de una carta mágica. Adicionalmente, se ha utilizado el *Animator* del *GameObject PlayerArmsModel* para que, al atacar, este realice una animación en la que moverá los brazos hacia delante para lanzar la carta.
- **Script MovementBehaviour:** Este script no solo será el encargado de realizar el movimiento del personaje, sino que también será el encargado de realizar las acciones de salto y embestida. Se decidió no usar un *Rigidbody* para las físicas del personaje principal, y en su lugar se implementó esas físicas por código en este script. Para la realización del movimiento se usó el *Transform* y el *CharacterController*, aplicando una fuerza al objeto, mediante la cual este se movería, saltaría y embestiría. Para dar una impresión de que el jugador está saltando, se decidió usar el *Animator* del *PlayerBodyModel*, ya que este tendría una animación para el salto, la cual se realizaría en el momento en el que el jugador lo saltase.
- **Script PlayerBehaviour:** Este script es el encargado de controlar las estadísticas del combate y de los cambios que pueda producirse en estas. Esos atributos son el poder de ataque del jugador, la cantidad de vida que tiene, el poder que tiene de cada elemento, la velocidad a la que la carta es lanzada, la frecuencia a la cual puede lanzar cartas y el dinero que tiene en ese momento. Las funciones que tiene este script son las siguientes:
 - Detectar cuando un *GameObject SphereAttack* ha colisionado con él *Player* y en ese caso reducir la vida del *Player*. Para ello, se ha utilizado el componente *CharacterController* que tiene el objeto *Player* y de esta manera detectar las colisiones. El daño recibido será el poder del ataque del enemigo multiplicado por el daño que causa la esfera.
 - Controlar la vida que tiene el jugador y en cuanto esta sea menor de 0 avise y que de esta manera aparezca el menú de *GameOver*.
 - La funcionalidad de que si hay un *GameObject InGameUpgrade* en frente del jugador y se usa el botón del teclado al cual se le ha asignado la acción de recoger mejora, ese *GameObject* desaparezca y el jugador mejorará las estadísticas

correspondientes que el objeto `InGameUpgrade` indicaba. Para ello se ha usado el *Transform* de la `Main Camera` y de esta manera saber a dónde estaba mirando el jugador. Además, se ha usado el componente *Animator* de `PlayerArmsModel` para que cuando realice esta acción el modelo realice la animación de recogida de mejora.

- Cargar al inicio la información del sistema de guardado utilizando el script `SaveSystem` y almacenar al final una partida la información en el archivo de guardado, independientemente de que haya ganado o perdido la partida.

El siguiente *GameObject* a analizar será el `TotemEnemy/TotemBossEnemy`, los cuales serán los enemigos del videojuego:

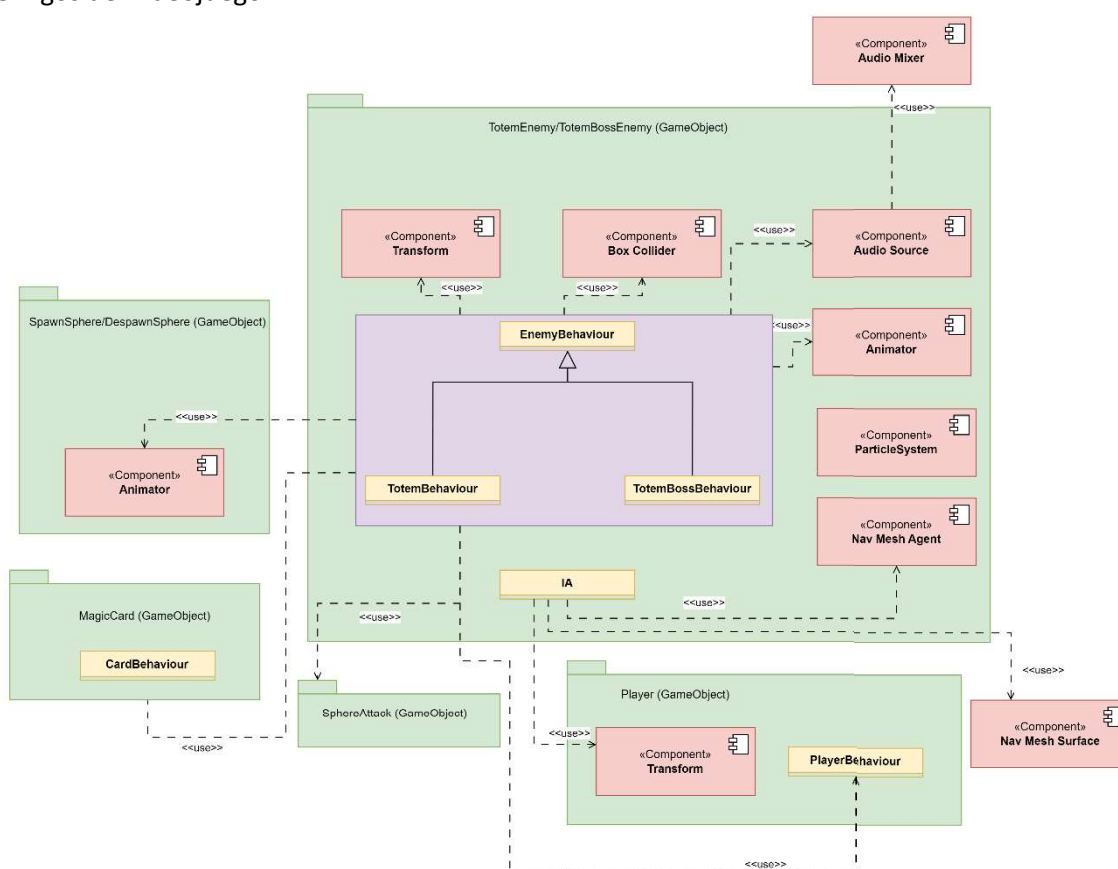


Figura 19: Diagrama del GameObject TotemEnemy/TotemBossEnemy

Como se puede observar en la figura se ha agrupado varias clases en un cuadrado morado. Esto se ha hecho debido a que de esta manera se reducen la cantidad de flechas a usar, ya que si no el diagrama no es entendible. Aun así, en la explicación se explicará cuáles son las relaciones. Ahora se detallará todas las clases y otros *GameObjects* que tienen los enemigos y como se relacionan con el resto de los objetos:

- **Script IA:** Este script será el encargado de crear una IA usando *NavMesh* y que se encargará de perseguir al jugador. Para ello se ha usado el componente *NavMeshAgent* de este *GameObject* para establecer el comportamiento del objeto a la hora de perseguir al jugador, el *Transform*

del objeto `Player` para establecer como destino del agente la posición del personaje del jugador y el `NavMeshSurface` que se generará mediante el `GameObject` `DungeonCreator` y el cual indicará el terreno por el cual el enemigo puede moverse.

- **Script `EnemyBehaviour`:** Este script es el encargado de encapsular todo el comportamiento y atributos que tendrán en común los enemigos. Los atributos que todos los enemigos tendrán en común son su cantidad de vida, su poder de ataque, la resistencia o debilidad al daño que tiene al recibir un ataque de una `MagicCard` de un elemento determinado, su velocidad de ataque, la frecuencia a la cual puede atacar y la acumulación de efectos especiales causados por los impactos recibidos de `MagicCard`. Las funciones que este tiene son:
 - Detectar si este `GameObject` ha recibido una colisión con un `GameObject` `MagicCard` lanzado por el jugador. Para ello se usará el *Box Collider*, en caso de que haya sido golpeado por una `MagicCard` se le aplicarán los efectos especiales de esa carta. Estos efectos son los mencionados en el apartado 3. Además, este `GameObject` recibirá el daño en base a la siguiente formula:

```
health -= (cardDamage * playerDamage) * (
  ((cardFireDamage * playerFireDamage) / fireResistance) +
  ((cardWaterDamage * playerWaterDamage) / waterResistance) +
  ((cardElectricDamage * playerElectricDamage) / electricResistance) +
  ((cardEarthDamage * playerEarthDamage) / earthResistance)
);
```

- Detectar cuando este `GameObject` tiene menos de 0 de vida y en ese caso después de un tiempo este `GameObject` se destruye, dando oro como recompensa al jugador accediendo al script `PlayerBehaviour` del `GameObject` `Player`. Para no hacer que desaparezca en frente del jugador, se hizo que apareciese una `DespawnSphere` (mencionada ya en este apartado) y activar su animación, ocultando la desaparición del enemigo.

De este script se han creado dos hijos, siendo el `TotemBehaviour` el que se agregará a los enemigos normales llamados `TotemEnemy`, y el otro siendo `TotemBossBehaviour`, el cual se agregará al jefe del piso llamado `TotemBossEnemy`. Ambos scripts cuentan con una función la cual creará un `GameObject` `SphereAttack` y lo lanzará contra el `Player` mientras realiza una animación usando el componente *Animator* y generando un sonido usando el componente *Audio Source*. Aunque estos scripts realizan la misma acción, sus tipos de ataque tienen ciertos cambios entre enemigos normales y jefes de piso, por lo que se ha decidido no agregar esta funcionalidad en el script padre `EnemyBehaviour`. Otra diferencia es que el script `TotemBossBehaviour` hace que cuando el jefe del piso al cual está agregado este script haya sido destruido aparezca una mejora en la sala y el portal que llevará al jugador al siguiente piso.

El siguiente `GameObject` a analizar será el `GameObject` llamado `Room`, el cual representará todas las salas de la mazmorra y sus diferentes tipos:

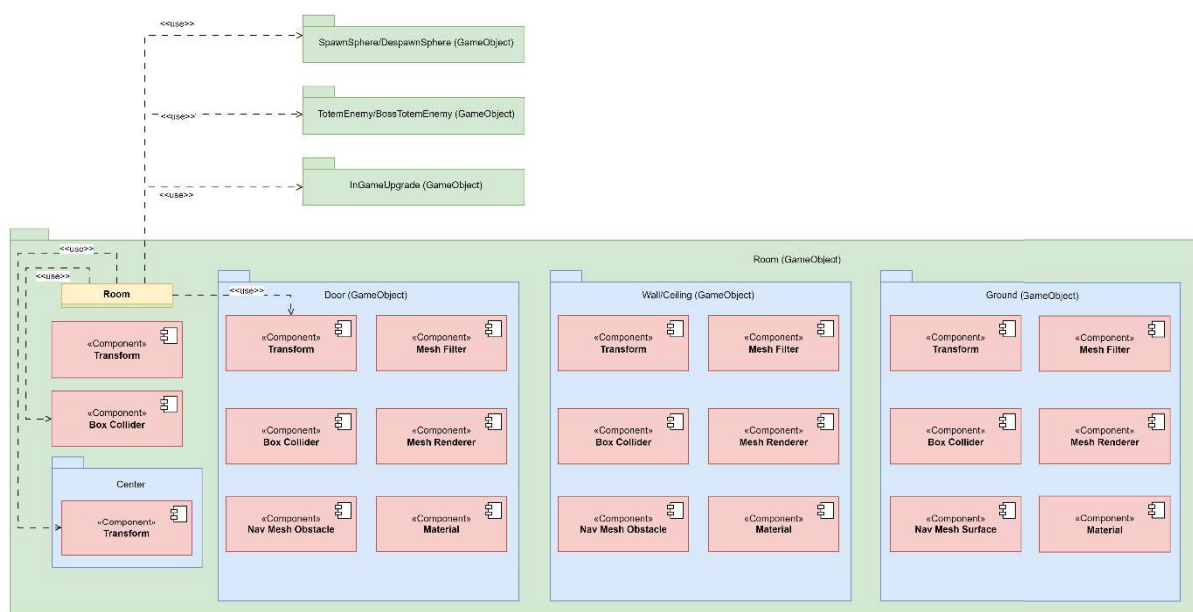


Figura 20: Diagrama del GameObject Room

Al igual que pasaba con el `GameObject HallwayRoom` de la escena `StartMenu` mencionado en el apartado 5.4.1, este está compuesto por otros `GameObjects`. La diferencia radica en que al contrario que el objeto `HallwayRoom` es que además de contar con el techo, las paredes y el suelo, también cuenta con otro `GameObject` que representa las puertas y un `GameObject` vacío llamado `Center` situado en el centro de la sala. El `GameObject Room` cuenta también con un script llamado `Room`, el cual es el encargado de varias funciones, siendo estas las siguientes:

- Es el encargado de crear y colocar todos los objetos que se le ha determinado en el script `DungeonGenerator` del `GameObject DungeonCreator`, al mismo tiempo que generará `SpawnSphere/DespawnSphere` para enmascarar la aparición de los enemigos mediante la animación de estos `GameObjects`.
- Es el encargado de detectar que si el jugador está lo suficientemente cerca del centro de la sala (motivo por el cual usaremos el `Transform` del `GameObject Center`) se abrirán las puertas de esa sala dejando pasar al jugador. En caso contrario se cerrarán hasta que el jugador vuelva a estar cerca.
- Es el encargado de cerrar las puertas de la sala en caso de que el jugador entre en la sala (lo cual se sabrá cuando el componente `Box Collider` detecte una colisión y esta sea la del `GameObject Player`) siempre y cuando esa sala fuese una sala de combate o de jefe de piso. Una vez el `Box Collider` de una sala no detecte ningún enemigo (debido a que habrán sido destruidos al ser eliminados por el jugador) las puertas se abrirán. Para abrir y cerrar las puertas se hace uso del `Transform` del `GameObject Door`.

El penúltimo `GameObject` que se explicará será el encargado de generar la estructura de la mazmorra de manera procedural y determinar que `GameObjects` aparecerán en la sala. Este `GameObject` ha sido llamado como `DungeonCreator`:

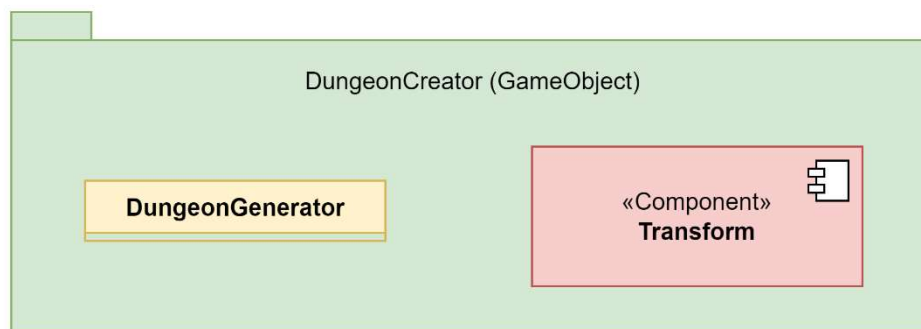


Figura 21: Diagrama del GameObject DungeonCreator

Este *GameObject* cuenta con un único script llamado *DungeonGenerator*, el cual no solo es el encargado de crear la estructura de la generación procedural de la mazmorra (la cual se profundizará en el apartado 5.5.10), sino también será el encargado de instanciar las salas, determinar el destino de los portales, establecer el coste de las mejoras, colocar al jugador en el primer piso y en la sala inicial, establecer a los enemigos la posición destino a la cual estos se tienen que mover (la cual es la posición del jugador), determinar cuántos y que tipos de enemigos habrá en cada sala, potenciar a los enemigos en función de en qué piso estén (cuanto más alto sea el piso más poderosos serán los enemigos) y establecer que elementos conformarán a la sala.

Para terminar con este apartado se analizará la estructura que se diseñó para el *GameplayCanvas*, la cual está representada en la siguiente figura:

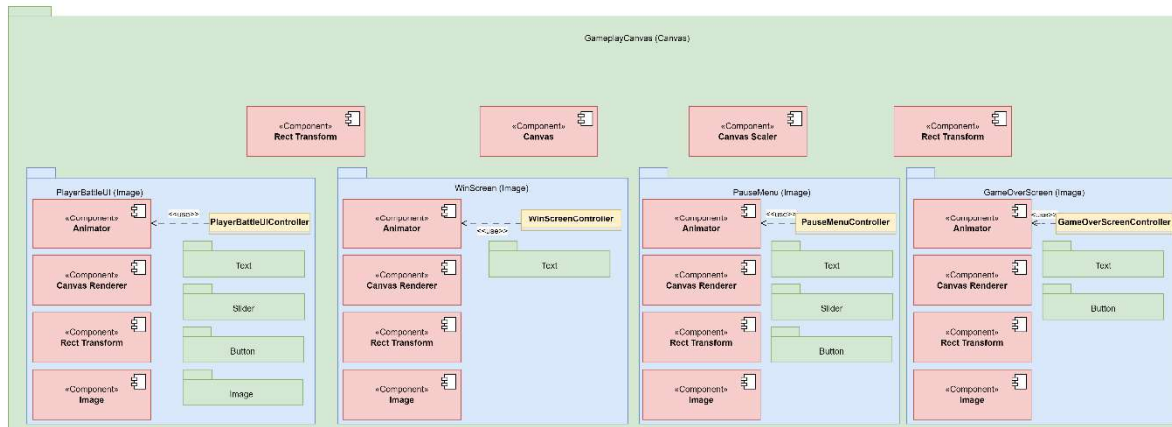


Figura 22: Diagrama del GameplayCanvas

En esta figura podemos ver como el *GameplayCanvas* contiene, aparte de sus elementos básicos, varios menús e interfaces, las cuales tienen un script. Ese script lo que hará en cada uno de sus menús correspondientes será controlar mediante el *Animator* cuando esos menús aparecen por pantalla o como estos cambian cuando el jugador realiza una determinada acción. Además, en el caso del *PauseMenu*, su script usa también el componente *Audio Mixer*, ya que al igual que pasaba en el menú inicial, en este menú el jugador puede reducir o aumentar el volumen del juego.

5.5 DESARROLLO DEL VIDEOJUEGO

Una vez hemos hablado del diseño del videojuego, es hora de hablar como se desarrolló los objetos, componentes y funcionalidades principales y más importantes que componen este videojuego.

5.5.1 Modelos y animaciones

Como se mencionó en el apartado 5.2, en este proyecto se ha usado Blender para la creación de los modelos 3D y sus animaciones. Gracias a esta herramienta se han creado tanto el modelo del personaje del jugador como el de los enemigos.

Como el proceso de creación de modelos 3D es un proceso muy largo, se decidió en la creación de ambos modelos ser lo más minimalista con los detalles, creando modelos simples, pero a la vez lo suficientemente complejos como para que no den una mala impresión al jugador.

En el caso del personaje del jugador se diseñó de manera que pareciese una persona que llevaba una especie de túnica, ya que de esta manera no sería necesario agregar piernas al modelo, puesto que llevaría mucho tiempo realizar la animación de caminata. Además, se decidió agregar al jugador unos guantes, ya que eso reduciría mucho la complejidad del modelo, debido que no haría falta agregar los dedos con todos sus huesos. Por último, se le puso al personaje del jugador una máscara para ahorrar así la necesidad de agregarle rasgos faciales. Esta última decisión se tomó porque, si bien al ser un videojuego en primera persona nunca se vería la cara del personaje, en un principio un tipo de enemigos iban a ser unas copias del personaje principal con otros colores, por ende, a este tipo de enemigos sí que se les vería la cara, por lo que el uso de una máscara era necesario. Al final se acabó descartando esta idea.

Para crear el modelo del personaje, lo que se hizo fue crear un cubo y cortarlo por la mitad en la pestaña de edición y eliminar una de las mitades. Esto se hizo porque para crear un modelo del personaje simétrico solo es necesario la mitad de ese modelo y aplicar al modelo el modificador de espejo (mirror). Al hacerlo todo lo que se cambie de un lado también lo hará en el otro de manera automática, lo cual supone un gran ahorro en de tiempo.

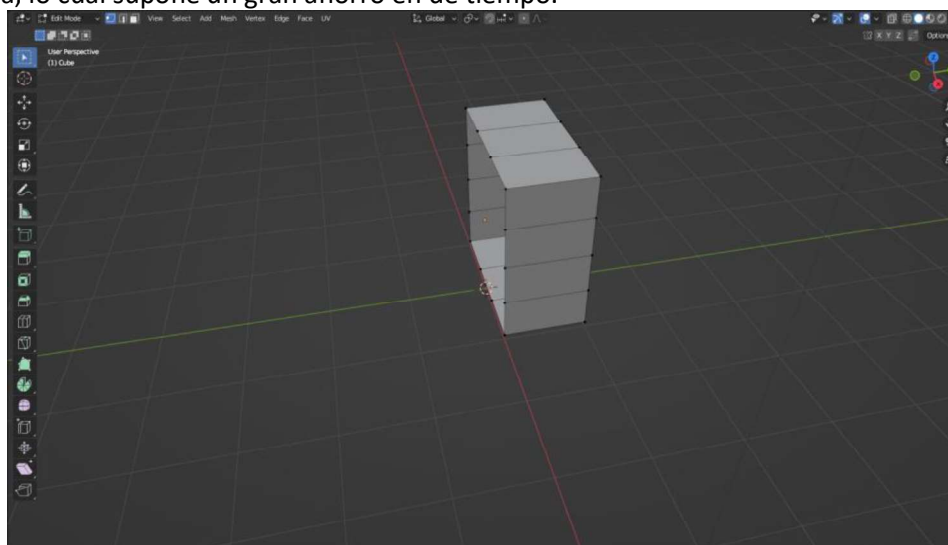


Figura 23: Cubo partido a la mitad

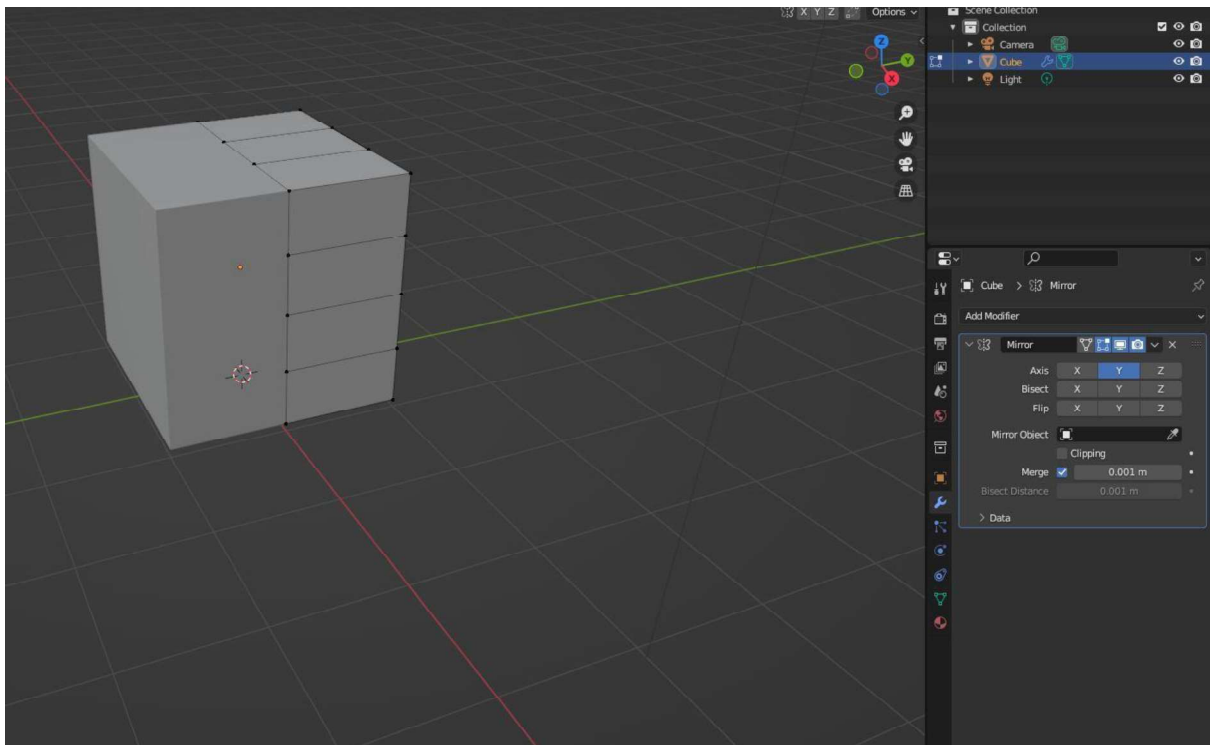


Figura 24: Aplicación del mirror al cubo

Luego en la pestaña de edición que tiene Blender, en la cual puedes modificar la forma de cualquier objeto, entre otras muchas cosas, se empezó a realizar cortes al cubo para generar más caras en este, y mediante las herramientas de extrusión se empezó a seleccionar caras y desplazarlas. Usando ambas herramientas se creó a un personaje cuadrado, el cual tenía una cara, un cuerpo, unos brazos y manos. Después, para conseguir simular que en el cuerpo tenía una túnica, se eliminó la cara de abajo del todo del modelo.

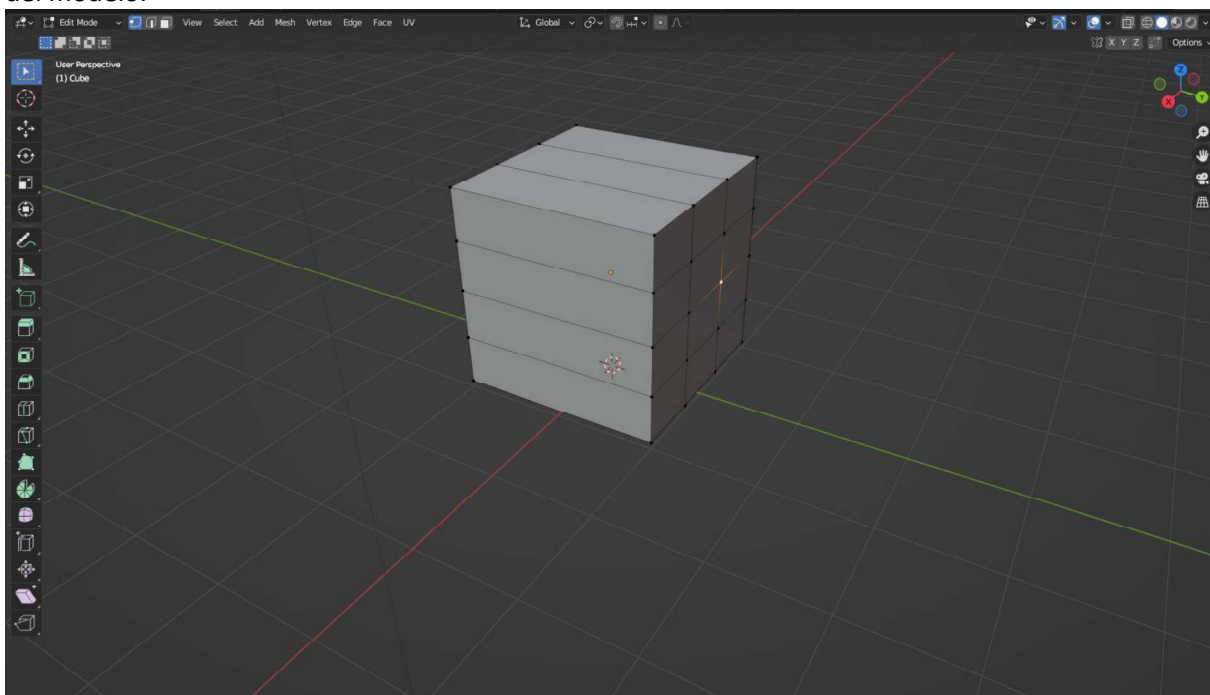


Figura 25: Realización de los cortes al cubo usando la herramienta de corte

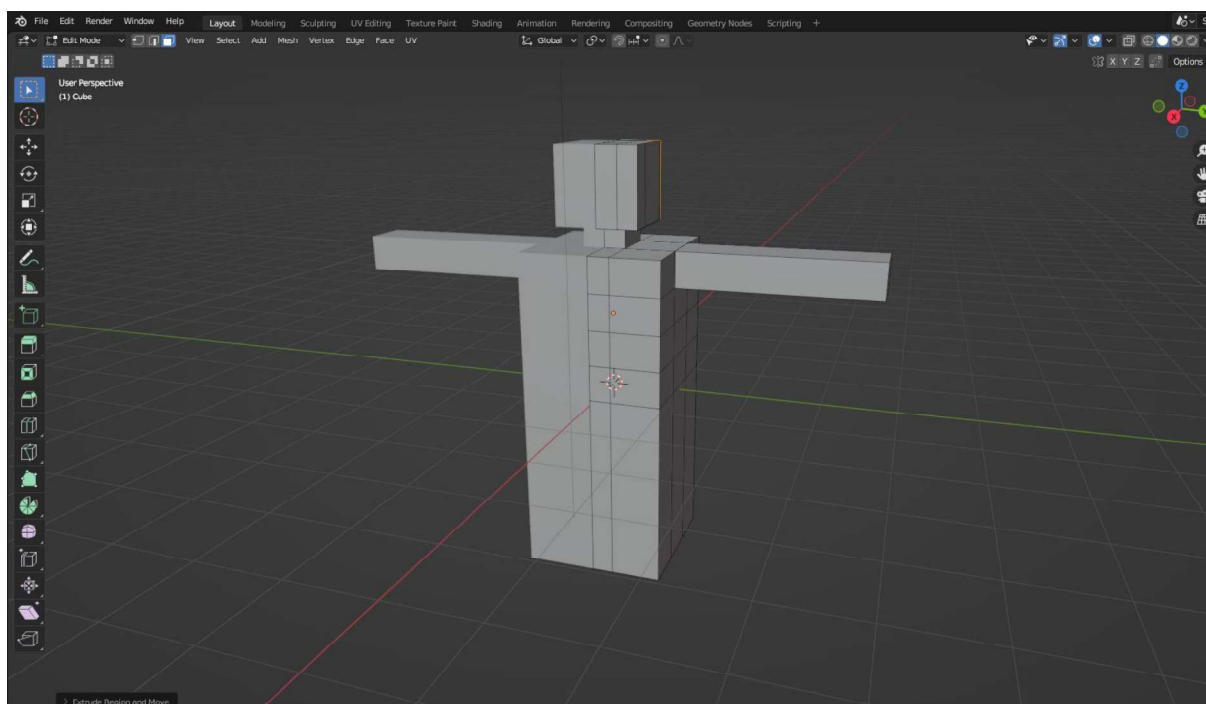


Figura 26: Caras del cubo estiradas usando la herramienta de extrusión

Una vez hecho el modelo principal, se agregó a este modelo un modificador llamado subdivisión. Este modificador lo que hace es interpolar entre vértices y caras para crear una superficie más suave y que de esta manera parezca más realista. Una vez aplicado este modificador el modelo del personaje cambio completamente, teniendo una apariencia más redonda. A continuación, se retocó el resultado obtenido agregando más caras al modelo y moviendo vértices para darle una mejor forma.

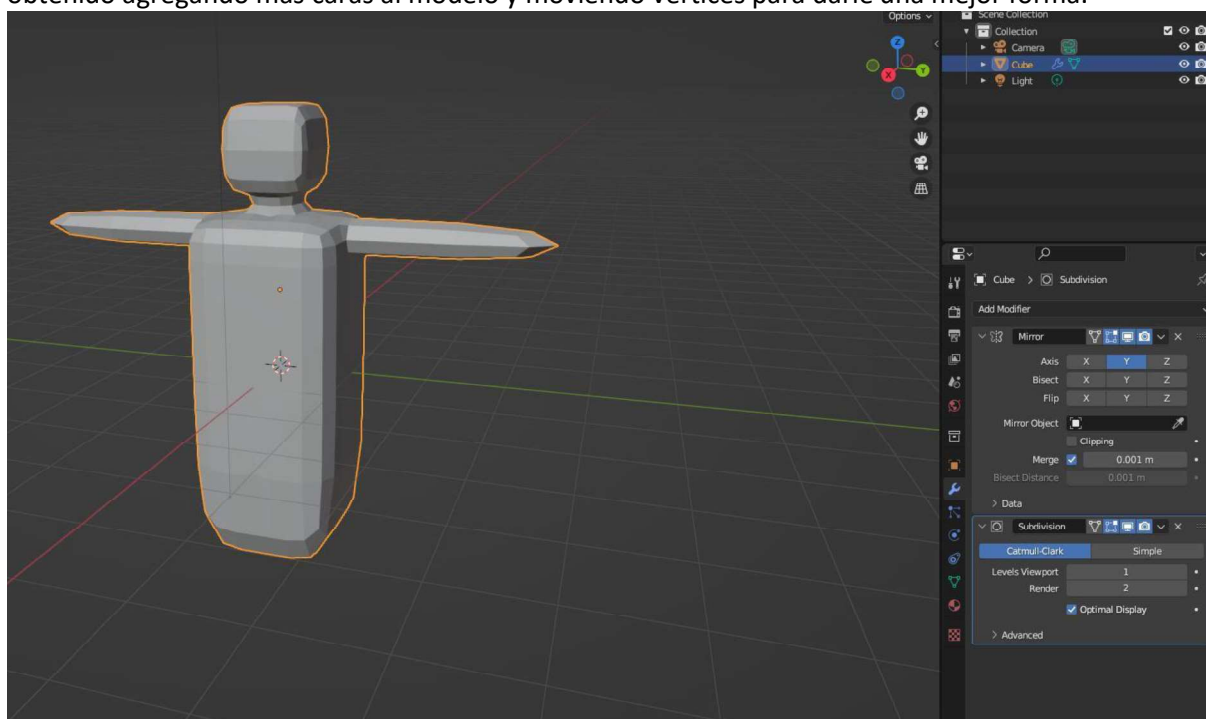


Figura 27: Modelo de personaje con el componente subdivisión

Una vez se consiguió una apariencia aceptable se le agregó una máscara. Para ello se creó una esfera, se partió de manera que solo quedará un cuarto de la esfera y se modificó el tamaño para que pareciera una máscara. Una vez se consiguió esa mitad de máscara, se fusionó la máscara al personaje, convirtiéndola parte de él y gracias al modificador del mirror que tenía el personaje, esa mitad de máscara se transformó en una máscara completa.

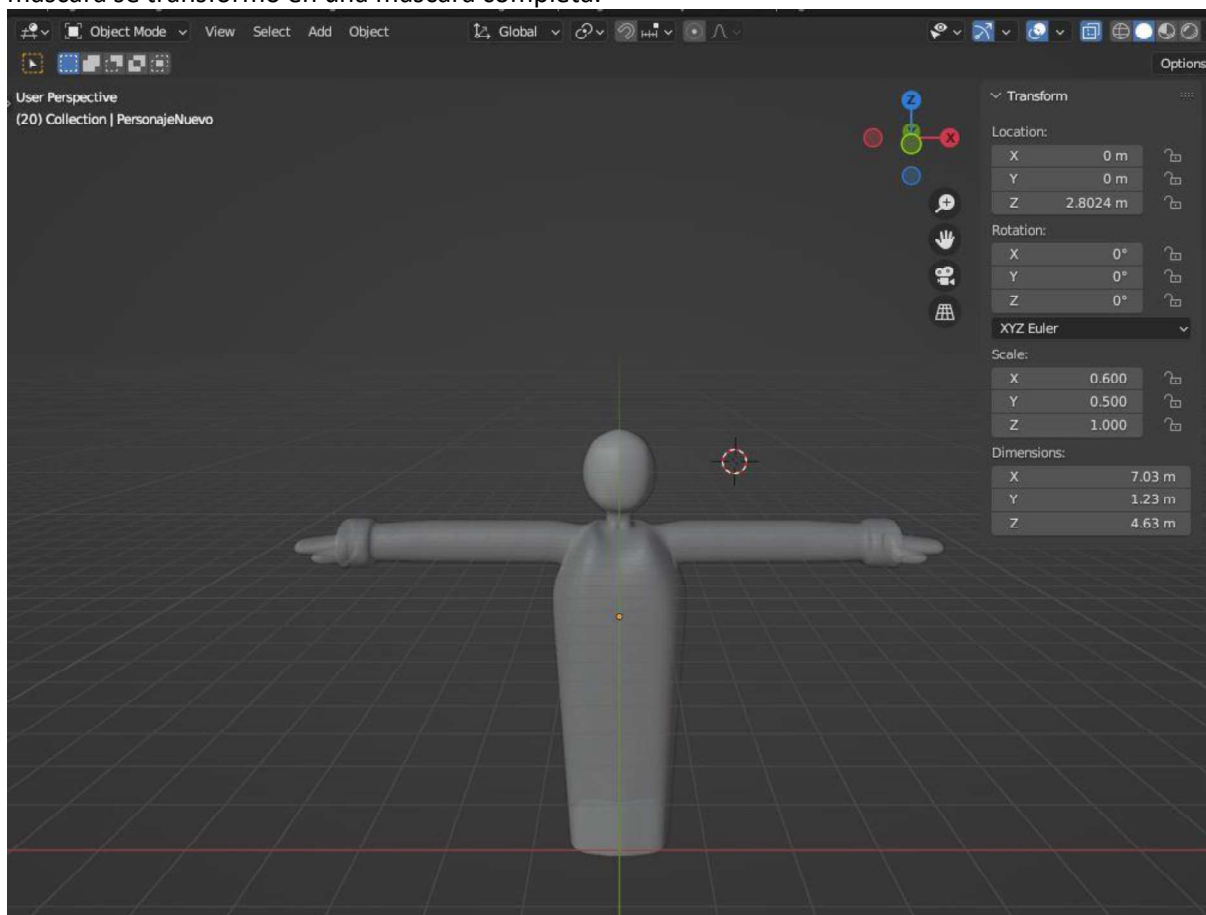


Figura 28: Modelo del personaje con máscara

Habiendo realizado el paso de la creación de la estructura del modelo empezó a pintarlo. Para ello lo primero que se hizo fue agregar un material al objeto, mediante el cual se le podía cambiar el color del modelo completo y la intensidad con la cual el modelo refleja la luz entre otras muchas más cosas. Como se quería pintar el modelo 3D se agregó al material una textura, la cual inicialmente estaba en blanco.

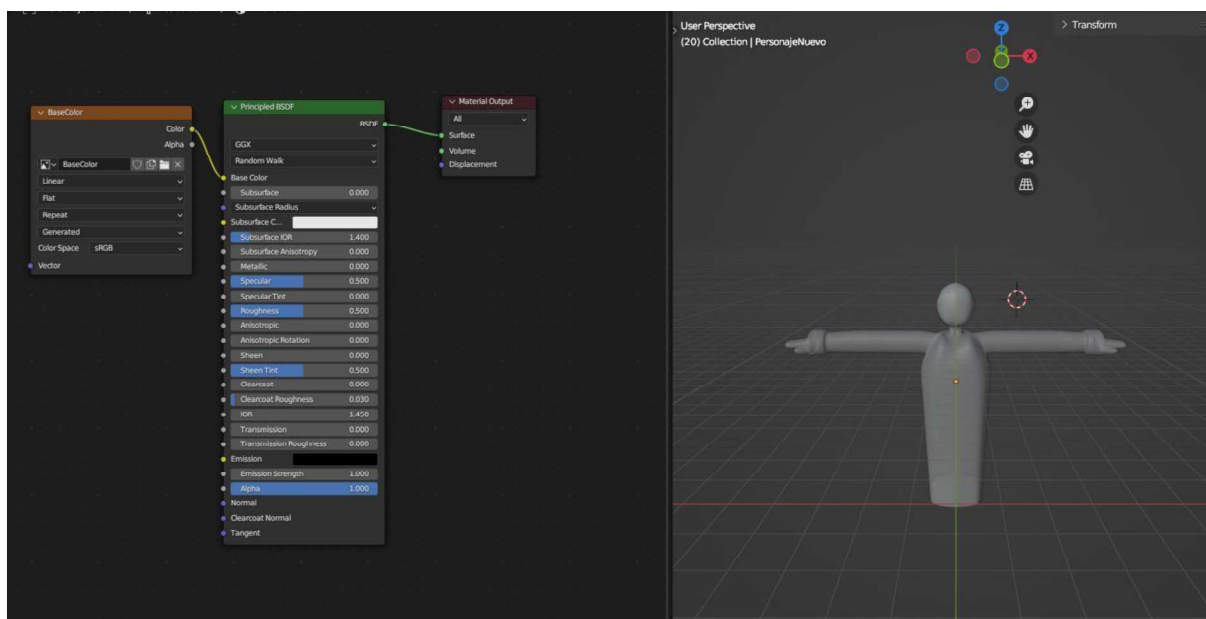


Figura 29: Material agregado al modelo

Antes de continuar veo conveniente explicar cómo funciona el modo en el que Blender pinta los modelos. Lo que hace Blender es usar la técnica de *unwrap* (desenvolver), mediante la cual coge el modelo y lo desenvuelve, dejando el modelo tridimensional como una figura de dos dimensiones. Una vez hecho eso, Blender comenzará a pintar todo lo que este unido a ese modelo 2D de un mismo color. Sin embargo, en el caso nuestro personaje, ya se ha dicho que los colores que tendrá serán diferentes entre sí. Para lograrlo, lo que hay que se hizo realizar ciertos cortes especiales (a los cuales me referiré como marcas a partir de ahora para no confundirlos con los cortes usados para crear las caras adicionales en el cubo) en los bordes del modelo, y de esta manera se separó en cachos para que cuando Blender desenvuelva el modelo 3D haya varios trozos. De esta manera cada trozo podrá ser pintado de un color diferente.

Para ello lo que se ha hecho es ir a la ventana de edición y realizar varias marcas en los vértices. En el caso de este personaje, se ha realizado marcas entre el guante y la manga de la túnica, entre el cuello del personaje y la túnica, y por último entre la máscara y la cabeza. Habiendo hecho esto, al desenvolver el modelo 3D y gracias al mirror, se ha conseguido separar los guantes, la túnica, la máscara y la cabeza.

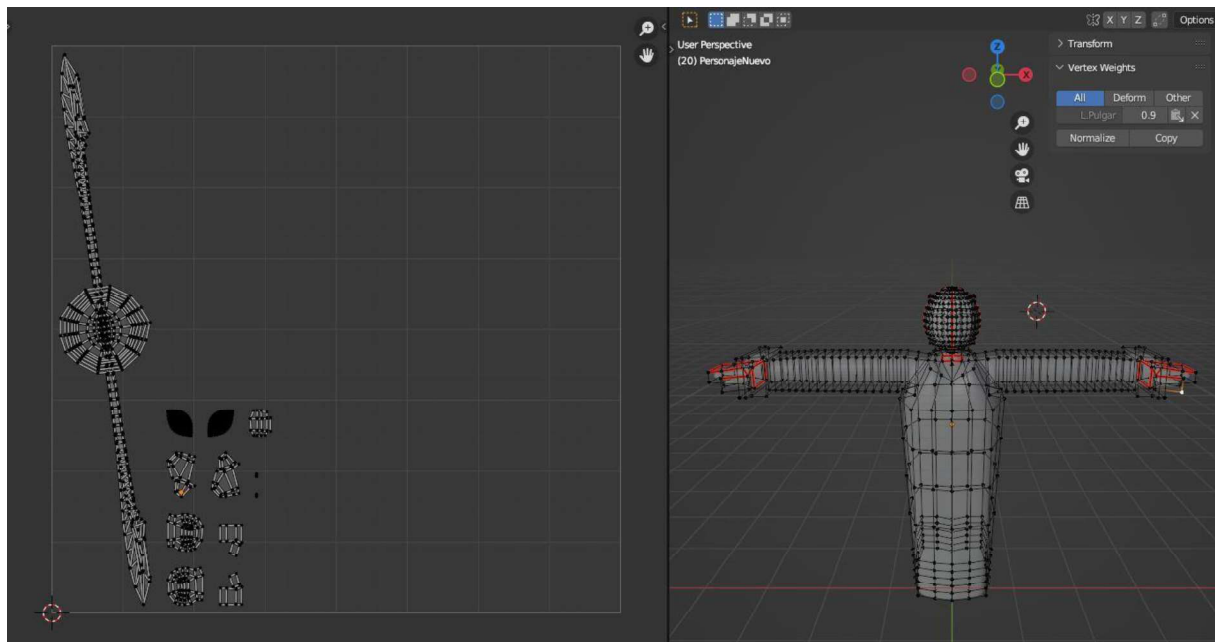


Figura 30: Marcas agregadas al modelo y realizado el unwrap

Finalizado el proceso de separación de las partes del cuerpo, se continuó con el proceso de pintado. Lo que se hizo fue seleccionar las partes que se querían pintar de un color (por ejemplo, los guantes) en la ventana de edición, e ir a la ventana *texture paint*, en la cual, seleccionando la cubeta y el color que se quería queramos, al dar clic en una de las caras de la parte seleccionada, esta se pintará del color seleccionado. Esto se realizó hasta que se pintaron todas partes del personaje. Cabe resaltar que lo que se está pintando es la textura asociada al personaje, y esta misma pese a estar vinculada al personaje en Blender, al exportar el modelo a Unity no se exportara la textura, ya que esta es independiente del modelo. Por ello es importante guardar esta textura en una carpeta segura para luego en Unity agregársela al modelo.

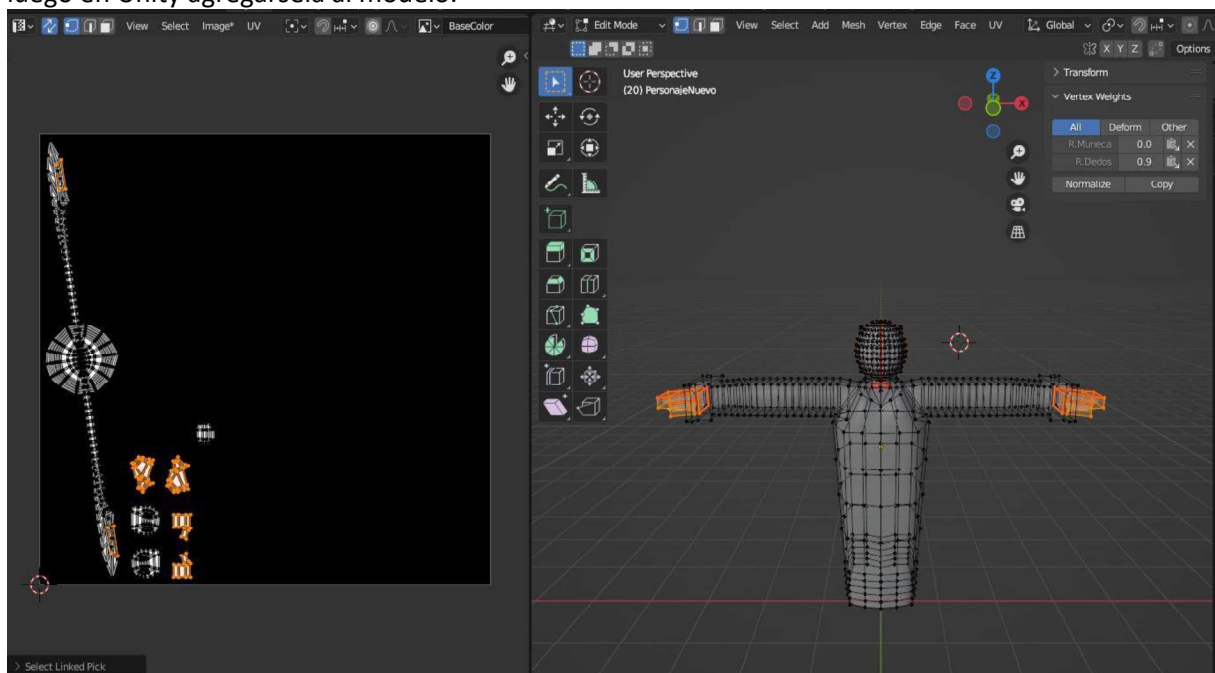


Figura 31: Guantes del personaje seleccionados para pintarlos

El siguiente paso fue agregar al personaje un esqueleto al personaje. Este esqueleto permite al personaje mover las partes de su cuerpo, lo cual es crucial para realizar las animaciones del personaje. Para ello crearemos un objeto llamado *Armature-SingleBone*. Al hacerlo aparecerá objeto que simula el comportamiento de un hueso. A partir de este hueso principal, se puede crear varios huesos que serán hijos de este. Usando este método se creará la estructura ósea del personaje. Una vez se creó la estructura ósea, se establecerá esta estructura como un hijo del modelo del personaje. Al hacerlo, Blender calculará de manera automática la cantidad de influencia que tiene un hueso en el modelo del personaje. La influencia que tiene un hueso en un modelo se la conoce como peso (weight).



Figura 32: Agregado el esqueleto al modelo

Al realizar este proceso en mi modelo, me percaté de que algunos huesos tenían pesos en partes del cuerpo que no le correspondían. Debido a ello, se decidió modificar los pesos que tenían los huesos sobre el modelo de manera manual. Para ello se seleccionó el modelo y se abrió la ventana de Weight Paint. En esta ventana aparecerán el modelo separado en vértices y pintado de colores. Estos colores van desde el azul al rojo y al seleccionar un hueso los colores del modelo cambiarán de color y de zona. Estos colores indican la influencia que tiene un hueso en el modelo. Las partes que están en azul representan que el hueso seleccionado no tiene ninguna influencia en esa parte del modelo, mientras

que las partes rojas representan que ese hueso tiene una influencia total en esa parte del cuerpo. Los colores más intermedios representarán diferente influencia, por ejemplo, los colores azulados y verdosos representan que hay poca influencia, mientras que los más rojizos y anaranjados representan que ese hueso tiene bastante influencia.

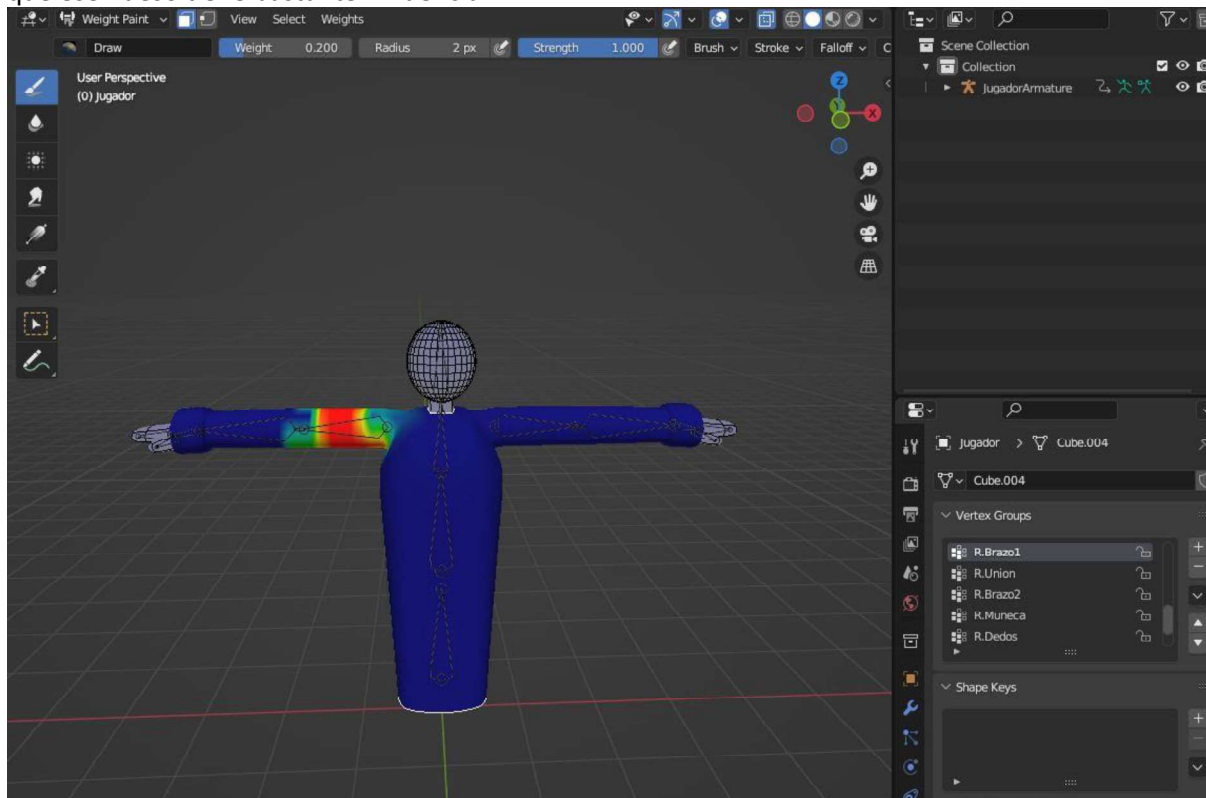


Figura 33: Ventana Weight Panel

Para cambiar el peso que tiene un hueso en el modelo hay que usar la herramienta de pintado de huesos que proporciona esta ventana. Esta herramienta consiste en un pincel, el cual puede pintar colores desde el azul al rojo y por ende cambiar la influencia que tiene un hueso en el modelo. Este pincel solo pintará cuando se pase por encima de un vértice, pintando alrededor de este. Mediante este proceso, se consiguió que el hueso del personaje tuviera una correcta influencia en su modelo, eliminando los errores debido a la automatización de los pesos realizada al unir el esqueleto al modelo.

El último paso antes de exportar el modelo a Unity consiste en crear las animaciones que se usaran posteriormente en el videojuego. Para ello, Blender proporciona una herramienta llamada *Timeline*, en la cual se puede guardar la información relacionada con la posición, rotación y tamaño de un modelo, entre otras muchas más variables en un determinado punto en el tiempo, o lo que sería lo mismo, se guardará en un fotograma como estaba el personaje en ese momento.

Lo que se hará será seleccionar la estructura ósea del modelo e ir a la ventana *Pose Mode*. En esta ventana se puede mover con total libertad los huesos que componen el esqueleto, y al hacerlo se mueve también el modelo del personaje. Para crear las animaciones de este personaje lo que se hizo fue mover los huesos y guardar en el *Timeline* un fotograma el cual contendrá la información de la posición y postura del personaje. Creando varios fotogramas a lo largo del *Timeline* se consiguió crear las animaciones. En el caso de este personaje, este cuenta con cuatro animaciones, una animación de

ataque, una animación de salto, una animación de recogida de objeto y una animación cuando el jugador no esté realizando ninguna otra acción. Todos los fotogramas que componen a todas estas acciones han de estar en el Timeline.

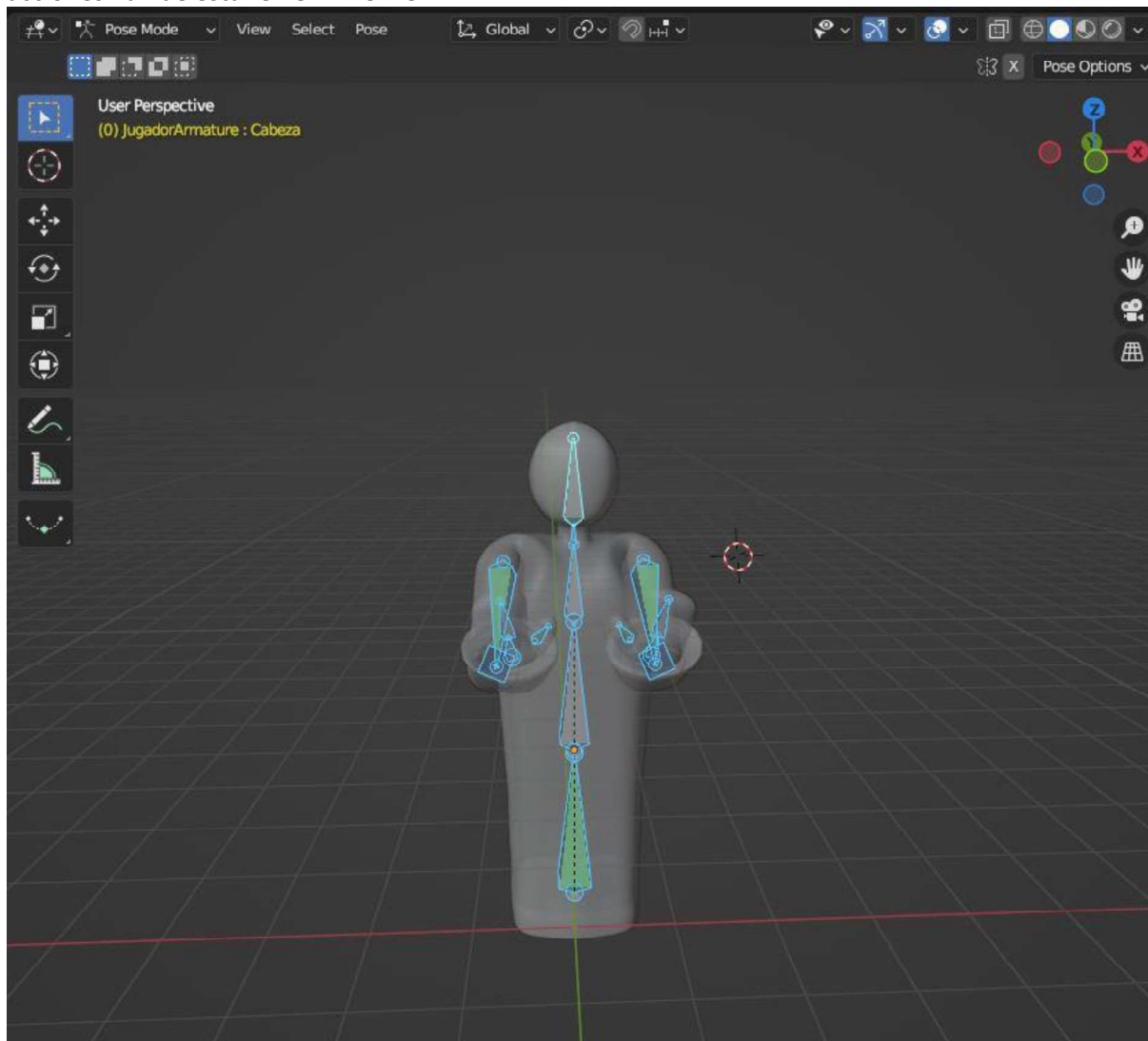


Figura 34: Postura del personaje modificada

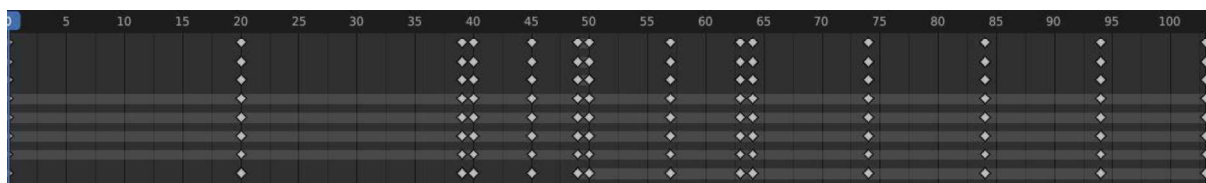


Figura 35: Timeline con los fotogramas del personaje

Habiendo realizado todos los pasos anteriormente explicados, se decidió con este modelo separarlo en dos modelos diferentes, uno siendo únicamente los brazos y el otro siendo el resto del cuerpo. Esto se hizo así, ya que en el planteamiento del videojuego los brazos del jugador deberían de estar apuntando hacia donde estuviese mirando la cámara, mientras que el cuerpo solo tendrá que girarse de manera lateral. Se decidió que para lograr eso la mejor solución era dividir los brazos del resto del cuerpo. Para ello se creó una copia del modelo de Blender, teniendo así dos modelos, y en uno se eliminó el cuerpo y en el otro se eliminaron únicamente los brazos. Una vez hecho eso, se exportó tanto ambos modelos del personaje como la textura de este a Unity.

Para los enemigos del juego se decidió simplificar mucho más tanto la estructura como el proceso de la creación del modelo. Esto se decidió ya que el paso del pintado y de la colocación del esqueleto en el modelo consumía mucho tiempo. Por ello, se decidió que los modelos de los enemigos fuesen cubos voladores, los cuales para atacar se abrirían a la mitad para lanzar un ataque. En total se crearon cinco copias de este cubo (cuatro para cada elemento y uno para el jefe del piso).

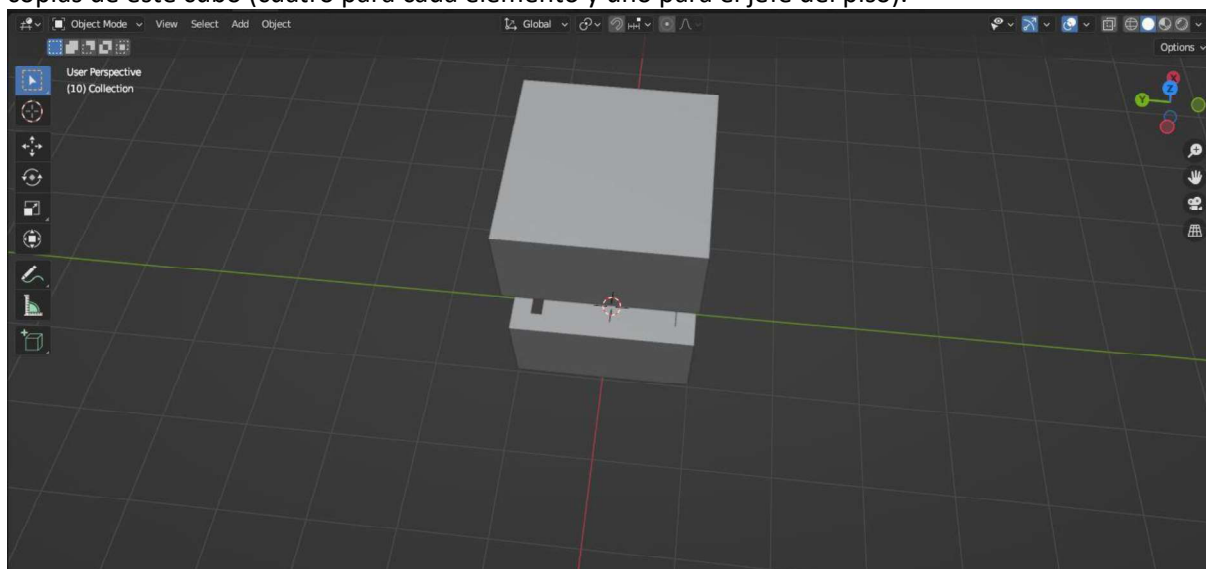


Figura 36: Modelo del enemigo

Para crear este diseño, se cogió un cubo, se partió por la mitad y se extruyeron hacia dentro las caras centrales. Una vez hecho esto creó la animación de ataque, la cual, como comenté anteriormente, hará que se abra el cubo durante unos segundos y luego volverá a cerrarse. Para finalizar se le agregó un material (gracias al cual en Unity se le podrá agregar una textura) y se exportó a Unity.

Estos modelos en Unity no son considerados como *GameObjects* en el momento en el que son importados, sino como un componente que tiene varias ventanas, las cuales son la ventana del modelo, la ventana del rig, la ventana de animación y la ventana de materiales. Para este proyecto las únicas modificadas para todos los modelos han sido la de animación y la de materiales.

Al agregar los modelos a Unity, todas las animaciones están juntas en un mismo clip de video. Como en ambos modelos del personaje teníamos cuatro animaciones diferentes que se realizarán en momentos diferentes, era necesario dividir ese clip en cuatro partes. Para ello se crearán cuatro clips los cuales tienen dos parámetros, el fotograma en el que inicia la animación y el fotograma en el que finaliza la animación respecto al clip inicial (el cual tenía las cuatro animaciones juntas). Teniendo la *Timeline* de referencia de Blender, crearemos clips colocando el fotograma de inicio y el final de cada animación. Los clips creados se llamaron *Attack*, el cual tiene la animación de ataque, *Jump* el cual tiene la animación de salto, *Pick* el cual tiene la animación de recogida de mejora, y *PlayerIdle* el cual tiene la animación predeterminada que se realiza cuando el jugador no esté realizando ninguna otra acción. En el caso del cubo, como solo tenía una única animación no será necesario separar el clip original. A este clip original se le nombró como *Attack*, ya que la única animación que había ahí era la del ataque del enemigo.

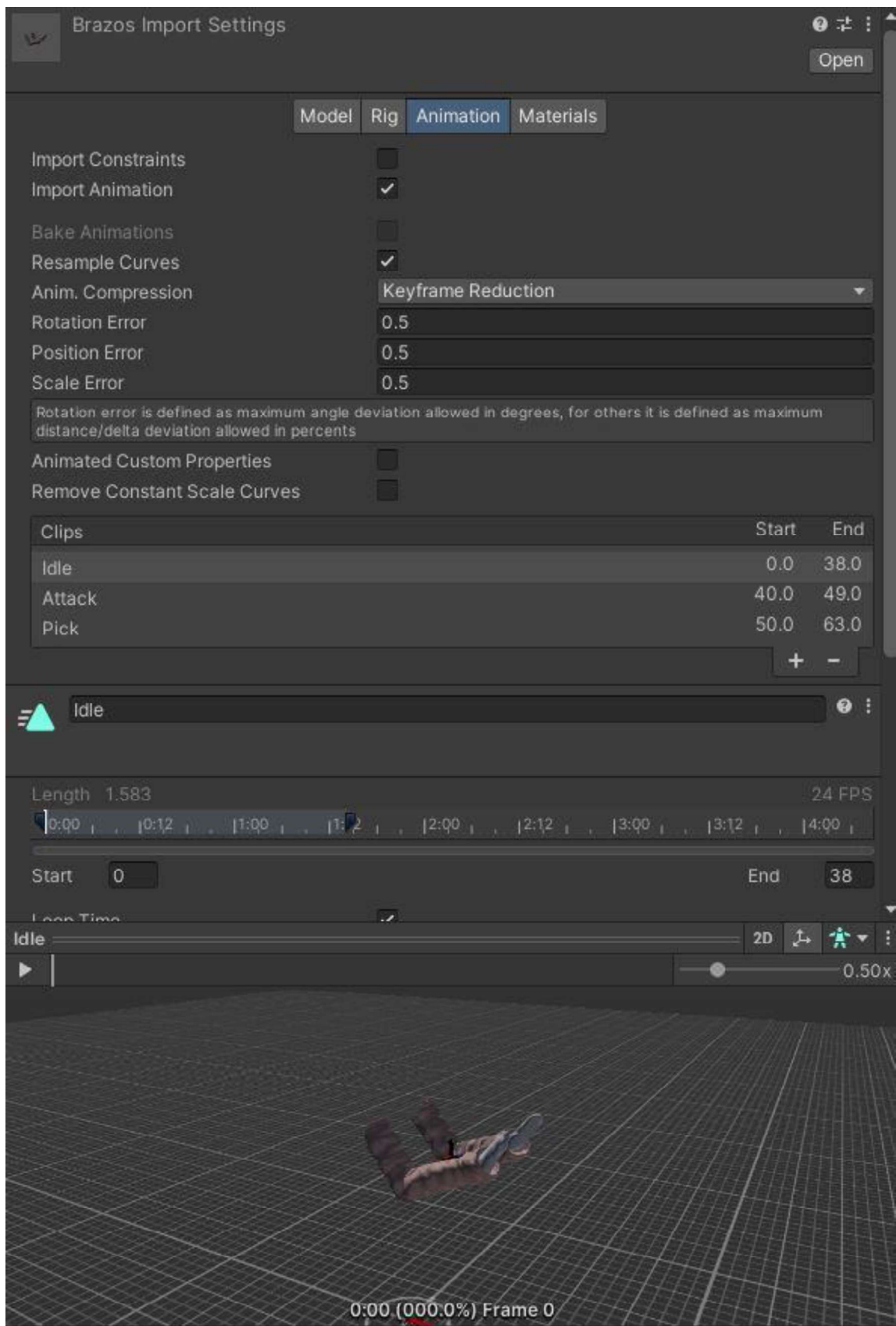


Figura 37: Separación de los clips de los brazos del personaje

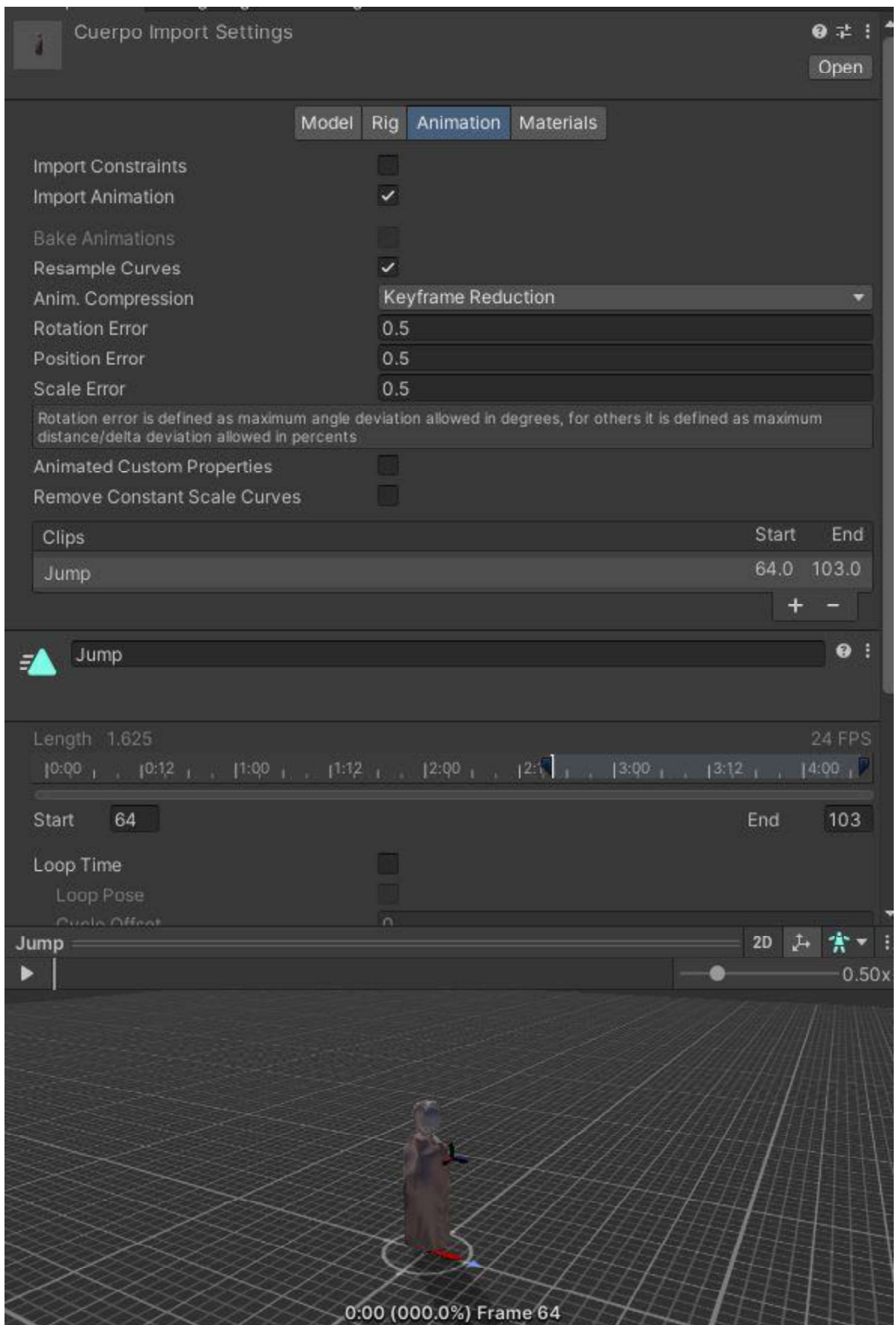


Figura 38: Separación de los clips del cuerpo del personaje

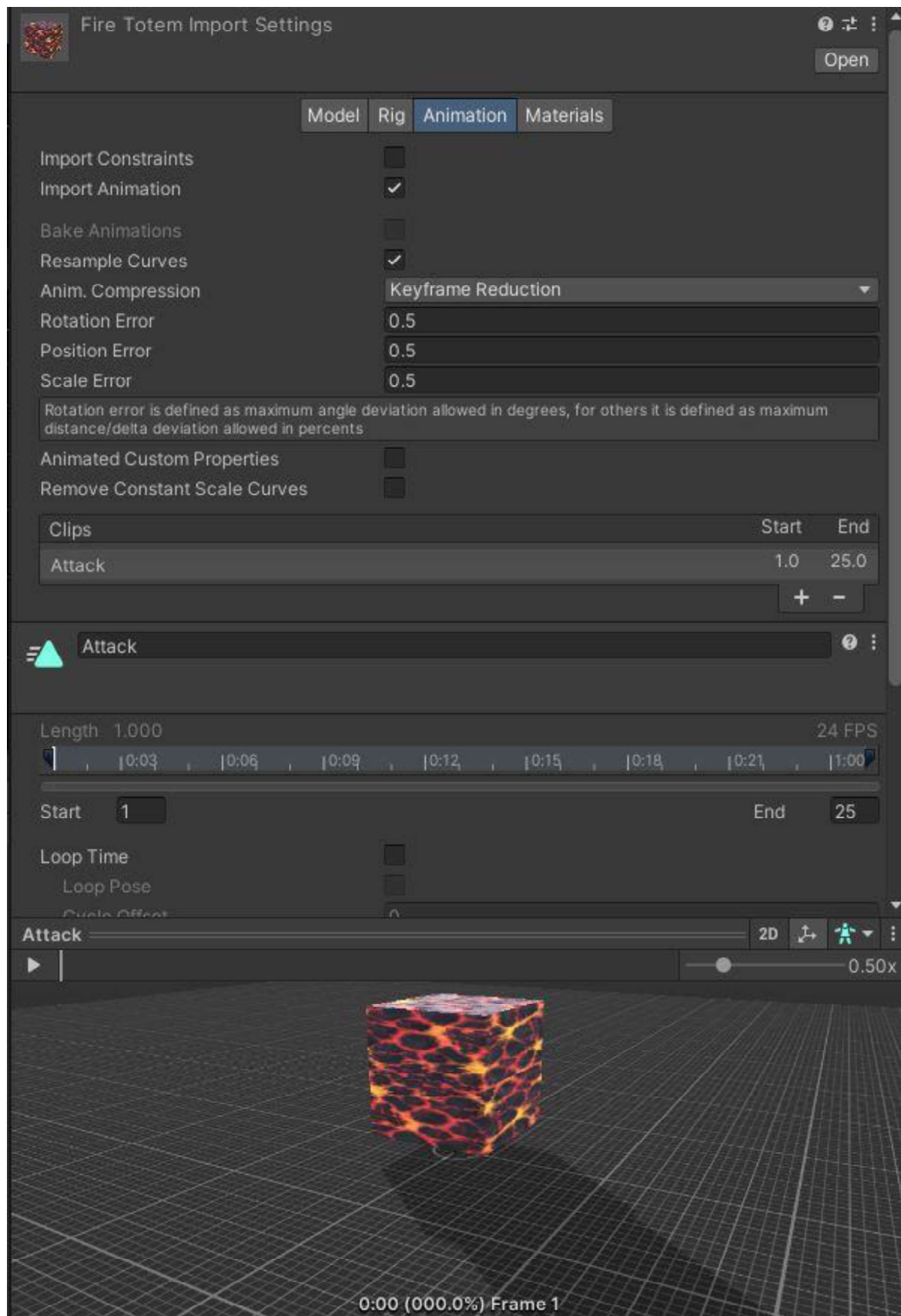


Figura 39: Separación de los clips del modelo del enemigo

Otro problema es que al importar ambos modelos en Unity les coloca un material de color blanco a ambos modelos por defecto. Para que los modelos creados tengan un material, se usó un método para el personaje principal y otro para el enemigo. Para el personaje principal se creó un nuevo material en Unity, se recogió la textura creada en el paso del pintado y se agregó esa textura al material. A este material también se le agregó una imagen que representaba la altura de ese material, dando una impresión de que ese material era tela. Ese material después se le agregó a ambos modelos del personaje principal en Unity y al hacerlo, ya tenía los colores establecidos en la fase de pintado. Para los enemigos se cogieron desde la UnitAssetStore diferentes materiales (los cuales se mencionarán en el apartado 5.5.2) y se agregó a cada cubo un material diferente (uno que representase el fuego, otro el agua, otro la tierra, otro la electricidad y otro diferente para el jefe del piso).

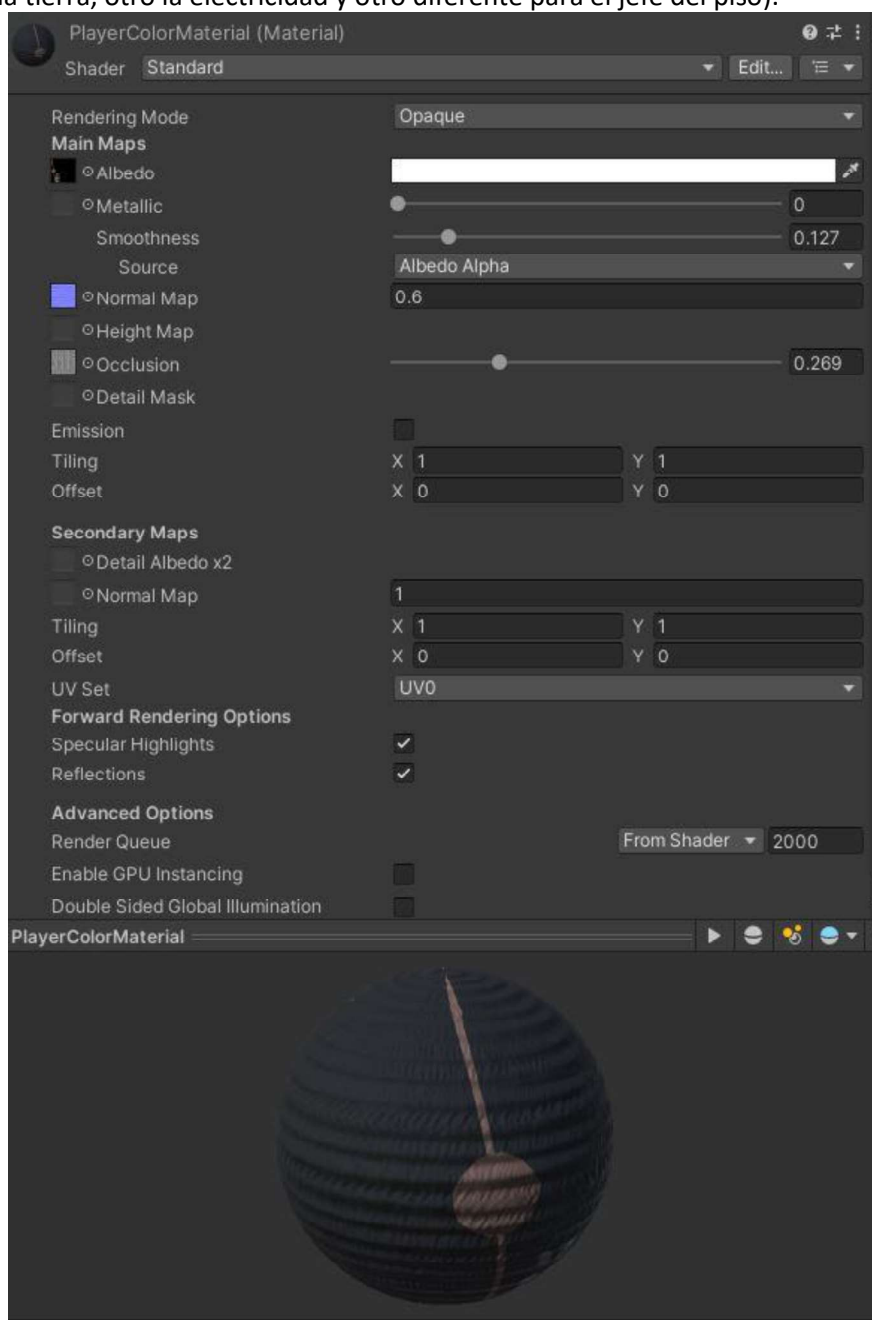


Figura 40: Material usado para los modelos de los brazos y cuerpo del personaje

5.5.2 Materiales

Los materiales, entendiendo como material el componente que da textura al objeto, usados para este proyecto han sido extraídos de la página Unity Asset Store. Los materiales extraídos [30]–[33] pertenecen a los paquetes “Stylized Lava materials”, “Stylize Water Texture”, “Yughues Free Metal Materials” y “50 Free PBR Materials”. Adicionalmente, se han extraído texturas de la página “PolyHaven” con las cuales se ha creado la textura del entorno.

5.5.3 Efectos de partículas

Los efectos de partículas se realizaron en una fase tardía del proyecto. Sin embargo, se ha decidido que, para ayudar a la comprensión de este apartado, era necesario explicar el método empleado para la generación de partículas antes.

Para crear los efectos de partículas en este proyecto, se decidió emplear el *Particle System*, el sistema de generación de partículas que viene en Unity por defecto. Mediante este sistema de partículas se pueden crear todo tipo de efectos visuales, desde llamas, burbujas, electricidad hasta nubes de humo y rocas orbitando en círculos. Estos sistemas se pueden luego añadir a un *GameObject* como si fuese un componente.

En este proyecto se han creado múltiples efectos mediante la creación de múltiples componentes *Particle System*. Para crear diferentes efectos de partículas en el videojuego se han modificado diferentes valores de estos componentes, siendo algunos de estos el tiempo de vida y la velocidad de las partículas generadas, los atributos encargados de la emisión de las partículas, la forma en la que se dispersan estas, el color y tamaño que tendrán en diferentes etapas de su tiempo de vida, la forma de la partícula que se expulsa y la turbulencia que tendrá una partícula entre otros muchos más valores. En la figura se ve un ejemplo de un efecto de partículas:

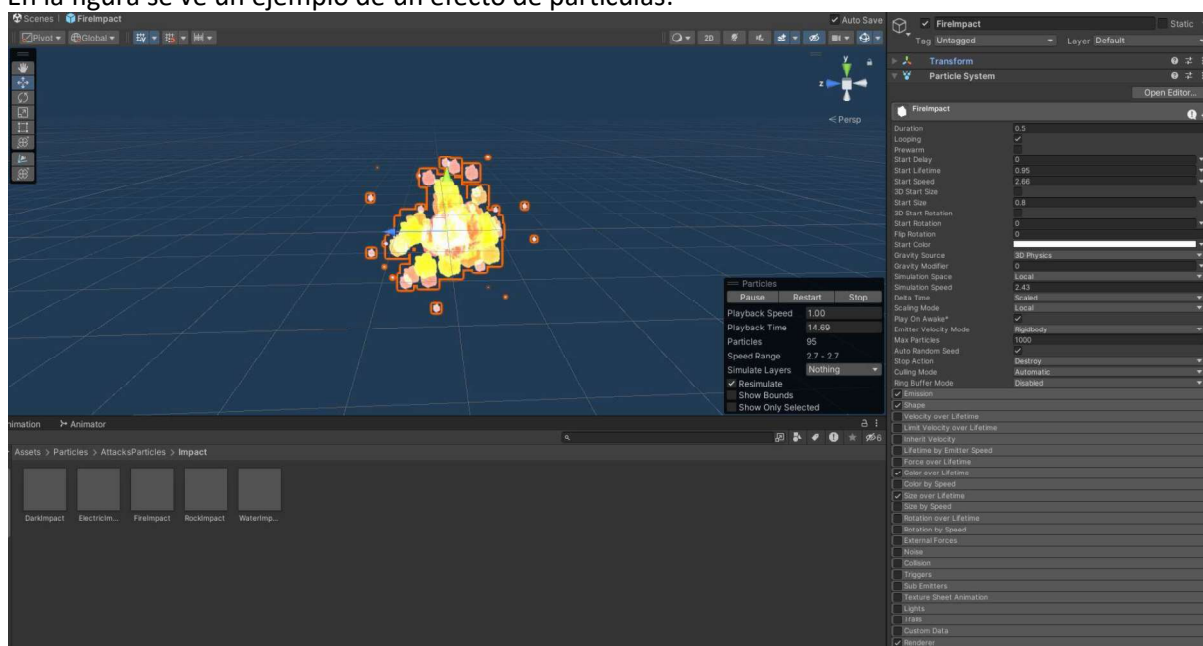


Figura 41: Efecto de partículas de la explosión de un ataque de fuego

Los efectos de partículas creados se pueden agrupar en cinco categorías: efecto de partículas de las cartas mágicas, efectos de partículas de los proyectiles de los enemigos, el efecto de partículas que tienen los enemigos, el efecto de partículas que tienen tanto los proyectiles enemigos como las cartas al impactar y los efectos que tienen las mejoras. En las cuatro primeras categorías hay mínimo un efecto de partículas por cada elemento, y en algunos casos existe un quinto, el cual es exclusivo del jefe final del piso.

5.5.4 Sonido y volumen

Para el sonido de los enemigos y ataques, se ha creado varios componentes *Audio Source* a aquellos componentes que se quería que tuviesen sonido y para la música de fondo del videojuego se ha creado un *GameObject* vacío que solo tiene como componente un *Audio Source* el cual se reproducirá en bucle, para que cuando termine la música vuelva a sonar. Adicionalmente, se ha vinculado estos *Audio Source* a un *Audio Mixer* para que, tal y como mencionaremos posteriormente, desde los menús el jugador pueda cambiar el volumen del videojuego. La música usada para este videojuego fue obtenida en la *UnitAssetStore* la cual se llama *Destructive Force* [34].

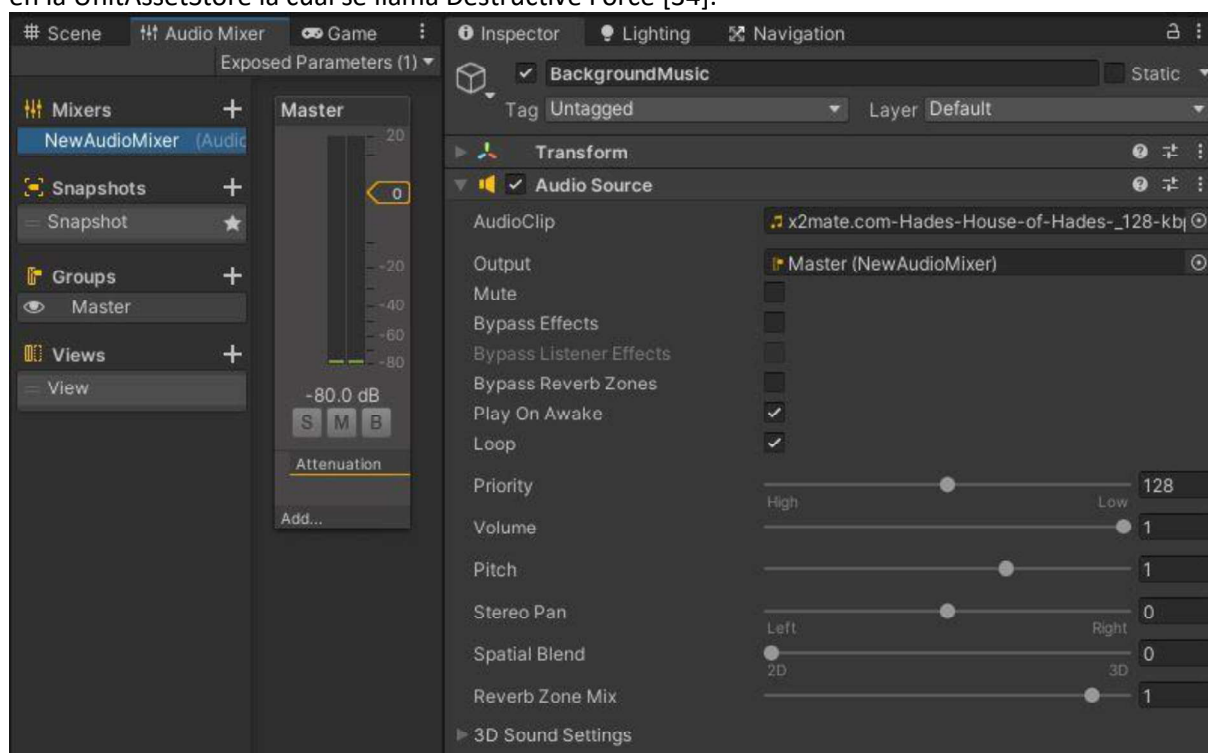


Figura 42: Ejemplo de AudioSource y el Audio Mixer

5.5.5 Personaje principal

Para crear al personaje principal se creó un objeto vacío al cual se llamó *Player*, en el que se agregaron dentro de él varios objetos, los cuales eran la cámara, un objeto llamado *GroundCheck* que nos servirá para detectar cuando el jugador está en el suelo y el modelo del personaje (el cual está compuesto por dos, uno siendo los brazos y el otro siendo el cuerpo).



Figura 43: Componentes del personaje principal

Estos dos modelos al agregarlos al personaje se convirtieron en *GameObjects* y tenían un componente *Animator*. Mediante el uso de este componente, se puede hacer que cuando suceda algún evento en los scripts del personaje, este realice una determinada acción. Para ello había que crear nuevos clips de animación a estos *GameObjects* (ya que los clips creados en el apartado 5.5.1 son solo de lectura y por ende no pueden usarse en el entorno de juego de Unity). Para ello se seleccionará el modelo, se abrirá la ventana de Unity llamada *Animation*, se creará un nuevo clip de animación copiando los fotogramas de uno de los clips en formato de solo lectura y se pegaran al nuevo clip. Esto se repetirá hasta haber creado un clip para cada animación.

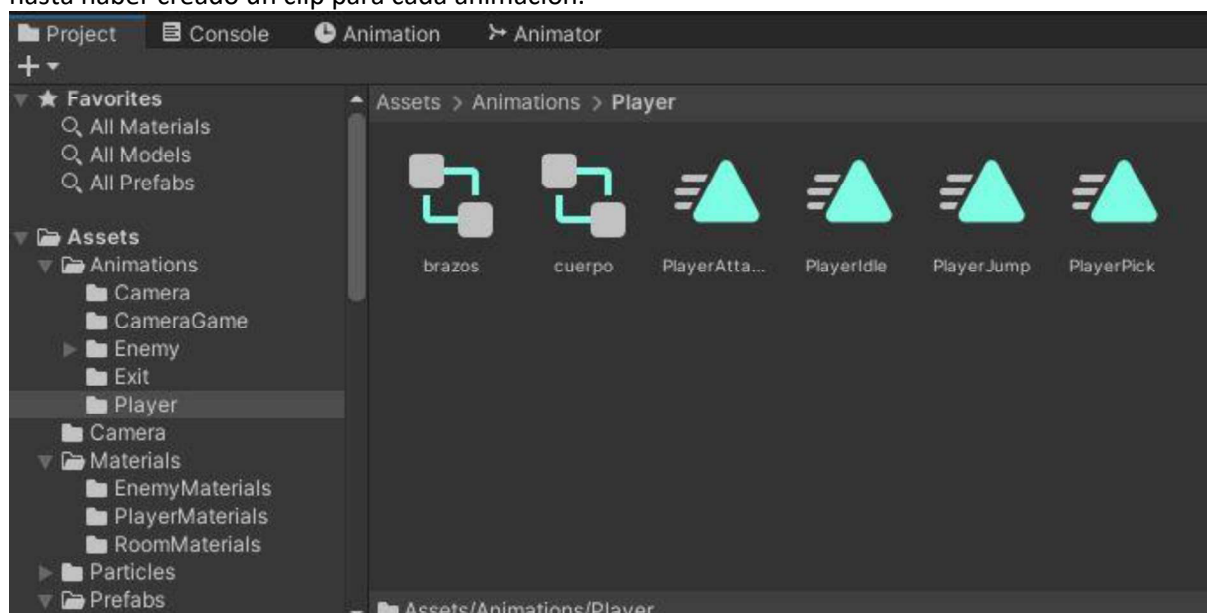


Figura 44: Animator de ambas partes del cuerpo y sus clips

Una vez hecho esto, mediante la ventana de Unity llamada *Animator* crearemos la relación que tendrán las animaciones entre ellas. Por ejemplo, hace falta crear una relación de entre la animación predeterminada con la de ataque para crear una transición natural de una animación a otra. Además de crear relaciones entre animaciones, también se puede definir que animación es la que será predeterminada y por ende se realizará en bucle. En el caso de los brazos será la animación llamada *PlayerIdle* y en el del cuerpo del personaje como no queremos que tenga ninguna predeterminada, asignamos un estado vacío, ya que como es obligatorio que tenga un estado predeterminado, al agregarle uno vacío es como si no tuviese ninguna animación predeterminada. También se puede especificar en qué condiciones una animación se realizará o no, por ejemplo, si el jugador presiona la tecla de salto, el personaje realizará la animación de salto. Las relaciones entre animaciones quedarían tal y como están puesta en las siguientes dos figuras:

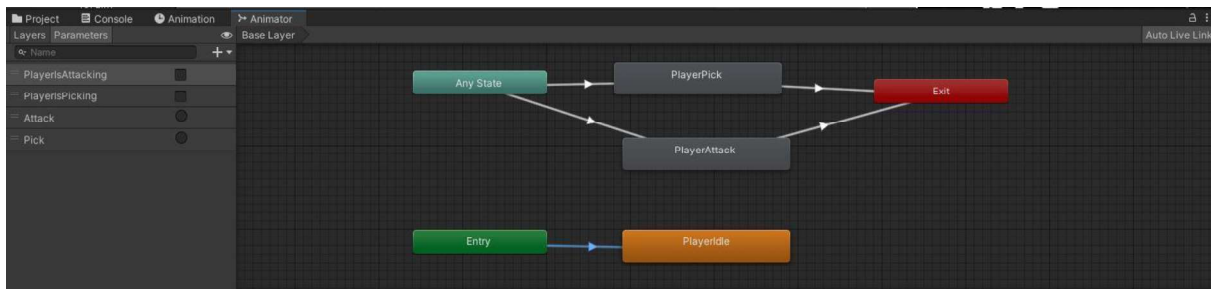


Figura 45: Relación entre animaciones de los brazos del personaje

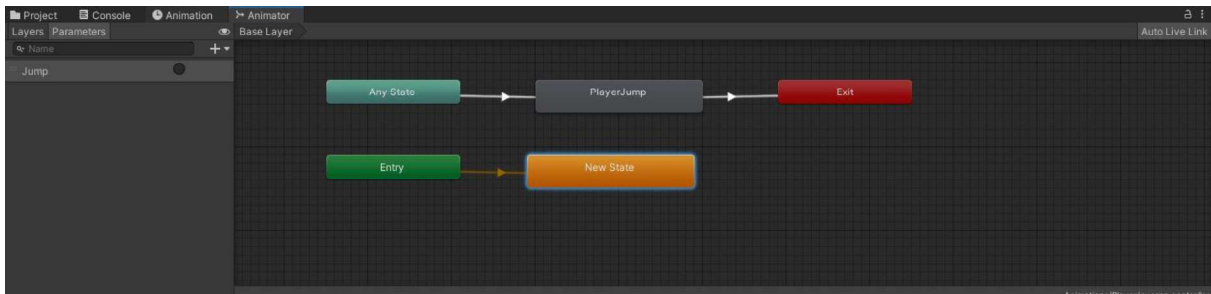


Figura 46: Relación entre animaciones del cuerpo del personaje

Como la cámara principal estaba dentro del objeto `Player`, al moverse y rotar también lo hacía la cámara. En el objeto `Player` se le agregó un *Character Controller*, el cual se ha estado usando tanto para detectar colisiones como para aplicarle fuerzas para realizar el movimiento del personaje. Para la gravedad del personaje se decidió no usar el componente *Rigidbody* e implementar código que simulase la gravedad. Esto se hizo ya que de esta manera se facilitaba el método para la realización del salto. Otro componente que se le agregó era el *Audio Source* para que, al lanzar las *MagicCards*, se genere el sonido del lanzamiento de la carta.

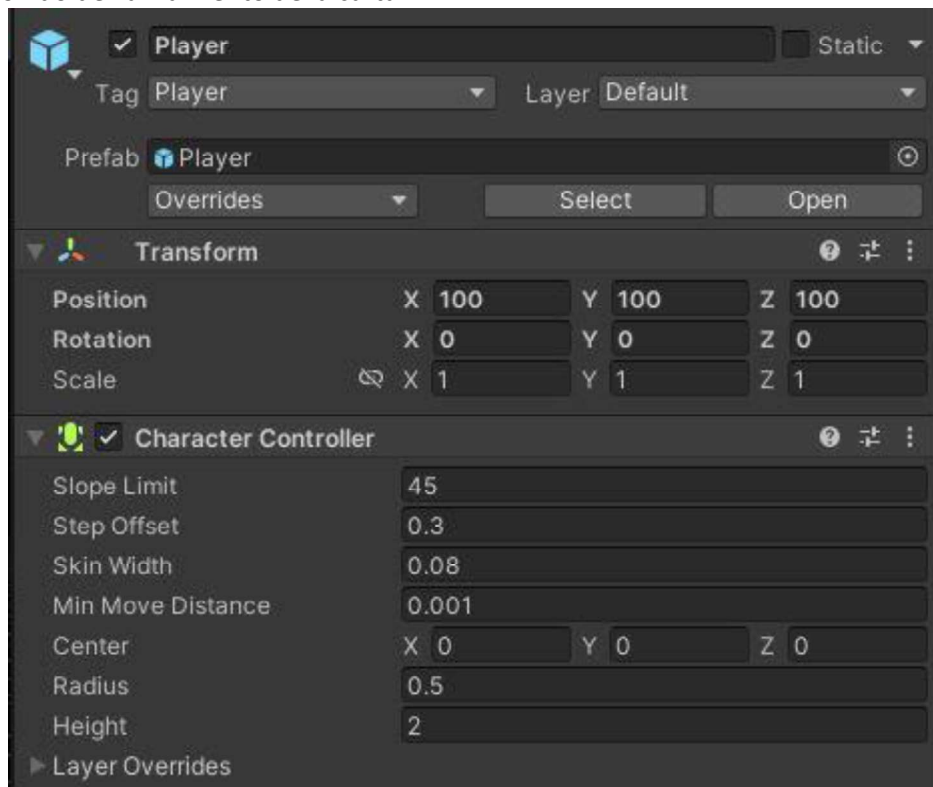


Figura 47: Character Controller del personaje

Además de los componentes mencionados anteriormente, en el objeto `Player` se le han agregado una serie de scripts para que este cumpla con todas sus funcionalidades (los cuales están mencionados en el apartado 5.4). Estos scripts serían los siguientes:

- **MovementBehaviour:** Este script es el encargado no solo de controlar el movimiento básico del personaje usando el componente *Character Controller* agregado al *GameObject*, sino que también es el encargado de realizar las acciones de salto y de embestida mencionadas en apartados anteriores aplicando fuerzas y cambios de velocidad.
- **PlayerBehaviour:** Este script es el encargado de atribuir al personaje del jugador sus estadísticas de combate, además de ser el encargado de todo lo relacionado con la gestión del daño que el jugador sufre por ataques enemigos y detectar cuando la vida del jugador es menor a cero y por ende ha perdido la partida. También es el encargado de la acción de recoger objetos y aumentar las estadísticas en función del objeto recogido. Por último, este script gestiona parte de la interfaz de usuario que representa por pantalla las estadísticas del personaje de manera gráfica.
- **CameraBehaviour:** Este script es el encargado de hacer que el jugador pueda mover la cámara alrededor y que los brazos del personaje también se muevan hacia donde esté mirando la cámara.
- **AttackBehaviour:** Este script es el encargado de dar la capacidad al jugador de arrojar a los enemigos las *MagicCards* y que suene el efecto de sonido del lanzamiento de la carta, además de dar la capacidad al jugador elegir que carta se quiere usar en pleno combate. Este script también tiene un papel importante en la interfaz de usuario que representa la información del personaje por pantalla, ya que gracias a este script el jugador podrá ver que carta está actualmente usando incluso antes de lanzarla.

Pese a que las funcionalidades que contienen los scripts se podían haber introducido en un único script, se ha optado por segmentarlas y de esta manera tener mejor estructurada la jerarquía del proyecto. En las siguientes figuras, se puede observar la estructura final que acabó teniendo el personaje:

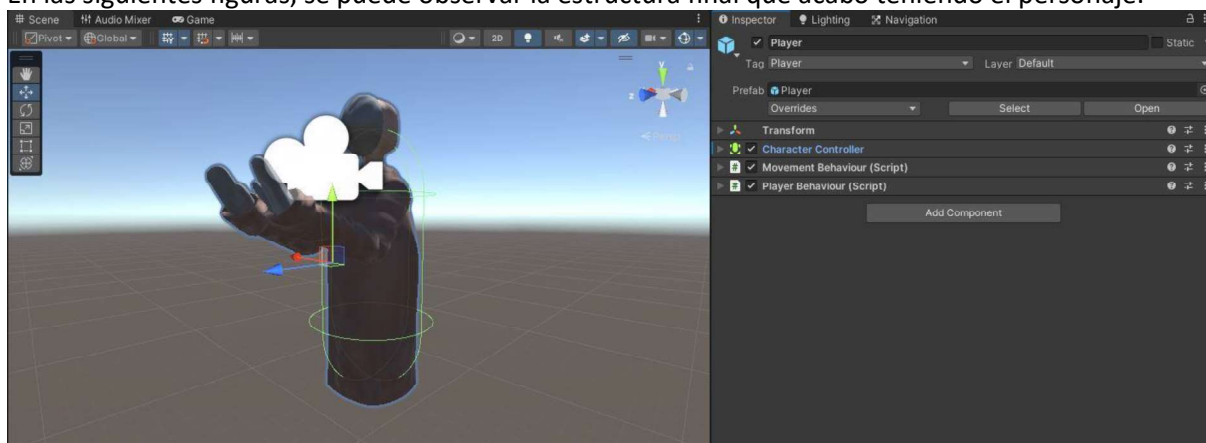


Figura 48: *GameObject del personaje y sus componentes*

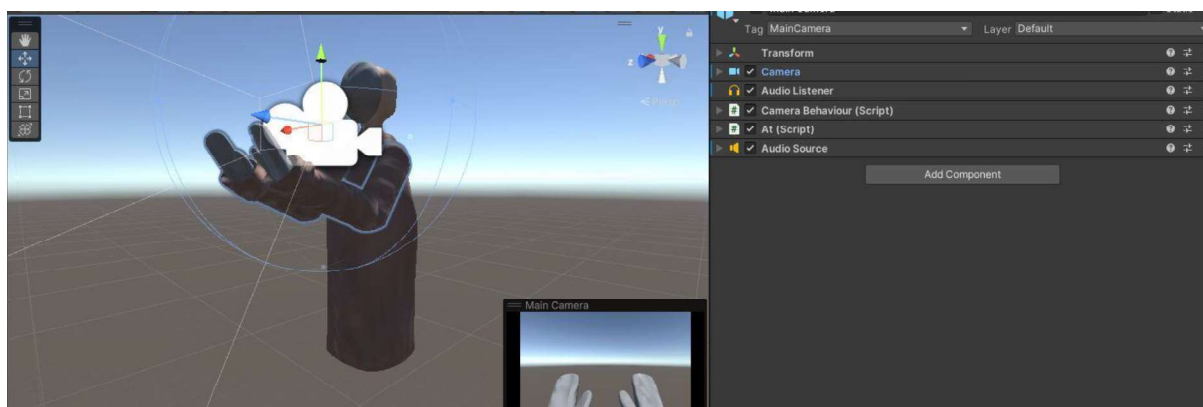


Figura 49: La cámara y sus componentes

5.5.6 Enemigos

En este subapartado se detallará en profundidad el cómo se ha decidido crear a los enemigos de este videojuego, cuáles son sus componentes y cuál es el sistema empleado con el cual ellos siguen al personaje del jugador.

Al tener que crear varios enemigos cada uno con diferencias y similitudes no solo en estadísticas sino también en comportamiento, se decidió crear un script padre llamado `EnemyBehaviour`, el cual contiene todo el comportamiento común que tendrán todos los enemigos, tanto los que se hayan en las salas de combate como los que se hayan en las salas de jefe de piso. Esta clase padre contiene los atributos que todos los enemigos tienen en común, como lo pueden ser la vida que este tiene, el daño que causa, la velocidad de ataque, el tiempo que hay que esperar para que el enemigo vuelva a atacar y las resistencias elementales. Además de esos atributos, este script también contiene la función del cálculo de daño y aplicación de este al recibir un ataque del personaje del jugador y la función que detecta si su vida es inferior a cero y por ende que desaparezca y que dé al jugador una recompensa en forma de oro. Antes de desaparecer se creará una esfera oscura en la posición del enemigo, la cual empezará siendo de tamaño cero, se irá haciendo grande y una vez alcance un tamaño esta volverá a hacerse pequeña para luego destruirse. Esto se hace para enmascarar la desaparición de los enemigos y que así parezca que la esfera los ha absorbido.

A partir de esta clase padre se generarán dos clases hijo. Los enemigos creados a partir de estas clases tendrán diferentes valores en sus estadísticas dependiendo del tipo elemental al que pertenezca el enemigo. Además de las estadísticas estos cuentan con un método que se encarga de la realización de sus ataques. Este método no se ha creado en la clase padre debido a que la manera de realizar los ataques de los enemigos es diferente para cada uno de ellos. Los jefes de piso tienen adicionalmente una función que al destruirse aparecerá una mejora para que el jugador la recoja y el portal mediante el cual el jugador subirá al siguiente piso de la mazmorra. Por ende, se han creado dos scripts hijo, siendo uno de ellos el de los enemigos normales llamado `TotemBehaviour` y el otro siendo el que se le agregará a los jefes del piso llamado `TotemBossBehaviour`. Los enemigos normales se llamarán `TotemEnemy` y los jefes del piso se llamarán `BossTotemEnemy`, tal y como se estipuló en el diseño (apartado 5.4.2).

El método para atacar que tienen ambos scripts es lanzando una esfera hacia la dirección en la que el personaje del jugador esté. Si el enemigo es de fuego la esfera que lanzará será una esfera de fuego y así con el resto de los enemigos. Estos proyectiles son el *GameObject* llamado *SphereAttack* que tiene como componentes un *Sphere Collider*, un material, un *Particle System*, un *Transform* y un script llamado *SphereAttackBehaviour*, el cual controlará el daño que la esfera realizará al impactar en el *Player* y una función que se activará cuando toque algún objeto del escenario o al *Player*. Cuando se active, la esfera desaparecerá y generará un efecto de partículas de explosión.

Para crear cada enemigo se ha creado un objeto vacío con el nombre que se le ha querido atribuir a cada enemigo, los cuales tienen el componente *Collider* el cual nos servirá para detectar colisiones que este reciba, la clase hija del *Enemy Behaviour* correspondiente en función de que enemigo sea, el modelo 3D creado en Blender, un *Audio Source*, un *Particle System*, un componente llamado *NavMeshAgent*, un script llamado *IA* y un componente *Animator*. Los efectos de partículas, para que quedasen mejor, se colocaron no en el objeto completo, sino en cada parte del cubo. En las siguientes figuras se puede ver el cubo con sus componentes:

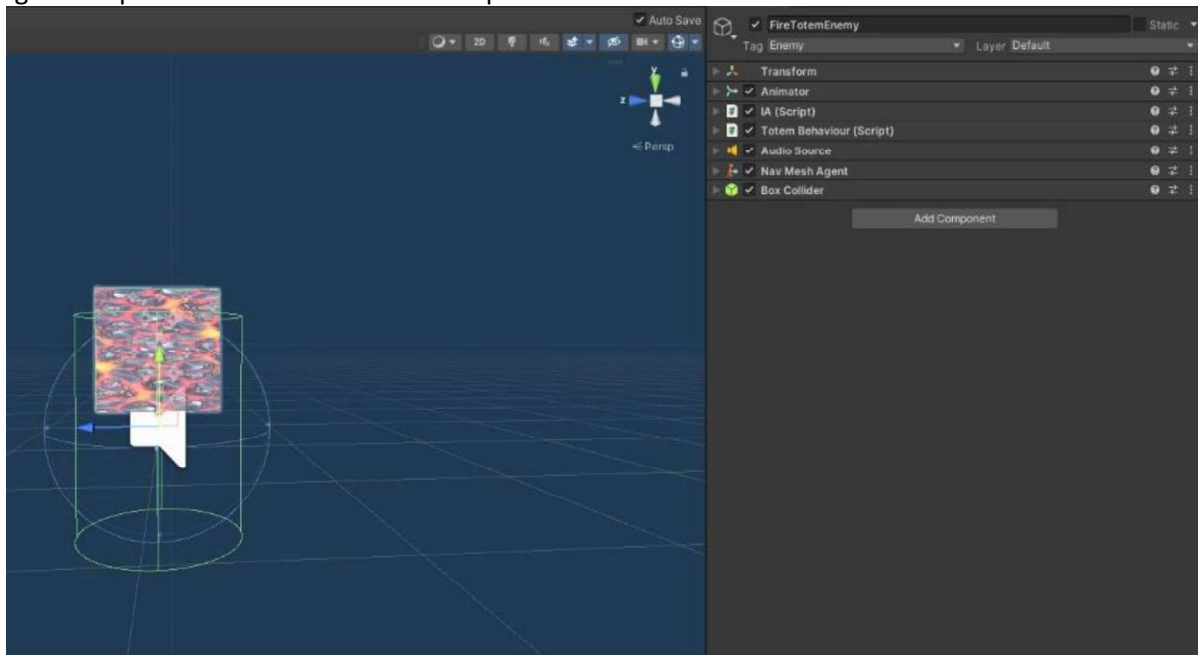


Figura 50: Enemigo tótem de fuego y sus componentes

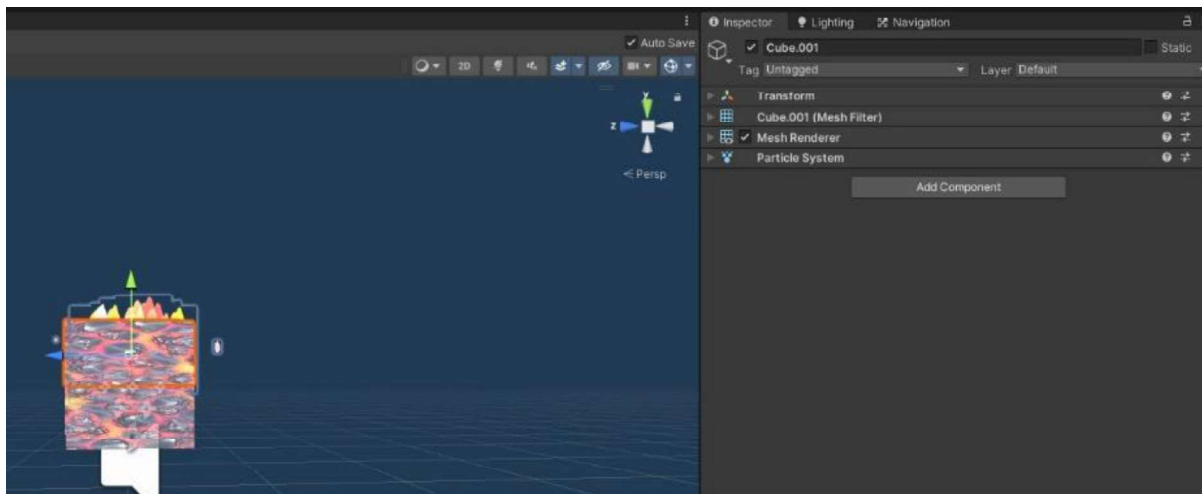


Figura 51: Enemigo tótem de fuego y los componentes de cada parte del cubo

El componente *NavMeshAgent* es un componente el cual aplicará inteligencia artificial a nuestros enemigos, mediante la cual el enemigo se podrá mover de manera autónoma por el entorno. Para que este componente funcione correctamente será necesario crear una malla de navegación. Esta malla de navegación lo que hará será definir las áreas por las cuales los objetos que contengan ese componente de puedan mover. Se explicará más en profundidad sobre la malla de navegación cuando se hable sobre las salas. En este componente se le ha modificado ciertos atributos para aumentar la velocidad del movimiento y la distancia a la cual se tiene que detener el enemigo. En la siguiente figura se puede observar el *NavMeshAgent* de un enemigo:

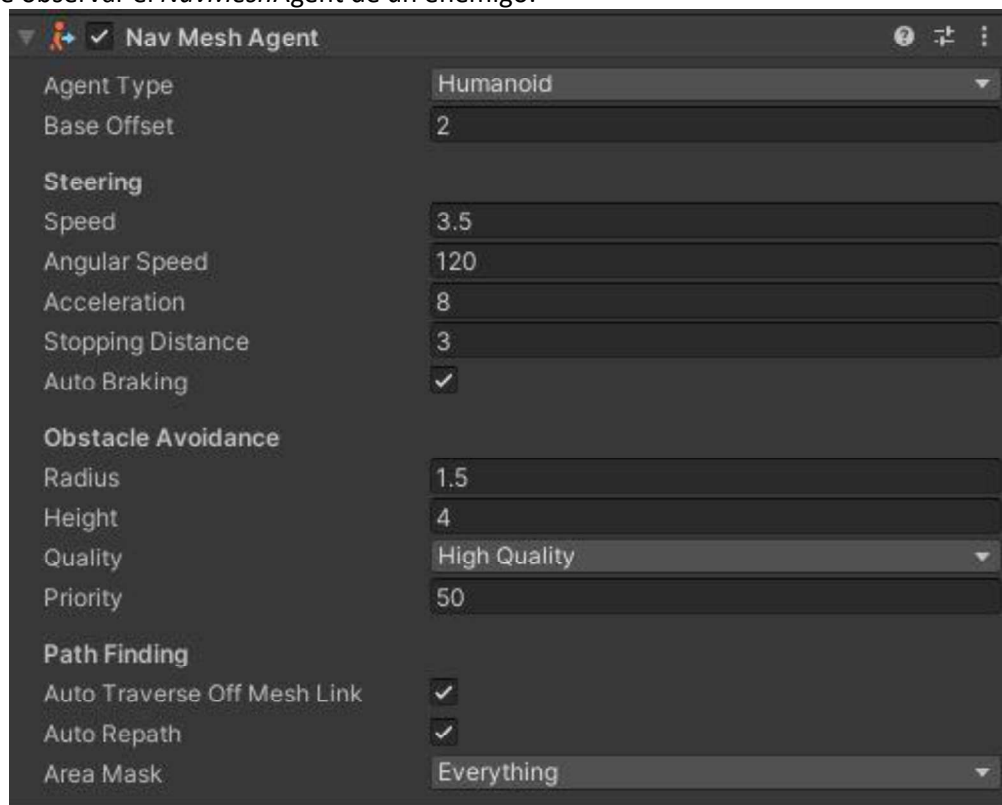


Figura 52: NavMeshAgent del enemigo

Una vez definida la malla, el enemigo se podrá mover por ella a la dirección a la cual tenga marcada como destino. Para hacer que siga al jugador se ha creado y agregado a cada enemigo el script `IA` el cual lo que hace es marcar permanentemente como ubicación destino la posición en la que está el jugador en todo momento. De esta manera, aunque el jugador se mueva, el enemigo le seguirá.

Tal y como se ha mencionado, a los enemigos se les tendrá que agregar un componente *Animator*, para que, al igual que pasaba con el personaje del jugador, al realizar una acción esté realice la animación correspondiente a esa acción. Además de la animación de ataque llamada `Attack` creada en Blender y posteriormente creada en Unity al igual que se ha hecho con el modelo del personaje principal, se ha agregado una nueva animación, mediante la cual el enemigo empezará siendo pequeño y luego se volverá más grande. Esta animación tiene el nombre de `Spawn`. En la siguiente figura se ve como ha quedado las relaciones entre animaciones de los enemigos:

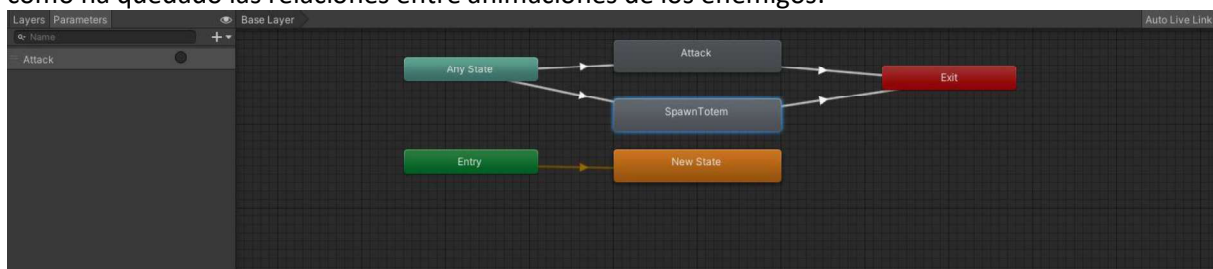


Figura 53: Relación entre animaciones del enemigo

5.5.7 Cartas mágicas

Las cartas mágicas como se comentó en el capítulo 3, son la principal arma que tiene nuestro personaje para realizar daño a los enemigos y existen cuatro tipos de cartas: una de fuego, una de agua, una de rayo y una de tierra. Para crear los *GameObject* de las cartas, se creó un cubo y se le modificó las dimensiones para que pareciese una carta. Luego, se convirtió este *GameObject* a un *Prefab* y se creó cuatro copias (una para cada elemento) y a cada una se le agregó un material diferente en función del elemento. Se usaron materiales y texturas externas para aportar mayor realismo a las cartas (materiales y texturas ya mencionados en el apartado 5.5.2). A estos objetos se les agregó inicialmente un *Box Collider* y un *Rigidbody* sin gravedad para que entre ambos sean capaces de detectar las colisiones. Esto serviría para destruir la carta en el momento que toque cualquier objeto. Además, de estos componentes se les agregó a cada una de las cartas un *Particle System* creados en el apartado 5.5.3 para darles cierto efecto especial a las cartas. Por ejemplo, a la carta de fuego se le agregó un efecto de partículas que simulaba llamas alrededor de la carta.

Una vez estaban creados los objetos, se creó un script llamado `CardBehaviour` el cual contendría todas las características comunes que todas las cartas tendrían entre ellas. Entre estas características se destaca el atributo que determina el daño que hace, el efecto de partículas que creará al impactar y la función que al tocar otro objeto se destruirá, generando antes el efecto de partículas mencionado anteriormente.

Después de crear ese script se crearon cuatro scripts hijos los cuales tendrían como atributos adicionales la cantidad de daño elemental que esa carta realiza y el efecto que tiene la carta, tal y como puede ser el efecto de quemadura, congelación o electrificación. Los nombres de estos scripts son

WaterCardBehaviour, FireCardBehaviour, ElectricCardBehaviour y EarthCardBehaviour.

Otro script que se creó para agregar un efecto vistoso a la carta fue un pequeño script llamado RotateBehaviour. Mediante este script se conseguía que la carta estuviera rotando de manera continua sobre sí misma. De esta manera se simularía el efecto que tendría una carta en la vida real si se lanzase con la mano.

Una vez se crearon esos scripts, se adjuntó a cada carta el script RotateBehaviour, y luego su respectivo hijo del script del CardBehaviour (en función de a que elemento pertenezca cada carta). Luego se le agrego en cada script hijo de CardBehaviour su efecto de partículas de impacto correspondiente (en el caso de EarthCardBehaviour se le agrego un efecto de partículas que generaba una gran nube de polvo por el impacto de la carta). En la siguiente figura se puede observar un ejemplo de una carta mágica y sus componentes:

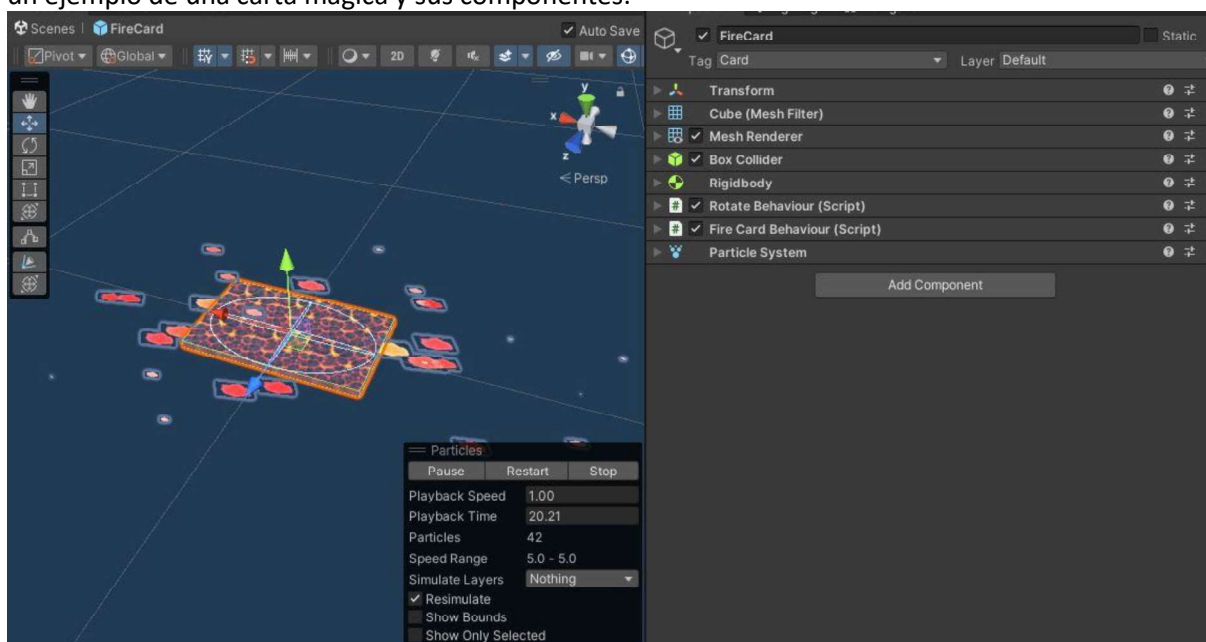


Figura 54: Carta de fuego con todos sus componentes

5.5.8 Mejoras

Como se mencionó anteriormente en este videojuego existen dos tipos de mejoras. Por un lado, están las mejoras que se pueden encontrar en la mazmorra las cuales tienen una forma física y que mejoran los atributos del personaje únicamente en el intento de huida ya sea comprándolas en la sala del mercader con el oro obtenido de los enemigos, obteniéndolas de manera gratuita en la sala de mejora o al derrotar al jefe del piso. Por otro lado, están las mejoras permanentes las cuales se comprar desde el menú principal usando el oro que se haya estado recolectando al terminar una partida.

Para crear las mejoras de la mazmorra, se creó varios *GameObjects* y se les agrego un *Collider* y un script llamado *InGameUpgrade*. Este script almacenará la información de que estadísticas del *Player* mejora ese objeto, cuanto las mejoran y la cantidad de oro que cuestan. Como la idea principal era que todas las mejoras podrían estar tanto en la tienda como en la sala de mejora y por

ende podrán ser gratis o costar oro dependiendo de la aleatoriedad, se decidió no generar una clase hija encargada de las mejoras que se vayan en colocar en la tienda. Como se ha comentado en el punto 5.5.5, es el jugador quien se encarga de aplicar los beneficios que ofrece la mejora a sí mismo y no al revés. En las siguientes figuras se pueden observar una de las mejoras implementadas en el videojuego con todos sus componentes:

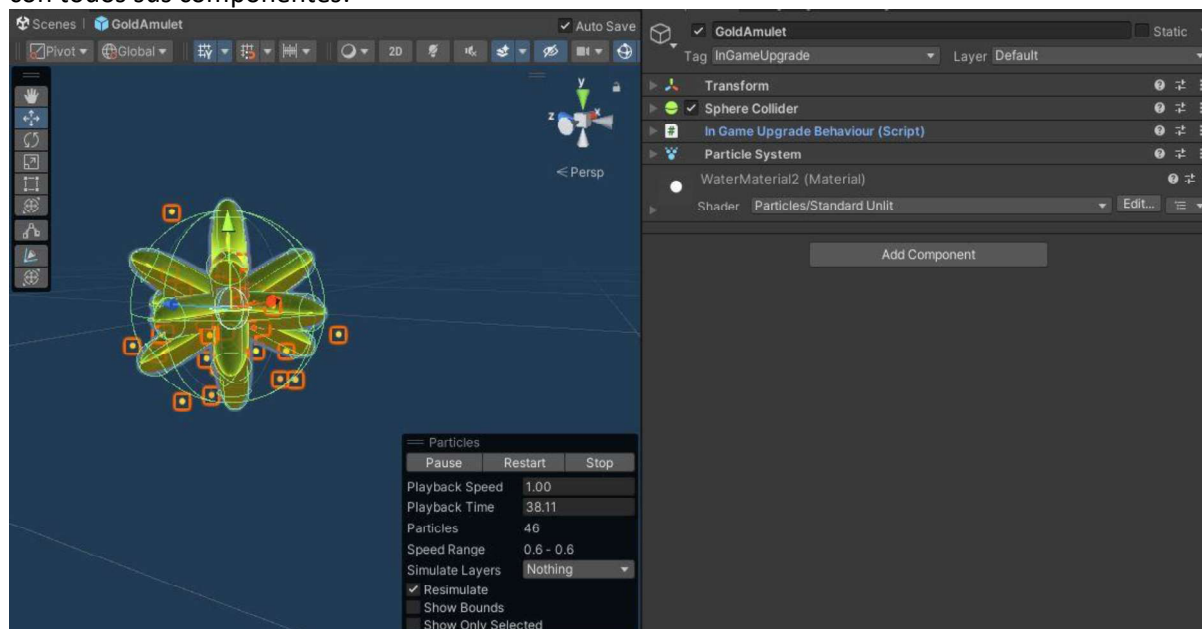


Figura 55: Ejemplo de una mejora con todos sus componentes

Por la parte de las mejoras permanentes, no se ha creado un script ni ningún *GameObject*, ya que como estas mejoras se obtienen a través del menú inicial, lo que se ha hecho para implementar las mejoras permanentes ha sido crear un menú en el que aparezcan las mejoras permanentes a comprar como botones del menú, y que en caso de que el jugador tenga oro para comprar esa mejora y de al botón de la mejora específica el script que controla el menú inicial almacenará en el archivo de guardado que el jugador ha comprado la mejora. Al hacer esto tanto el menú, encargado de proporcionar las mejoras, como el personaje del jugador sabrán que esa mejora ya está activada.

5.5.9 Salas

Las salas de este videojuego son salas cuadradas y cerradas y tienen un máximo de cuatro posibles salidas. Para la creación de estas salas se decidió primero crear de manera individual los objetos por los que iban a poder estar compuestas las salas. En una carpeta se crearon como *Prefabs* todas las puertas y paredes que podía tener, además del techo y el suelo. Todas ellas poseían varios elementos iguales, los cuales eran el *Transform*, el *Collider* y el material del que estaban compuesto. Sin embargo, para hacer que los enemigos, que tenían el componente *NavMeshAgent*, pudieran moverse por la mazmorra era necesario especificarles tanto la superficie por la que ellos podían ir como los obstáculos por los que no se podía pasar. Para ello se agregó a suelo el componente *NavMeshSurface* y al resto de objetos que componen una sala se les agregó el componente *NavMeshObstacle*. Una vez se le haya agregado todos los componentes, se realiza lo que se conoce un *Bake* a los componentes *NavMesh*, para generar los datos necesarios que necesitará el agente para moverse por el mapa. En la siguiente imagen se verá todos los componentes de las salas y la malla generada gracias al *Bake*:

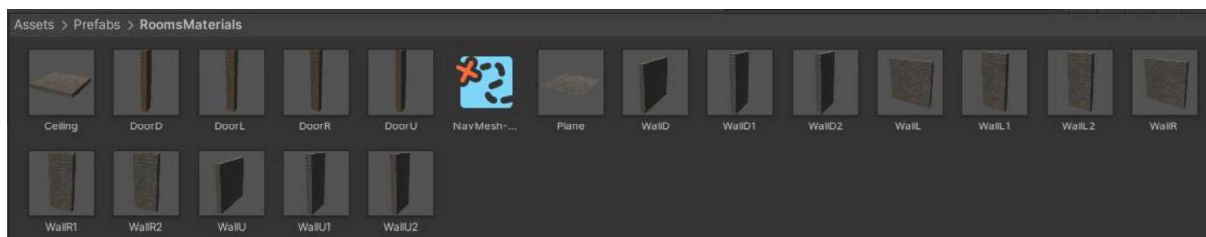


Figura 56: Materiales de la sala

Con los objetos que componen las salas ya creadas se empezaron a crear las salas. En el proyecto se crearon quince salas, una por cada combinación de posibles puertas disponibles en esa sala. A cada sala se le agrego un *Box Collider* y un script llamado *Room*. El script *Room* se encarga de hacer que aparezcan los enemigos o mejoras que hay en esa sala asignada en el momento en el que el jugador toque el *Box Collider* de la sala (cuando el jugador entre en la sala). Además, también es el encargado de cerrar las puertas cuando el jugador entre en esa sala, siempre y cuando sea una sala de combate o de jefe del piso, y de abrir las puertas cuando el jugador haya eliminado a todos los enemigos. Este script también se encargará también de hacer que aparezcan esferas oscuras en la posición de los enemigos y las mejoras con una animación de aparición y desaparición para dar un efecto más vistoso a la aparición de los enemigos y mejoras. En la siguiente imagen se puede ver un ejemplo de una sala con sus componentes:

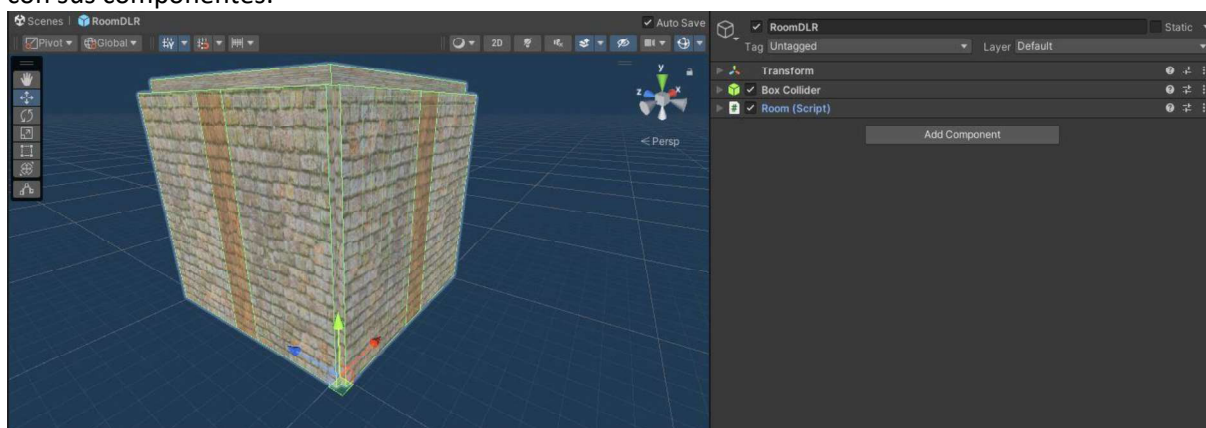


Figura 57: Ejemplo de una sala

Otro objeto que se ha creado es el portal, el cual es una esfera negra que, al tocarla, mediante un script llamado *Stairs* (inicialmente en el diseño iban a ser escaleras, pero se decidió que fuese un portal), pondrá la pantalla del jugador en negro y teletransportará al jugador al siguiente piso. Una vez teletransportado la pantalla dejará de tener el fundido a negro. En la siguiente imagen se puede ver el portal y su estructura de componentes:

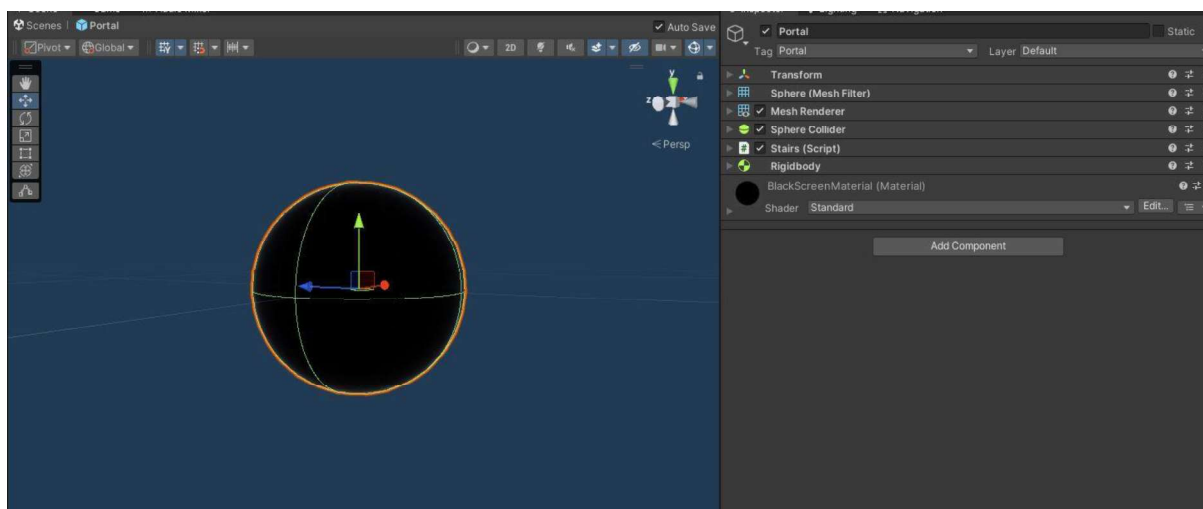


Figura 58: Portal del videojuego y sus componentes

5.5.10 Creador de la mazmorra

En este subapartado se describirá el objeto encargado de la generación procedural de la mazmorra y de todos sus componentes y el proceso de desarrollo de este.

Para la creación de la mazmorra se colocó en la escena un *GameObject* vacío llamado *DungeonCreator*, el cual como único componente tiene un script llamado *DungeonGenerator*. En este script es donde se alojará el algoritmo de generación procedural de la mazmorra, el cual se ejecutará nada más el videojuego comience.

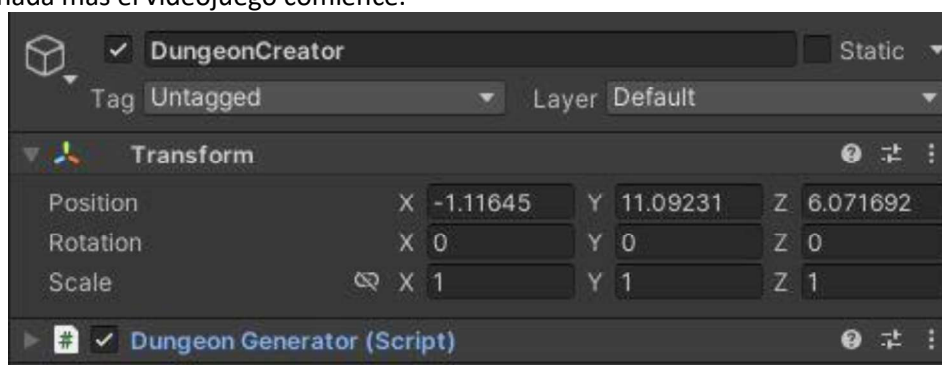


Figura 59: *GameObject* *DungeonCreator* con el script *DungeonGenerator*

Al inicio se investigó todos los algoritmos y métodos de generación procedural más usados y analizar a ver si alguno de ellos encajaba con el diseño que se tenía planeado para la estructura de la mazmorra del videojuego. Tras un largo análisis se decidió usar un algoritmo de generación procedural personalizado. Este algoritmo de generación procedural personalizado se basa en el que algoritmo empleado en el videojuego *The Binding of Isaac* [35].

El algoritmo de generación procedimental creado funciona mediante el uso de la función `Random()` (la que nos aporta `UnityEngine`). La función `Random()` devuelve un numero aleatorio entre la franja de números que nosotros le digamos, sin embargo, pese a lo que pueda parecer, ese número que nos devuelve no es aleatorio. Lo que hace la función `Random()` es usar un valor inicial, mediante el cual la función genera una secuencia de números que simulan aleatoriedad, pero que en realidad siguen

procedimientos para crearlos. Si se usa varias veces un mismo valor en la inicialización del generador de números aleatorio, la secuencia generada será siempre la misma en todos esos intentos. Ese valor inicial con el cual se genera esa secuencia de números pseudoaleatorios se la suele conocer como “semilla”.

En el algoritmo creado, se ha aprovechado de lo anteriormente mencionado para hacer que siempre que se vaya a crear una mazmorra con un parámetro de entrada, que siempre se creé la misma mazmorra si se usa ese mismo valor. Para ello se ha usado una de las subfunciones que tiene la función Random de Unity, la cual es InitState(). Esta función lo que hace es inicializar el estado del generador pseudoaleatorio de números introduciéndole la semilla que nosotros queramos. Haciendo esto, lo que se consigue es que la cadena de números pseudoaleatorios que vaya a generar la función Random() sea la misma, siempre y cuando se use la misma semilla. En caso de que no se introduzca ninguna semilla personalizada, el script generará de manera pseudoaleatoria una semilla nueva. En el videojuego final el jugador no podrá introducir la semilla de manera manual. El uso principal que se le dará a la personalización de la semilla será para crear una demo del videojuego para la defensa usando una semilla en concreto y de esta manera realizar una demo más rápida al conocerme ya el mapa de la mazmorra.

Una vez inicializada la semilla de la función Random(), lo que hará el algoritmo será empezar a crear la mazmorra. El proceso de la generación de los pisos de la mazmorra se basa en una matriz de nueve filas y nueve columnas que se instanciará con ceros y en la cual se colocarán números del uno al cinco. Estos números representarán los tipos de salas que ira en cada una de las posiciones siendo estas las siguiente: 1 para las salas normales, 2 para la sala de mejora, 3 para la sala del mercader, 4 para la sala del jefe del piso y 5 para la sala inicial, siendo el resto de los valores de la matriz serán 0, número que representará las salas vacías. Esta matriz se usará para luego instanciar las salas en el videojuego, usando la matriz para saber en qué coordenadas hay que instanciar las salas del piso de la mazmorra.

Una vez explicada la matriz y sus posibles valores, se explicará el método usado para rellenar esta matriz. A esta matriz llena de ceros se le pondrá un 5 (sala inicial) en el centro de esta y alrededor de manera horizontal y vertical de esta sala se le pondrá 1 (salas de combate). Esto se realiza así para aportar al jugador cuatro posibles rutas a escoger al aparecer en la sala inicial (una por cada sala de combate alrededor de la sala inicial). En la siguiente figura se puede observar la estructura que tendría la matriz al realizar este paso:

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	1	5	1	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Figura 60: Matriz de ejemplo después del primer paso

Una vez realizado el paso anterior, se recorrerá la matriz en busca de una sala, sabiendo que las salas tienen un valor distinto de cero. Una vez encontrado, comprobara las casillas adyacentes verticales y horizontales en busca de alguna que tenga un cero. Una vez encontrada esa casilla, se hará uso de la

función Random() para obtener un numero aleatorio y en función de si el valor de ese número cumple con una condición (si es mayor que cierto valor), se colocará un 1 (sala de combate) en esa posición donde antes había un cero. En caso de que el número no cumpla con la condición se comprobará el resto de las posiciones adyacentes a la posición de la sala que no tenía un cero anteriormente mencionada, y después se seguirá comprobando el resto de las posiciones de la matriz en busca de un valor que no sea cero y repetir el proceso. Esto se realizará en bucle hasta cambiar ocho posiciones que tenían un cero por un uno (salas de combate). En la siguiente figura se puede observar un ejemplo de cómo podría quedar la matriz después de realizar este segundo paso:

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	1	1	1	1	0	0
0	0	1	1	5	1	1	0	0
0	0	0	1	1	0	0	0	0
0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Figura 61: Matriz de ejemplo después del segundo paso

Una vez colocadas toda las salas de enemigos normales y la inicial, solo quedan colocar las salas especiales, las cuales son la sala de mejora (representada con un 2 en la matriz), sala del mercader (representada con un 3 en la matriz) y la sala del jefe del piso (representada con un 4 en la matriz). La colocación de esas salas se hará en ese orden (primero la sala de mejora, luego la sala del mercader y por último la sala del jefe del piso). Lo que se hará a continuación será recorrer la matriz en busca de posiciones con cero como valor y que en sus posiciones verticales y horizontales adyacentes solo haya un valor con un uno. Al encontrar una posición que cumpla con estas condiciones, se colocará en esa posición la sala especial correspondiente según el orden impuesto. Esto se realizará hasta que se hayan colocado todas las salas especiales. En la colocación de las salas especiales no se usará la función Random() para decidir si colocar en esa casilla la sala o no. Esto se ha decidido así debido a que si se implementaba el Random() en este proceso la generación de mazmorras tardaba mucho más tiempo por culpa de las condiciones tan estrictas que tiene la colocación de estas salas. En la siguiente figura se puede observar el resultado final de la matriz generada hasta ahora:

0	0	0	0	0	0	0	0	0
0	0	0	0	2	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	1	1	1	1	3	0
0	4	1	1	5	1	1	0	0
0	0	0	1	1	0	0	0	0
0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Figura 62: Matriz de ejemplo después de finalizar el último paso

Una vez creada la matriz que contiene el mapa de las salas del piso de la mazmorra, se almacenará en una lista de matrices (la cual representa el conjunto de pisos que tiene la mazmorra), y se repetirá hasta crear el número de matrices que se haya definido en el script (si se ha definido que se quiere 20 pisos se crearán 20 matrices), las cuales cada una de ellas representará el mapa de las salas de cada

piso que tendrá la mazmorra. Cabe resaltar que todos los pisos tendrán un mapa de salas completamente diferente entre ellas.

Una vez se hayan creado todas las matrices y se hayan almacenado en la lista de matrices, se empezará a instanciar mediante una función las salas en el entorno de Unity. Esta función lo que hace es recorrer cada matriz en la lista de matrices e ir colocando la sala con las características que le correspondan a ese tipo de matriz. Además de las salas creadas por este algoritmo, en el entorno ya existe una creada previamente, la cual esta apartada del resto de la mazmorra y es la sala final. A esta sala, la cual no tiene enemigos ni mejoras, tiene una puerta que da al exterior, y que al cruzarla el jugador habrá completado la partida. En la siguiente figura se puede observar la forma que tendría la sala final:

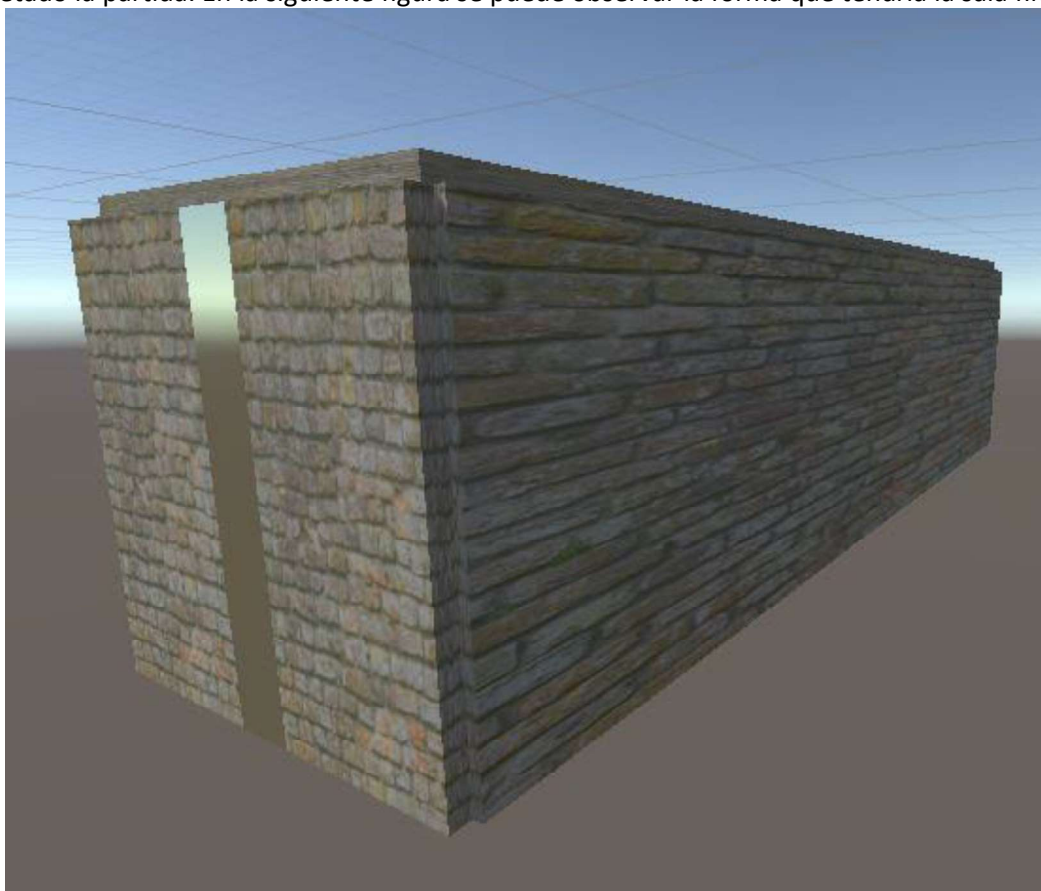


Figura 63: Sala final de la mazmorra

Además de colocar las salas, este algoritmo también se encarga de agregar los enemigos a cada sala de combate haciendo uso también de la función `Random()` para elegir tanto el número de enemigos a generar como para elegir el tipo de enemigo que se quiere instanciar y de potenciar las estadísticas de los enemigos en función de en qué piso estén (cuanto más alto el piso más poderosos serán los enemigos). También se encargará de hacer que los portales generados al matar al enemigo del piso teletransporten al jugador a la sala inicial del siguiente piso.

En las siguientes figuras se pueden observar la estructura de la mazmorra generada por este algoritmo desde diferentes perspectivas:

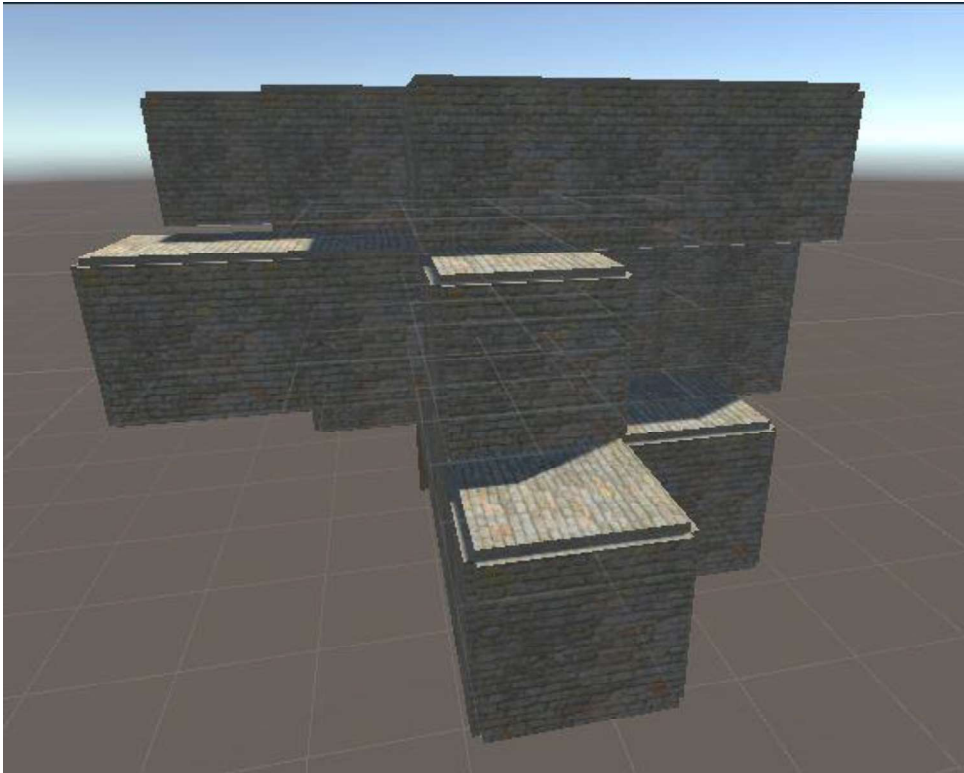


Figura 64: Primera imagen de una mazmorra generada de manera procedimental

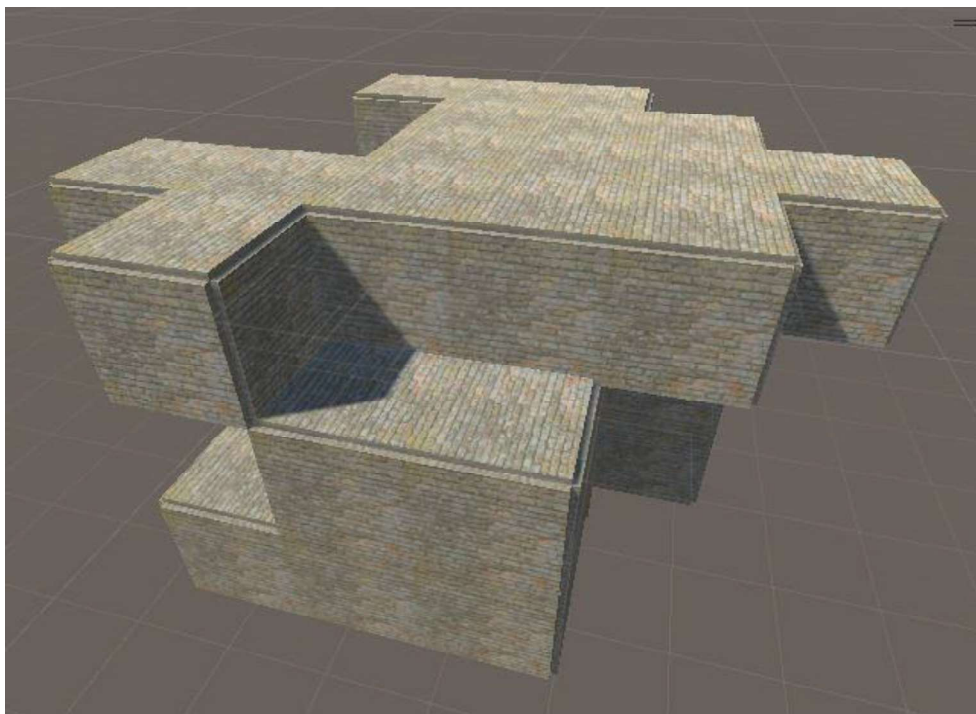


Figura 65: Segunda imagen de una mazmorra generada de manera procedimental

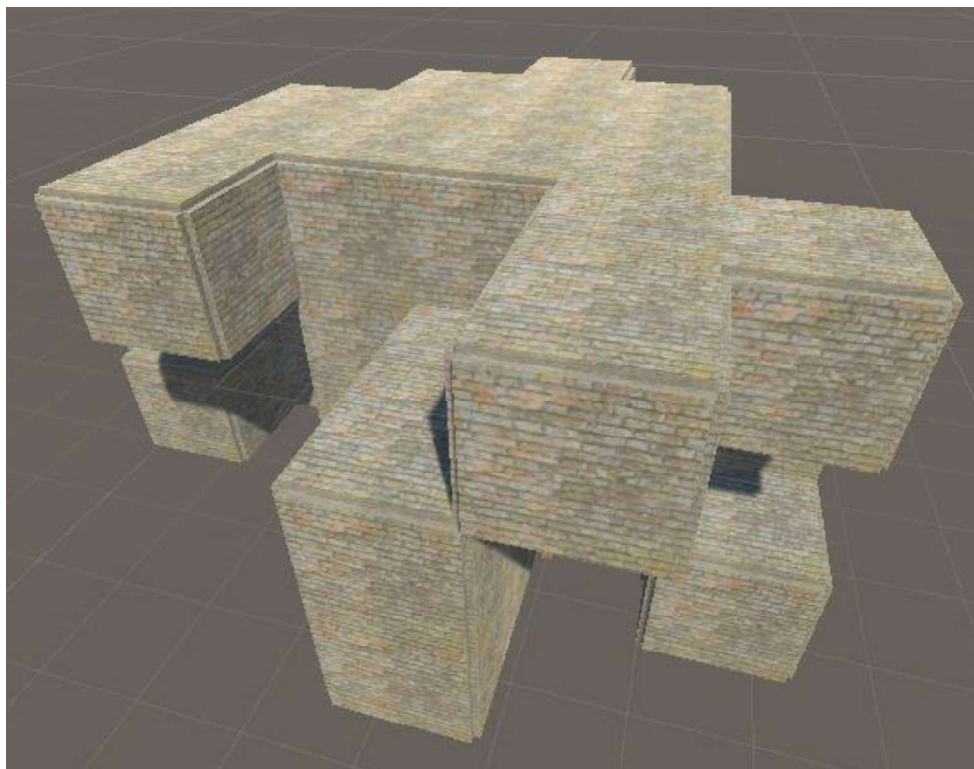


Figura 66: Tercera imagen de una mazmorra generada de manera procedimental

5.5.11 Sistema de guardado

Mediante el sistema de guardado implementado en este videojuego, se guardará la información importante de algunos datos importantes del jugador, como lo son las mejoras permanentes que este posea como la cantidad de oro total que este tenga acumulado. El sistema de guardado implementado es un sistema de guardado mediante un archivo binario. El motivo de porque se ha decidido implementar un sistema de guardado binario en vez de un sistema de guardado con JSON, es debido a que un sistema de binario es considerablemente más difícil de modificar que un JSON. Esto es bastante importante en este videojuego, ya que, si fuese fácil modificar el archivo de guardado, cualquier jugador podría modificar sus datos y obtener todas las mejoras e infinito oro aun no habiendo jugado ninguna partida.

Para realizar este sistema de guardado automático, se ha creado un script llamado `SaveSystem`, mediante el cual se podrá tanto almacenar la información deseada del jugador en un archivo binario, como extraer esa información para que otros scripts las usen.

5.5.12 Interfaz del usuario

Para que el videojuego sea más interactivo se ha diseñado varias interfaces de usuario. Empezando por la que se verá nada más empezar al jugar al videojuego, tenemos a la interfaz del menú inicial. En esta interfaz el jugador podrá crear una nueva partida borrando toda la información del videojuego anterior, podrá jugar una nueva partida conservando el oro total acumulado y las mejoras permanentes obtenidas, podrá abrir el menú de opciones o podrá cerrar el videojuego dándole al de salir. Al darle al menú de opciones, se cambiará la UI y mostrará nuevos elementos en la interfaz. En

las opciones de este menú el jugador puede modificar el volumen general del juego, en caso de que el jugador quiera bajarlo o subirlo. Dándole al botón de atrás la interfaz de usuario volverá a ser como era antes. Al dar al botón de continuar, al jugador le aparecerá nuevos elementos, siendo estos el oro acumulado y las mejoras disponibles. En esta ventana el jugador puede comprar las mejoras permanentes que él quiera. Una vez haya terminado de realizar sus compras, el jugador podrá empezar una nueva partida. El siguiente diagrama de flujo muestra de manera clara, todo lo que se puede hacer en este menú:

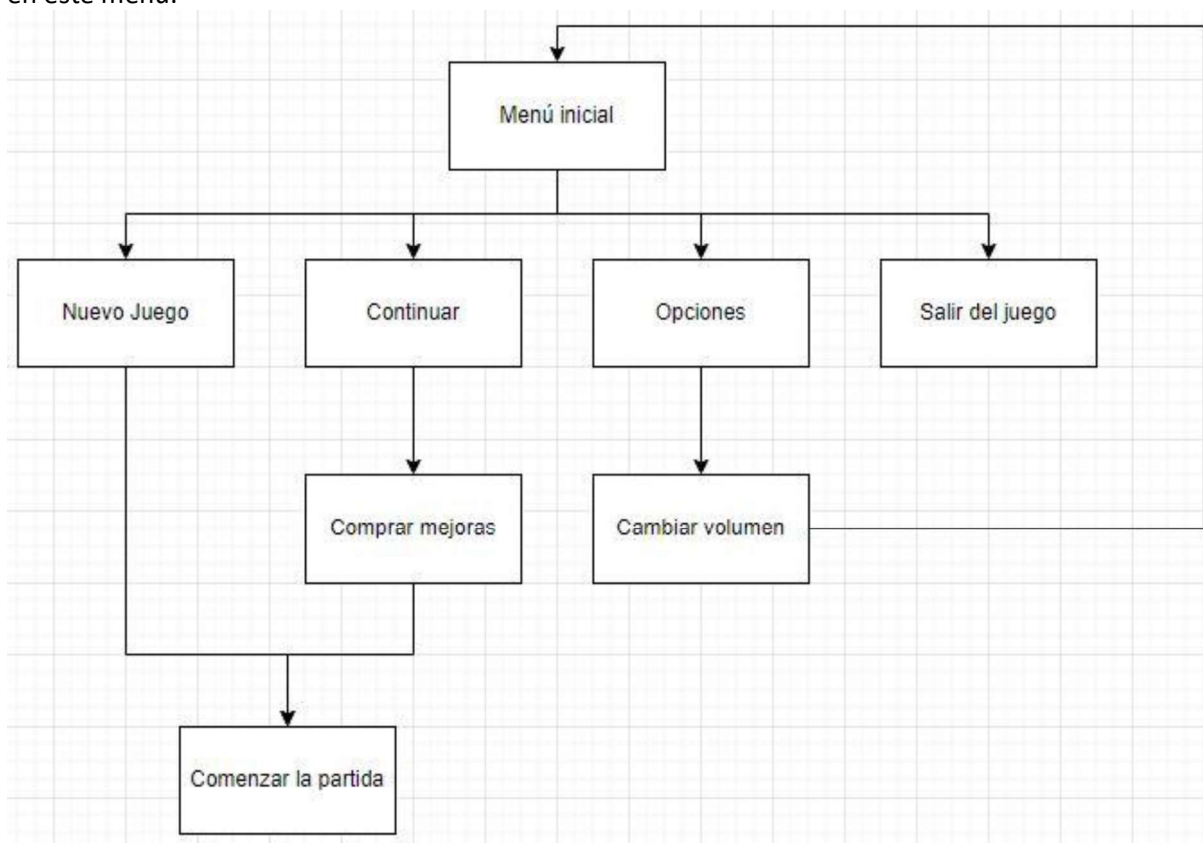


Figura 67: Diagrama de flujo del menú inicial

El resto de las interfaces de usuario se encuentran en la escena encargada de la jugabilidad del videojuego. Una de ellas es el menú de pausa, mediante la cual el jugador puede detener el tiempo de juego y reanudarlo cuando este lo desee. Además, también el jugador podrá cambiar el volumen del videojuego desde este menú. Por último, el jugador podrá salir y terminar la partida para volver al menú inicial mencionado anteriormente. El diagrama de flujo de este menú es tal y como se muestra en la siguiente figura:

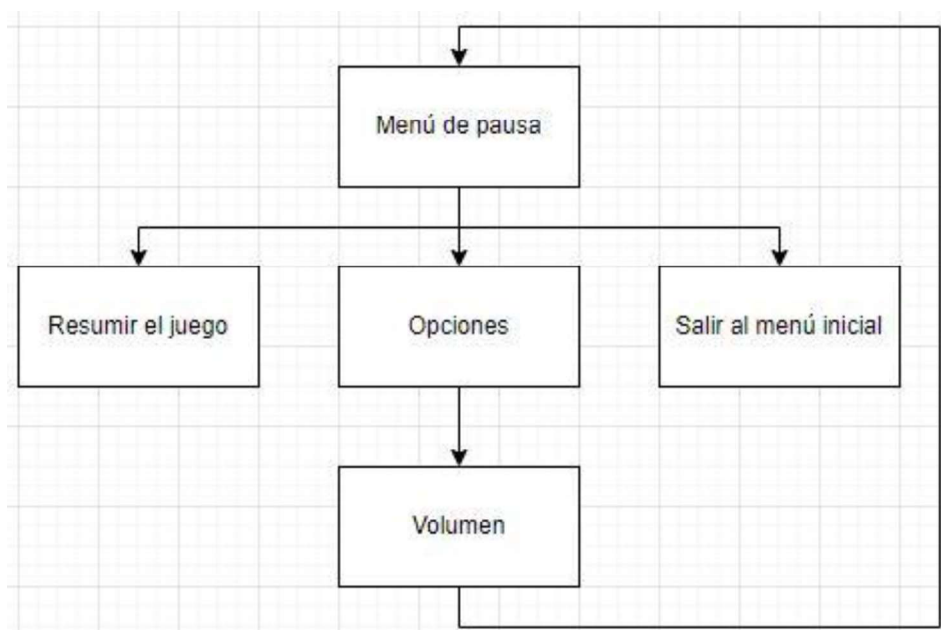


Figura 68: Diagrama de flujo del menú de pausa

Otra de las interfaces de usuario es la que estará presente a lo largo de la partida del jugador, la cual mostrará de manera permanente varios elementos relacionados con el jugador, siendo estos la cantidad de vida y la cantidad de oro que tiene en ese momento, y a la izquierda los cuatro elementos de las cartas, señalando con un color diferente la carta que en ese momento el jugador tenga equipada.

Las últimas interfaces de usuario son las que aparecerán cuando el jugador haya perdido o ganado una partida. Cuando el jugador consiga llegar a la salida, la pantalla se tornará blanca y aparecerá el mensaje "You have escaped!", para posteriormente devolverte al menú inicial. Cuando la salud del jugador sea cero o menos, indicador de que ha perdido, se mostrará por pantalla un mensaje que dice "Game Over" con una serie de botones. Cada uno de los botones tiene una funcionalidad diferente. Dando al botón de *Retry* el jugador volverá a comenzar de manera automática una nueva partida, dando al botón *Menu* el jugador volverá al menú inicial, y dando al botón *Exit*, el jugador saldrá del juego y este se cerrará.

Cabe resaltar que en cada interfaz de usuario se han agregado animaciones para causar una mayor y mejor impresión en el jugador. Algún ejemplo de estas animaciones puede ser el fundido en negro al salir del menú inicial o la transición que hay cuando el jugador consigue la victoria.

6. MANUAL DE USUARIO

En este capítulo se realizará a modo de guía para el jugador una explicación de como moverse por los menús, de los controles del videojuego y el aspecto visual del videojuego.

6.1 INICIAR EL JUEGO

Al iniciar el juego el videojuego aparecerá con la pantalla en negro, y poco a poco empezara a aparecer el menú inicial, el cual tiene cuatro opciones, tal y como se puede ver en la siguiente figura:



Figura 69: Menú inicial

El jugador puede presionar cualquiera de los botones, los cuales tendrán un efecto diferente. Al pulsar *Exit*, se creará un fundido en negro y el juego se cerrará automáticamente.

Al presionar *Options* los elementos del menú desaparecerán paulatinamente y aparecerán otros. Estos nuevos elementos, tal y como se ve en la siguiente imagen, servirán para modificar el volumen general del juego o volver de nuevo al menú inicial.

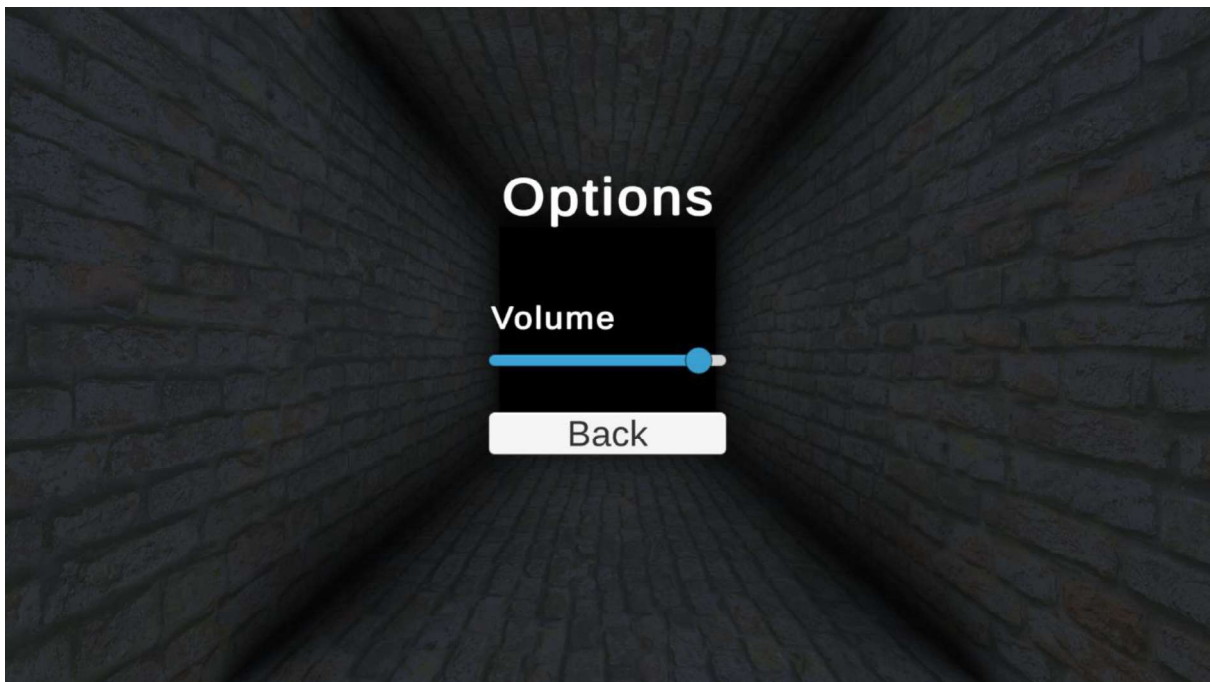


Figura 70: Menú de opciones del menú inicial

Al presionar el botón de *New Game*, el juego borrará todos los datos previos del jugador y comenzará la partida.

Dando a *Continue*, no se borrarán los datos previos y podrá jugar una nueva partida. Antes de jugar, la interfaz del menú volverá a cambiar y mostrará el oro total que tiene ese jugador y las mejoras que puede comprar al darle al botón de la respectiva mejora. Una vez que el jugador esté listo, podrá darle al botón *Play* y jugar la partida. La siguiente figura ilustra el menú de mejoras permanentes:

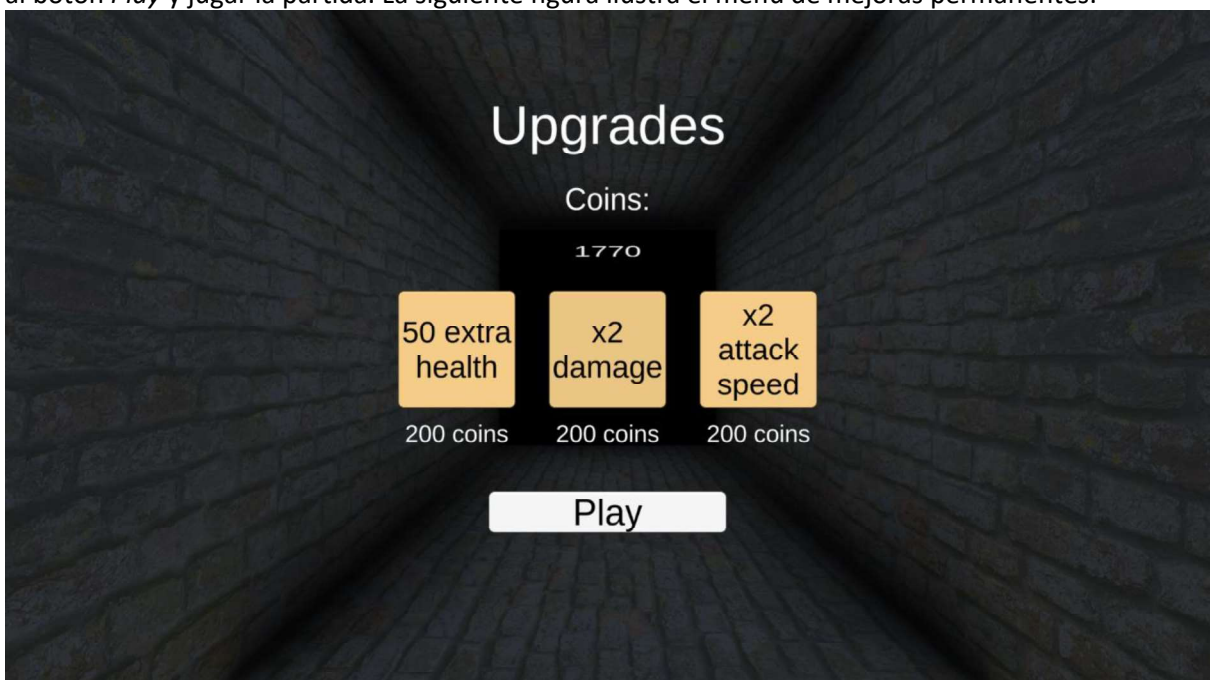


Figura 71: Menú de mejoras permanentes

Independientemente de que el jugador juegue una partida dándole al *NewGame* o mediante el *Continue*, en ambos casos sucederá un fundido paulatino a negro, el cual se quitará al de pocos segundos mostrando al jugador ya en la mazmorra.

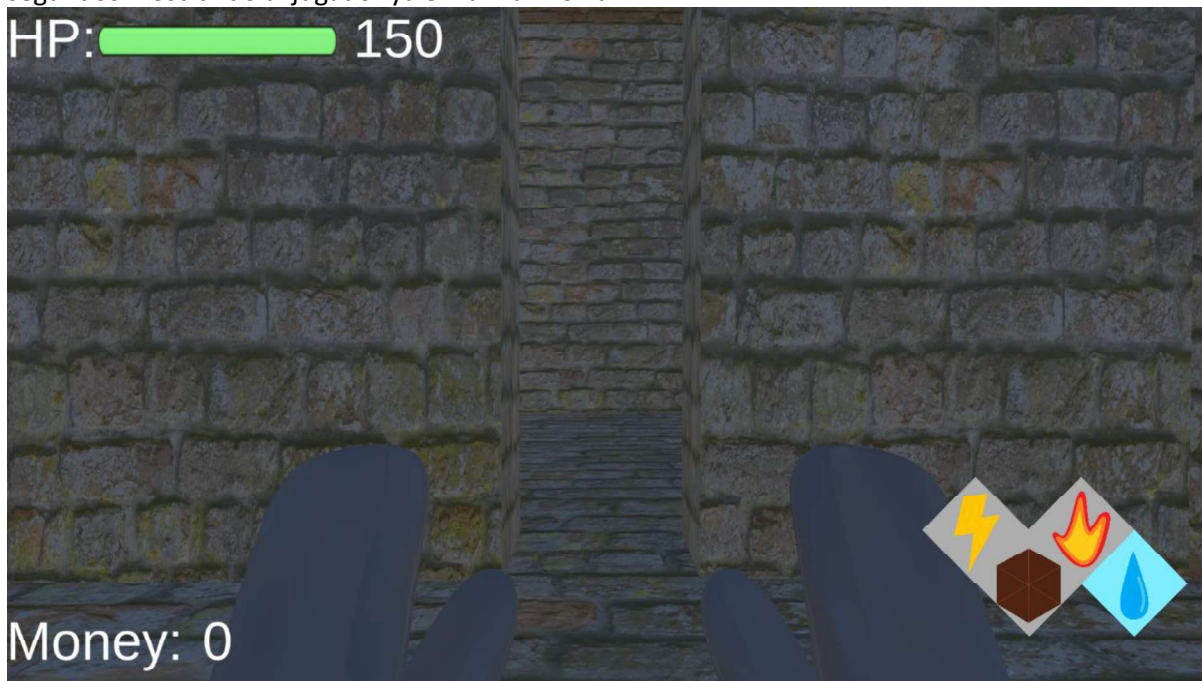


Figura 72: Interfaz del personaje del jugador

6.2 INTERFAZ DEL JUEGO

En la partida, el jugador verá siempre 3 elementos en la pantalla, los cuales estarán ahí siempre. Estos son:

- La cantidad de vida que tiene actualmente el jugador, la cual al recibir un golpe ira bajando tanto el número de salud como la barra verde, dejando una parte rojiza en su lugar tal y como se puede ver en la siguiente imagen:



Figura 73: Interfaz del usuario - Cantidad de vida

- La cantidad de oro, el cual a medida que lo vaya consiguiendo o gastando para comprar mejoras, ira aumentando o reduciendo respectivamente, tal y como se puede ver en la imagen:

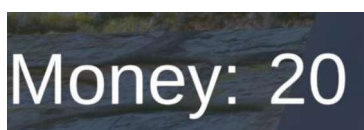


Figura 74: Interfaz del usuario - Cantidad de oro

- Los elementos de las cartas elementales, de los cuales se marcará de un color diferente aquel elemento que el jugador tenga equipado en ese momento, tal y como se ve en la imagen:



Figura 75: Interfaz del usuario - Elementos y elemento agua seleccionado



Figura 76: Interfaz del usuario - Elementos y elemento fuego seleccionado



Figura 77: Interfaz del usuario - Elementos y elemento tierra seleccionado



Figura 78: Interfaz del usuario - Elementos y elemento rayo seleccionado

Además de esta interfaz, existe el menú de pausa, el cual tal y como se ve en la siguiente imagen, tiene varias opciones, siendo una de ellas el continuar jugado al darla al botón *Continue*, abrir las opciones con el botón *Options* y cerrar el juego al darle al botón *Exit*:

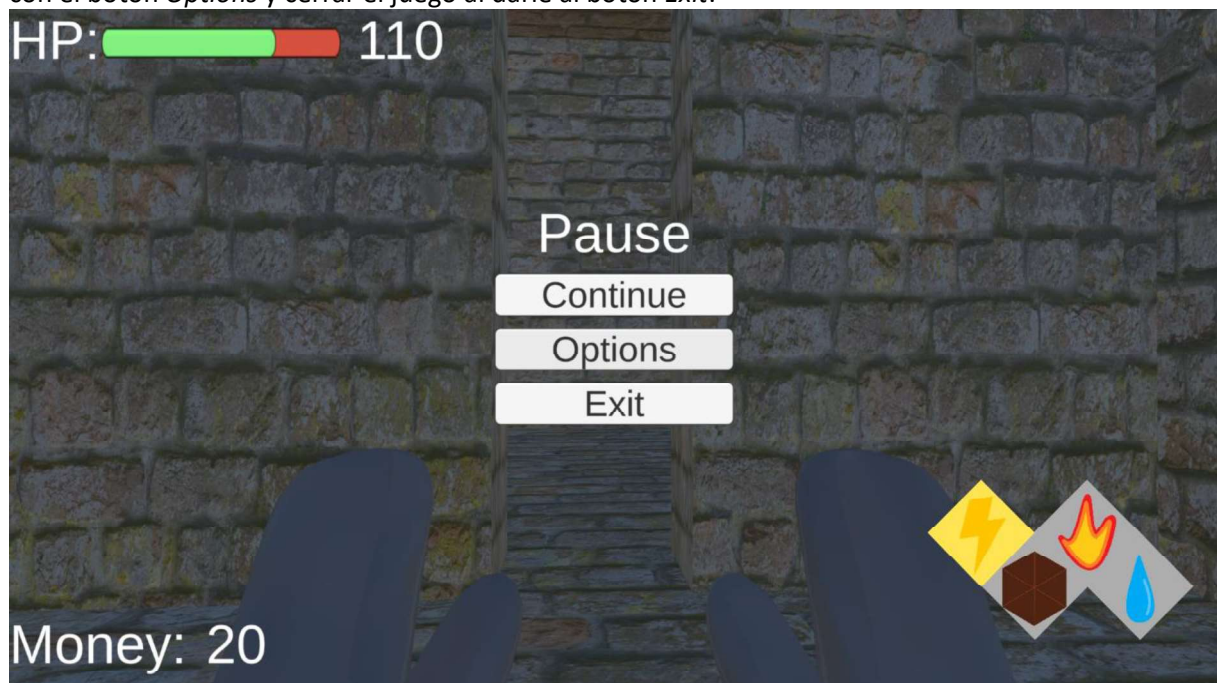


Figura 79: Menú de pausa

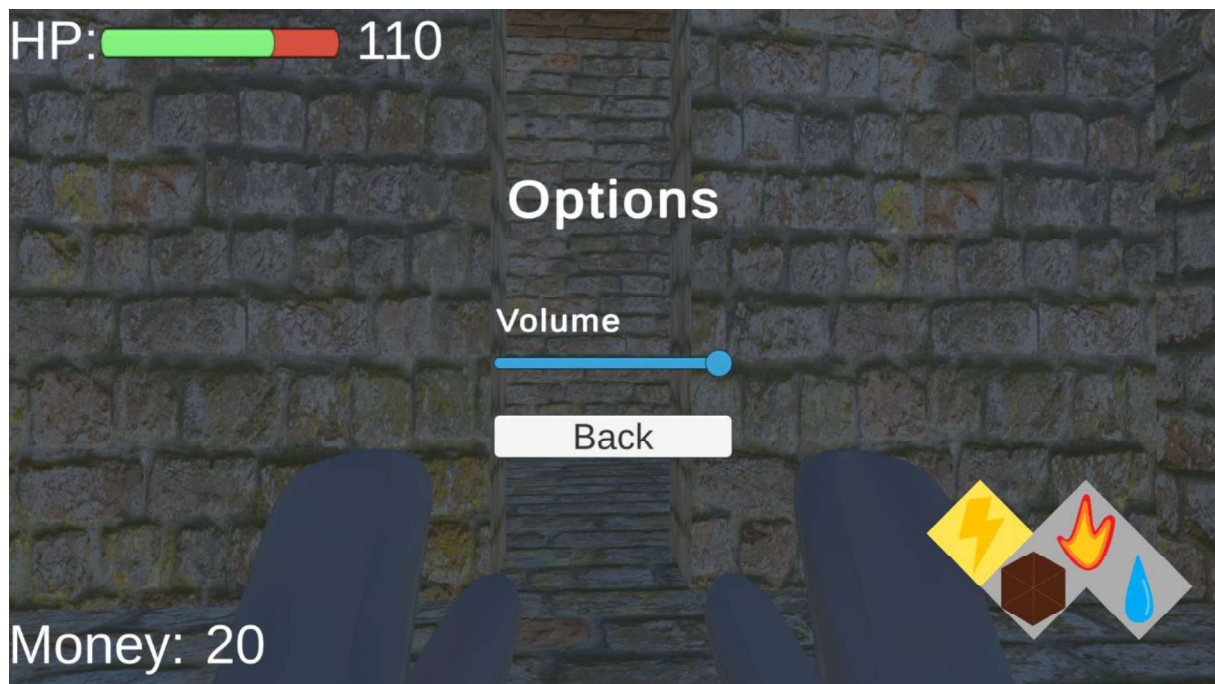


Figura 80: Opciones del menú de pausa

Adicionalmente, hay dos interfaces adicionales. Una que aparecerá cuando el jugador haya ganado la partida y que realizará un fundido en blanco para posteriormente dar el mensaje “You have escaped” y luego volver al menú inicial.

You have escaped!

Figura 81: Mensaje de victoria

Y otra apareciendo en el momento en el que la vida del jugador llegue a cero y con un mensaje de “Game Over”. En esta interfaz el jugador podrá jugar otra partida dándole al botón que pone *Retry*, volver al menú inicial dándole al botón que pone *Menu* o salirse del juego dándole al botón que pone *Exit*.



Figura 82: Interfaz de derrota

Es importante resaltar que todas estas interfaces poseen animaciones para hacer más natural la transición del juego a estas interfaces, como puede ser animaciones de fundido en negro.

6.3 CONTROLES

El jugador puede realizar varias acciones con las siguientes teclas:

- **Movimiento:** Con las teclas W, A, S y D el jugador podrá moverse hacia arriba, hacia la izquierda, hacia abajo y hacia la derecha respectivamente.
- **Girar la cámara:** Moviendo el ratón el jugador podrá mover la cámara de su personaje a los alrededores.
- **Atacar:** El jugador al pulsar el clic derecho del ratón lanzará una carta a la dirección a la cual este mirando. Esta acción es necesaria para derrotar a los enemigos.
- **Saltar:** Para realizar el salto, el jugador tendrá que presionar la tecla de ESPACIO.
- **Embestida:** El jugador al presionar la tecla E, realizara una embestida a la dirección a la cual se esté moviendo.
- **Recoger mejoras:** Para poder obtener los efectos beneficiosos de una mejora, el jugador tendrá que acercarse lo suficiente a la mejora y presionar la tecla F.

- Abrir el menú de pausa: Para que el jugador pueda abrir el menú de pausa, este deberá de presionar el botón ESC.

6.4 ENEMIGOS

A lo largo de la mazmorra aparecerán múltiples enemigos en las salas, los cuales el jugador tendrá que derrotar. Al entrar en una sala aparecerá una esfera oscura que ira creciendo rápidamente, y luego de ella saldrá el enemigo o mejora. Una vez haya pasado algunos segundos, la esfera reducirá su tamaño hasta desaparecer. En las siguientes imágenes se pueden observar la esfera de aparición y los modelos de los enemigos y mejoras:

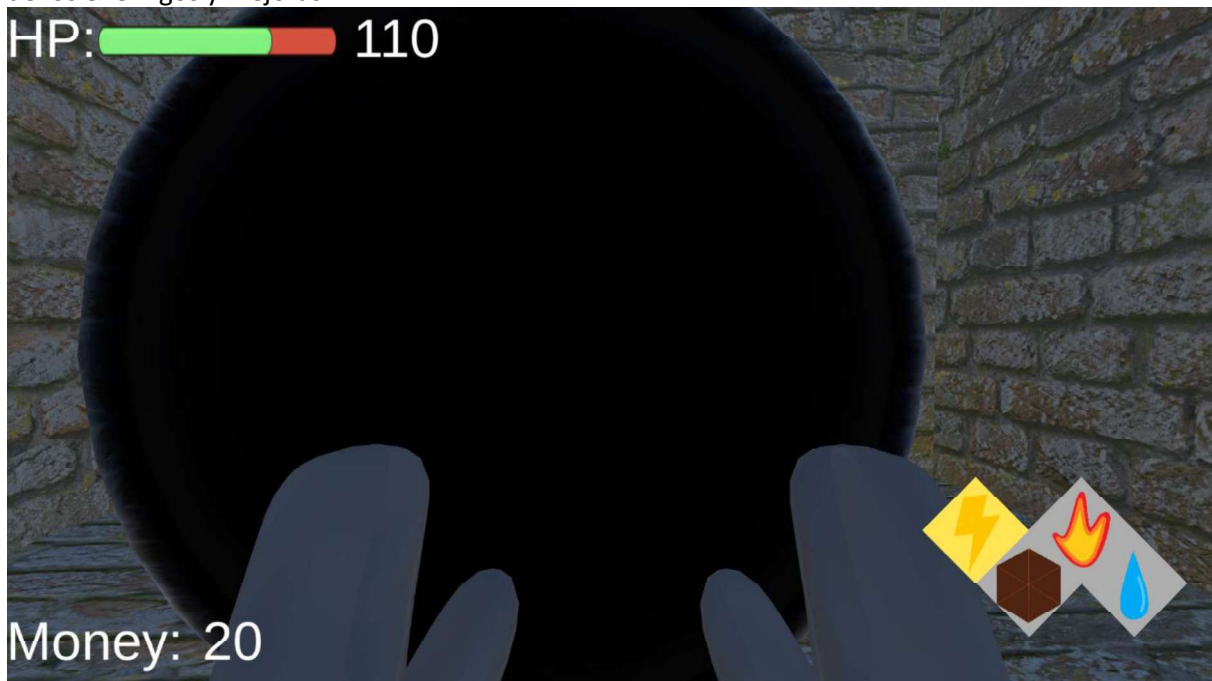


Figura 83: Esfera de aparición de enemigos y mejoras

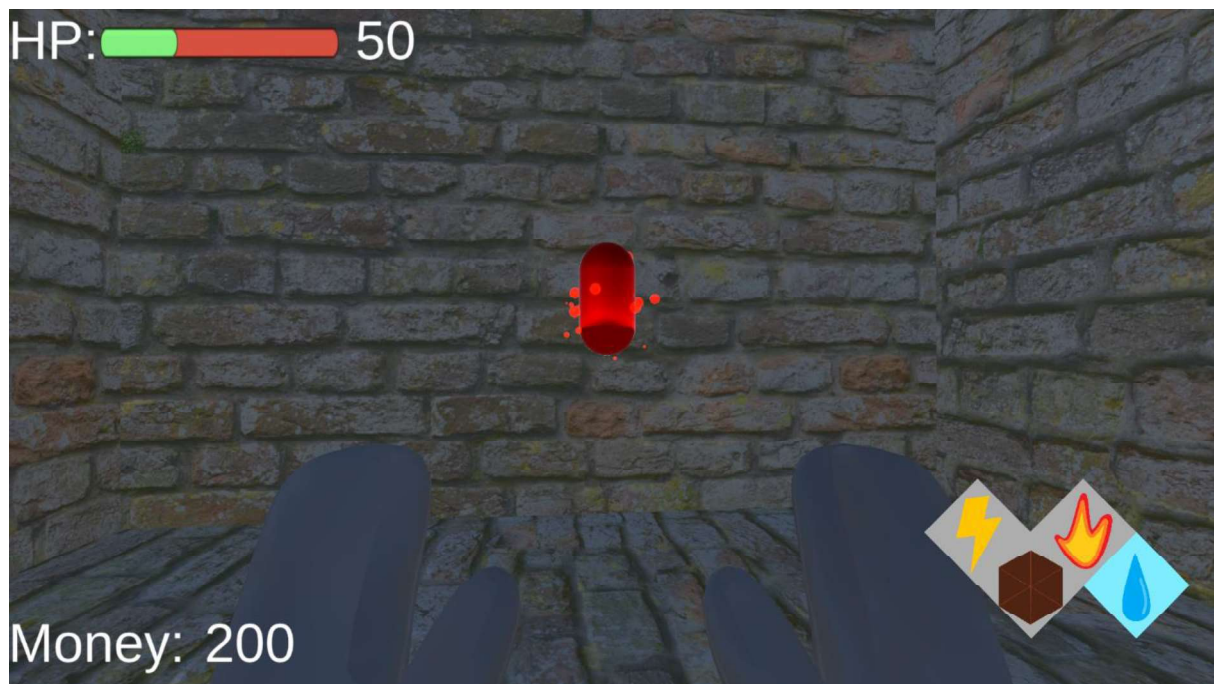


Figura 84: Mejora del videojuego de la sala del mercader



Figura 85: Mejora del videojuego de la sala de mejora

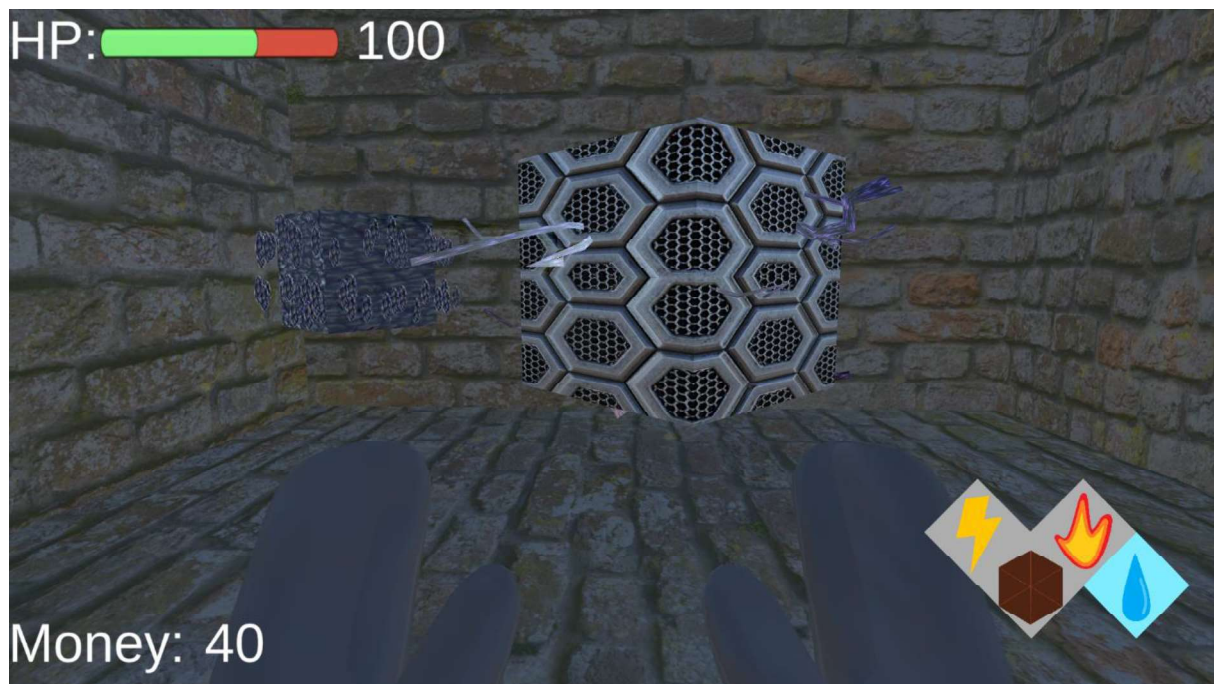


Figura 86: Enemigo eléctrico en el centro y enemigo de tierra a la izquierda

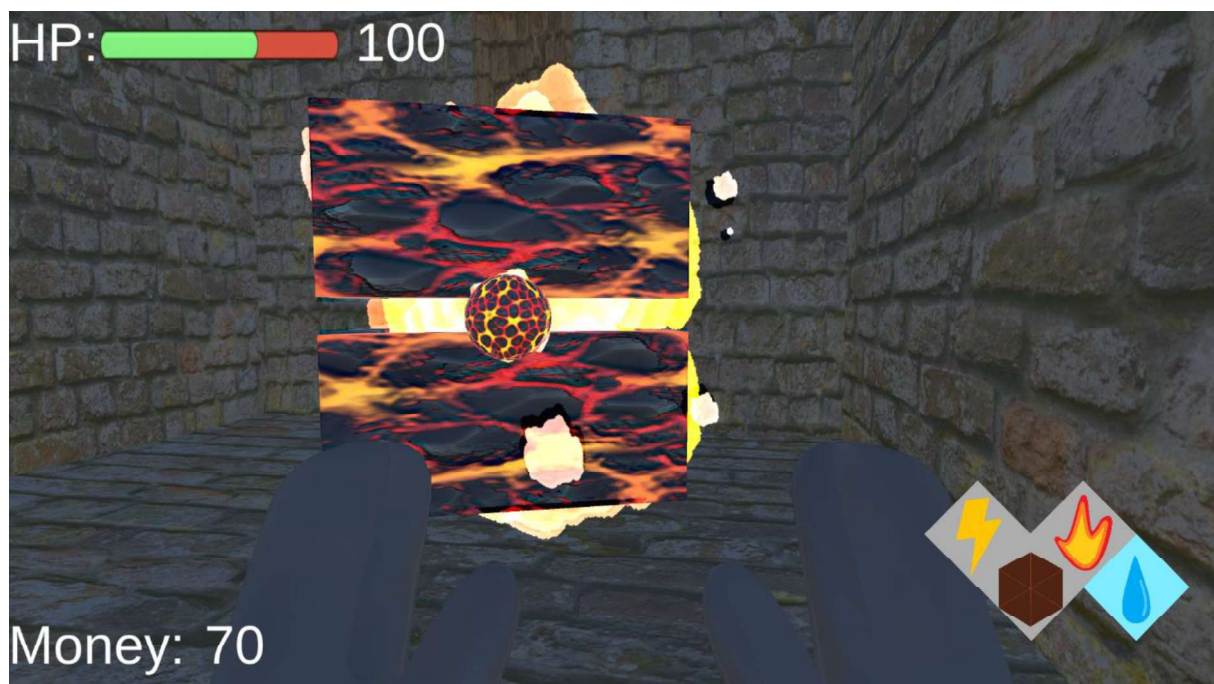


Figura 87: Enemigo de fuego lanzando una esfera de fuego

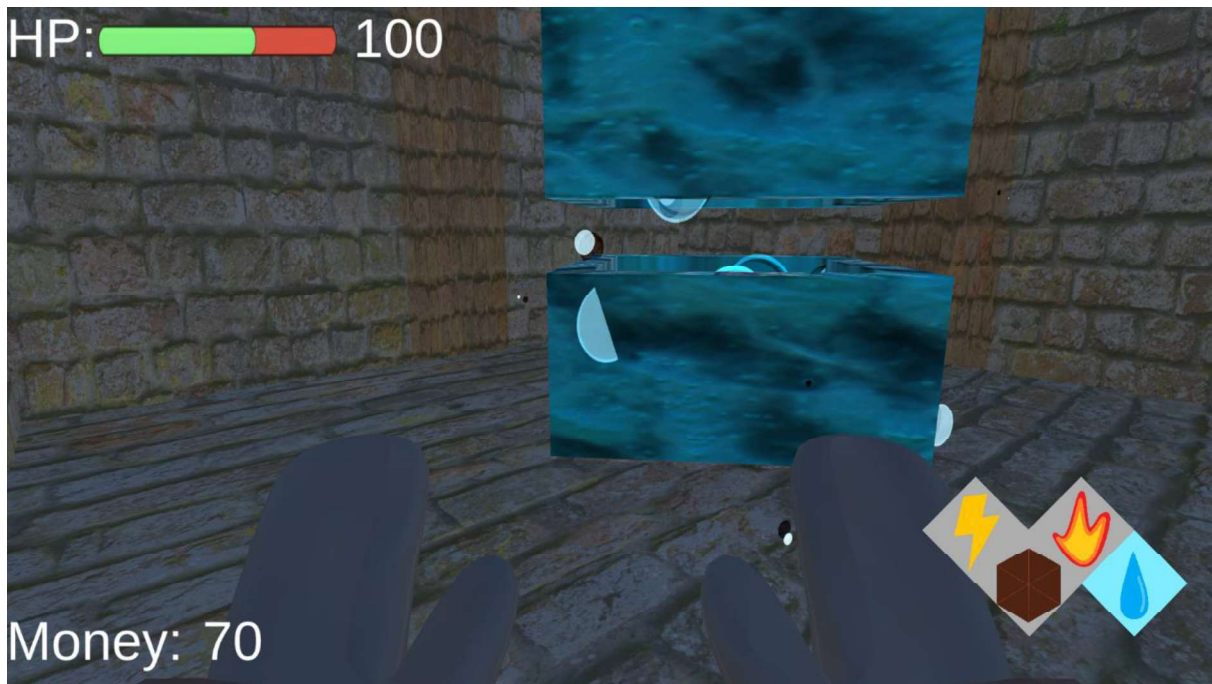


Figura 88: Enemigo de agua

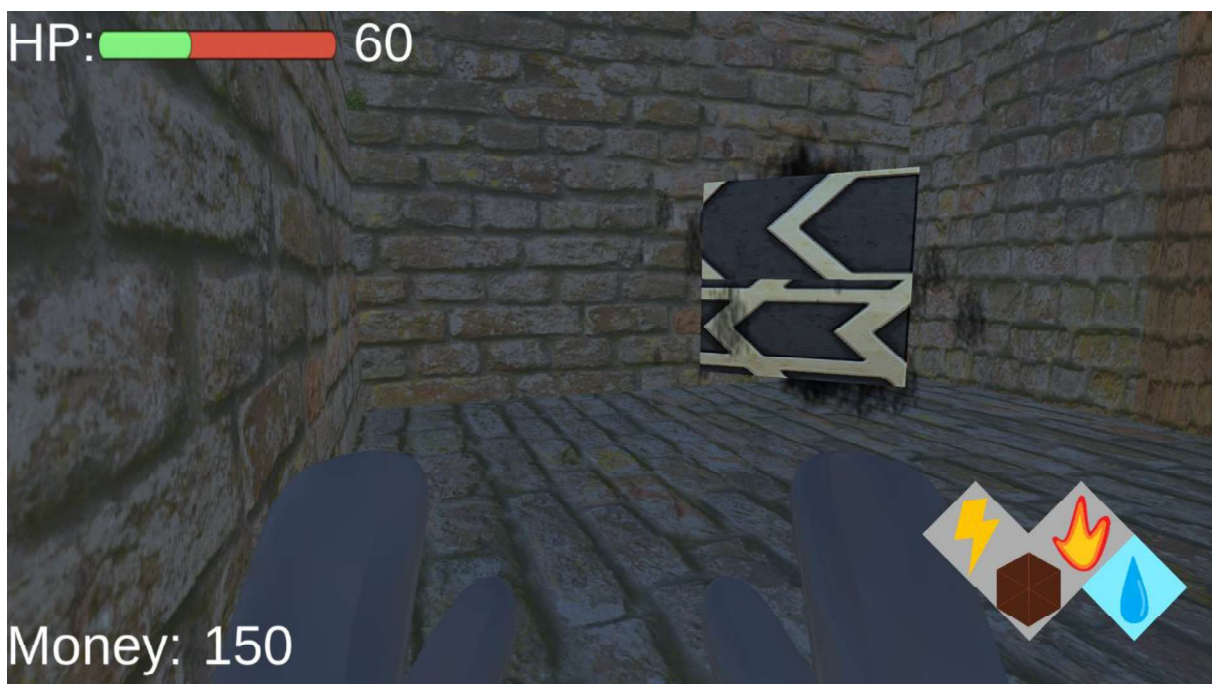


Figura 89: Jefe del piso

6.5 ENTORNO

En el videojuego habrá salas con hasta cuatro puertas posibles conectadas entre sí. Las salas tienen la misma forma que la de la siguiente figura:



Figura 90: Ejemplo del interior de una sala

Tras encontrar y derrotar al enemigo aparecerá una mejora y un portal. Al tocar el portal, teletransportará al siguiente piso o en caso de que ya no haya más pisos se teletransportara al jugador a la sala final, en la cual estará la salida, que al cruzarla el jugador habrá ganado la partida. En las siguientes imágenes se puede observar tanto el portal como la sala final:



Figura 91: Mejora por derrotar al jefe del piso y portal de teletransporte

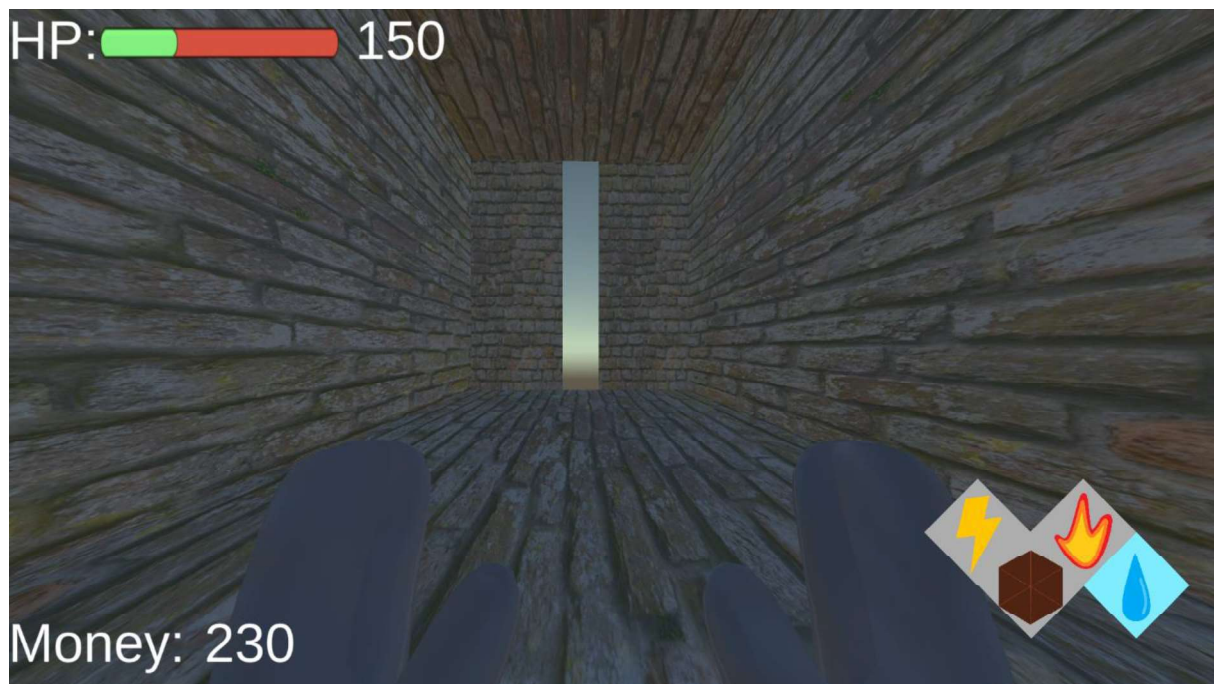


Figura 92: Sala final con la salida

7. VALORACIÓN ÉTICA DEL PROYECTO

Hoy en día los videojuegos tienen un papel muy importante en la vida de las personas y por consecuente en la sociedad. Debido a ello, cada vez más se le pone especial énfasis a que los videojuegos creados han de cumplir no solo con las normas legales, sino también reglas morales. Debido a esto, es importante analizar y comprobar si el trabajo realizado para este proyecto fin de grado es éticamente correcto.

Antes de comenzar, es importante recalcar la importancia de los beneficios que causan a los jugadores los videojuegos. Los beneficios generados por los videojuegos pueden ser diferentes en función de su género, por ejemplo, un videojuego de resolución de puzzles ayuda a generar un aumento en la capacidad de razonamiento del jugador, mientras que un videojuego de acción ayuda a mejorar sus reflejos. Sin embargo, existen varios beneficios que comparten todos los géneros de videojuegos, los cuales pueden ser el aumento de la creatividad y la atención visual del jugador entre otras muchas más.

En el caso del videojuego creado para este proyecto, esté cuenta con varios beneficios para los jugadores, los cuales son:

- Satisfacción personal: Debido a que es necesario muchos intentos para lograr la victoria en este videojuego, cuando se logra finalmente, provoca no solo una satisfacción por haber superado un reto que difícil, sino que además da al jugador un mensaje, de manera indirecta, el cual es que con esfuerzo todo es posible.
- Mejora en la capacidad de reacción del jugador: En este videojuego al ser un videojuego de acción el jugador tendrá que estar esquivando los ataques de los enemigos, por lo que será necesario contar con buenos reflejos. Este videojuego al consistir en jugar una y otra vez, el jugador irá mejorando gradualmente su respuesta motora.
- Mejora en el pensamiento rápido del jugador: Debido al sistema de elementos mencionados en el apartado 3, el jugador tendrá que usar la mejor carta en función de la situación en la que se encuentre en ese momento. De esta manera, el juego recompensará al jugador si este se adapta bien a las situaciones a las que se enfrentará. Esta adaptabilidad es posible mejorarla mediante ensayo y error jugando varias partidas.
- Socialización: Hoy en día los videojuegos es el pasatiempo más común entre los jóvenes de hoy en día (para el resto del mundo también es muy común sin embargo los jóvenes son el porcentaje más alto de jugadores de videojuegos), lo cual ayuda a muchos de estos a socializar entre ellos y crear nuevas amistades.

Como se puede ver, los videojuegos no son solo un pasatiempo como cualquier otro, sino que aporta una serie de beneficios que otros pasatiempos debido a sus limitaciones no pueden llegar a conseguir. Sin embargo, no todo son ventajas, ya que los beneficios de los videojuegos solo son una cara de una misma moneda. En la otra cara nos encontramos con los problemas que causan los videojuegos. En

este apartado con centraremos en las desventajas que este videojuego podría causar y de qué modo se han solventado. Es importante resaltar que este análisis se realizó antes de registrar el proyecto en la asignatura, ya que en caso de que no se pudiera solventar los problemas que este videojuego pudiera causar se cancelaría el proyecto.

Para realizar el análisis de problemas es importante recordar los géneros a los que pertenece este videojuego, ya que eso es de gran ayuda para identificar que desventajas podría causar. Como ya se sabe, este videojuego pertenece principalmente al género de mazmorras, género en que a base de la repetición de partidas el jugador llegará a alcanzarse con la victoria. Además del género principal también pertenece al género de acción y de disparos en primera persona. También es importante analizar las limitaciones que tiene este videojuego y que por ende quedaron fuera del alcance.

Una vez analizado a los géneros a los que pertenece el videojuego, se han identificado varios desafíos a los que se enfrentará este videojuego, los cuales son los siguientes:

- **Perdida de la noción del tiempo:** Los videojuegos, como hemos comentado anteriormente, es un pasatiempo, y como pasa con cualquier otro pasatiempo el jugador puede perder la noción del tiempo mientras juega a este debido a la inmersión que provoca. Esto es bastante problemático debido a que la mayoría de los jugadores de videojuegos suelen ser jóvenes los cuales son mucho más vulnerables a problemas como la adicción. Muchos estudios ya han demostrado que un alto porcentaje de los jóvenes de hoy en día son adictos a las nuevas tecnologías, y los videojuegos no son la excepción. Esta adicción puede influir negativamente no solo en el apartado profesional de los usuarios de videojuegos, sino también en el personal y social.
- **Derechos de autor:** Como se ha comentado en el apartado 2.3 hay material artístico que se ha tenido que recoger de Internet. Este material es la música, los efectos de sonido y las texturas. Esto puede presentar un problema de derechos de autor, ya que usar material del cual no se es propietario no es ni legal ni ético.
- **Contenido ofensivo o violento:** Los videojuegos hoy en día tienen regulaciones de edad para evitar que jóvenes jueguen a juegos de adultos que contengan contenido no apto para sus edades. Sin embargo, esto no es suficiente para evitar que jóvenes compren juegos con contenido ofensivo y violento que se supone no deberían de ver.

Habiendo analizado los problemas a los que el videojuego de este proyecto se enfrenta es hora de explicar el método para la resolución de estos problemas.

- Para resolver el problema con la pérdida de la noción del tiempo y de la adicción se planteó la aparición de mensajes en el menú inicial que servirá como recordatorio del tiempo que ha pasado en el videojuego. También se ha planteado la inclusión de temporizadores en las interfaces de usuario que indiquen al jugador cuánto tiempo lleva en la partida. Debido a que el videojuego creado para este proyecto no va a ser lanzado al mercado no se incorporó esto

al trabajo de esta memoria. Aun así, está planteado que en caso de que el videojuego saliese al mercado sería de vital importancia agregar esas funcionalidades.

- Para resolver el problema de los materiales externos se tomó la decisión de usar música, efectos de sonido y texturas que estén libres de copyright o cualquier derecho de autor, y de esta manera poder usarlos libremente.
- Para resolver el problema del contenido ofensivo y violento se ha eliminado todo componente que pueda representar violencia. Debido a ello en vez de usar una pistola se está usando cartas mágicas y en vez de enemigos humanoides se están usando otro tipo de modelos. Además, se ha evitado el uso de partículas de sangre cuando el jugador o los enemigos reciben un golpe, al igual que se ha eliminado de los títulos palabras inapropiadas usando otras más correctas, como puede ser el cambio del menú de fin de juego, en el que ponía originalmente "YOU DIED" y se cambió por "GAME OVER".

A modo de conclusión de este capítulo podemos afirmar que mediante los métodos y técnicas mencionados anteriormente se ha conseguido mitigar y reducir los impactos negativos que podría tener este videojuego sin afectar a los beneficios que este aporta.

8. CONCLUSIONES Y TRABAJO FUTURO

8.1 DISCUSIÓN DE LOS RESULTADOS OBTENIDOS

Tras finalizar el desarrollo del proyecto, se ha conseguido cumplir tanto el objetivo principal de este proyecto como los objetivos secundarios planteados en el apartado 2.3. El videojuego final tiene todas las mecánicas propuestas en el diseño inicial del proyecto y todas ellas funcionan a la perfección sin ningún tipo de error, además de contar con la posibilidad de poder crear una demo para cualquier tipo de ordenador, la cual pesa 651 KB.

Aunque a simple vista pueda parecer un videojuego muy simplista y sin nada de profundidad, ya sea por contenido o por diseños, hay que tener en cuenta de que el tiempo que se necesita para desarrollar un videojuego es bastante elevado, no solo en cuanto a la programación se refiere, sino también a la creación del apartado artístico, en el que se agrupa tanto modelaje, animaciones, efectos visuales e interfaces de usuario. Debido a este principal inconveniente, se decidió reducir contenido de manera que no afectase a la calidad del trabajo. Para ello se decidió simplificar lo máximo posible la anatomía de los modelos creados en Blender.

Pese a ese principal problema, este videojuego ha cumplido con lo esperado, y, si bien es cierto que necesitaría que se le agregase más contenido como puede ser salas con una estructura diferente o nuevos tipos de enemigos, este producto podría salir al mercado y tener una recepción bastante positiva por los usuarios.

8.2 TRABAJO FUTURO

Debido al tiempo reducido hay ciertas tareas que no fueron consideradas en el diseño inicial del videojuego debido a que quedaban fuera del alcance del proyecto. A continuación, se listarán estas tareas descartadas y las cuales se podrían implementar a futuro:

- Mayor variedad de enemigos, animaciones y patrones de ataques: Como se ha estado comentando a lo largo del documento, y en especial en el apartado del alcance del punto 2.3, para poder realizar este proyecto en el plazo previsto, se optó por reducir la variedad de enemigos y sus animaciones y patrones de ataques, ya que el crearlos personalmente consumiría mucho tiempo.
- Efectos especiales de las cartas mágicas al impactar en superficies: Una de las ideas que se tenía antes de crear el diseño, era que las cartas no solo aplicasen efectos especiales al impactar a los enemigos, como puede ser el efecto de quemadura o congelación, sino que generasen otro efecto adicional al impactar en superficies. Para comprender mejor esta idea, se planteó que, si la carta de fuego impactaba en el suelo, esta incendiaría la zona en la cual ha impactado, y que todo enemigo o jugador que pasase recibiría daño por segundo. Al final no se tuvo en cuenta esta idea en el diseño final, debido a que hubiera consumido mucho

tiempo y más que ser una ayuda para el jugador podría ser una molestia al llenar la pantalla de efectos visuales.

- Agregar diferentes tipos de salas: En el diseño se plantearon los cinco tipos de salas mencionadas en el apartado 3, y que, además, para ahorrar tiempo y recursos, tuvieran una misma apariencia. A futuro se ha planteado no solo modificar la apariencia de las salas ya existentes para que se puedan identificar mejor que tipo de salas son, sino que también se ha planteado agregar nuevos tipos de salas y que a medida que se vaya subiendo de pisos, las salas tengan diferentes texturas.
- Crear música, efectos de sonido y texturas propias: Una de las cosas que se han quedado fuera del alcance de este proyecto ha sido la creación de los elementos más artísticos y sin relación con la informática. Como en el diseño se consideró indispensable estos elementos artísticos para la creación de un buen juego, se decidió usar música, efectos de sonido y texturas creadas por otras personas. Sin embargo, en caso de querer lanzar el juego de manera oficial al mercado, habría que crear una música propia para este videojuego. Como se mencionó en el apartado 4.3, se contratará a profesionales en estos ámbitos para que nos den estos elementos artísticos.
- Lanzar el videojuego al mercado: El lanzamiento del proyecto a las plataformas de videojuegos estaba fuera del alcance de este proyecto, no solo por cuestiones económicas sino también con cuestiones personales. Pese a que el videojuego creado para este proyecto tiene la calidad suficiente como para ser aceptado por la mayoría de las personas de la industria, se ha decidido no lanzarlo de momento y pulirlo a lo largo del tiempo hasta que considere que es el momento del lanzamiento oficial.
- Diseñar y programar un modo multijugador: Como se comentó en el apartado de alcance del proyecto, se decidió que para este proyecto no se implementará un modo multijugador, ya que no solo sería necesario el alquilar un servidor para implantarlo, sino que además no aportaría ningún valor al videojuego, ya que ninguna de las dos ideas planteadas para el modo multijugador aportaba nada al producto final. Por ello se ha considerado a futuro el diseñar y programar el concepto de un modo multijugador que aporte valor al videojuego.

9. BIBLIOGRAFÍA

-
- [1] «Persona 5 Royal en Steam». https://store.steampowered.com/app/1687950/Persona_5_Royal/ (accedido 22 de junio de 2023).
- [2] «Rogue en Steam». <https://store.steampowered.com/app/1443430/Rogue/> (accedido 22 de junio de 2023).
- [3] «The Binding of Isaac: Rebirth en Steam». https://store.steampowered.com/app/250900/The_Binding_of_Isaac_Rebirth/ (accedido 21 de junio de 2023).
- [4] «Moonlighter en Steam». <https://store.steampowered.com/app/606150/Moonlighter/> (accedido 22 de junio de 2023).
- [5] «Hades en Steam». <https://store.steampowered.com/app/1145360/Hades/> (accedido 22 de junio de 2023).
- [6] «Gunfire Reborn en Steam». https://store.steampowered.com/app/1217060/Gunfire_Reborn/ (accedido 22 de junio de 2023).
- [7] «Steam, The Ultimate Online Game Platform». <https://store.steampowered.com/about/> (accedido 14 de junio de 2023).
- [8] «Gunfire Reborn - Gunfire Reborn has sold 1 million copies worldwide! Roadmap is available now! - Noticias de Steam», 15 de diciembre de 2020. <https://store.steampowered.com/news/app/1217060/view/2918859890108273954> (accedido 2 de junio de 2023).
- [9] «Gunfire Reborn Price history», *SteamDB*, 1 de junio de 2023. <https://steamdb.info/app/1217060/charts/> (accedido 2 de junio de 2023).
- [10] «Specification GT72 6QD Dominator G». <http://es.msi.com/Laptop/GT72-6QD-Dominator-G/Specification?%2FLaptop%2FGT72-6QD-Dominator-G%2FSpecification=> (accedido 16 de junio de 2023).
- [11] U. Technologies, «Unity - Manual: Scenes». <https://docs.unity3d.com/Manual/CreatingScenes.html> (accedido 15 de junio de 2023).
- [12] Unity Technologies, «Unity - Manual: GameObjects». <https://docs.unity3d.com/Manual/GameObjects.html> (accedido 15 de junio de 2023).
- [13] U. Technologies, «Unity - Manual: Prefabs». <https://docs.unity3d.com/Manual/Prefabs.html> (accedido 15 de junio de 2023).
- [14] U. Technologies, «Unity - Manual: Use components». <https://docs.unity3d.com/Manual/UsingComponents.html> (accedido 15 de junio de 2023).

- [15] Unity Technologies, «Unity - Manual: Camera component». <https://docs.unity3d.com/Manual/class-Camera.html> (accedido 21 de junio de 2023).
- [16] «Post-process Layer | Package Manager UI website». <https://docs.unity3d.com/Packages/com.unity.postprocessing@2.0-preview/manual/Quick-start.html> (accedido 22 de junio de 2023).
- [17] Unity Technologies, «Unity - Manual: Materials». <https://docs.unity3d.com/Manual/Materials.html> (accedido 15 de junio de 2023).
- [18] U. Technologies, «Unity - Manual: Mesh Renderer component». <https://docs.unity3d.com/Manual/class-MeshRenderer.html> (accedido 21 de junio de 2023).
- [19] U. Technologies, «Unity - Manual: Mesh Filter component». <https://docs.unity3d.com/Manual/class-MeshFilter.html> (accedido 21 de junio de 2023).
- [20] U. Technologies, «Unity - Manual: Transforms». <https://docs.unity3d.com/Manual/class-Transform.html> (accedido 15 de junio de 2023).
- [21] U. Technologies, «Unity - Manual: Rigidbody physics». <https://docs.unity3d.com/Manual/rigidbody-physics-section.html> (accedido 15 de junio de 2023).
- [22] Unity Technologies, «Unity - Manual: Collision». <https://docs.unity3d.com/Manual/collision-section.html> (accedido 15 de junio de 2023).
- [23] U. Technologies, «Unity - Scripting API: CharacterController». <https://docs.unity3d.com/ScriptReference/CharacterController.html> (accedido 22 de junio de 2023).
- [24] Unity Technologies, «Unity - Manual: Animation». <https://docs.unity3d.com/Manual/AnimationSection.html> (accedido 15 de junio de 2023).
- [25] U. Technologies, «Unity - Manual: Navigation and Pathfinding». <https://docs.unity3d.com/Manual/Navigation.html> (accedido 15 de junio de 2023).
- [26] U. Technologies, «Unity - Manual: Visual effects». <https://docs.unity3d.com/Manual/visual-effects.html> (accedido 15 de junio de 2023).
- [27] Unity Technologies, «Unity - Manual: Audio». <https://docs.unity3d.com/Manual/Audio.html> (accedido 15 de junio de 2023).
- [28] U. Technologies, «Unity - Manual: Scripting». <https://docs.unity3d.com/Manual/ScriptingSection.html> (accedido 15 de junio de 2023).
- [29] Unity Technologies, «Unity - Manual: Create user interfaces (UI)». <https://docs.unity3d.com/Manual/UIToolkits.html> (accedido 15 de junio de 2023).

- [30] «Stylized Lava materials | 2D Textures & Materials | Unity Asset Store». <https://assetstore.unity.com/packages/2d/textures-materials/stylized-lava-materials-180943> (accedido 21 de junio de 2023).
- [31] «Stylize Water Texture | 2D Water | Unity Asset Store». <https://assetstore.unity.com/packages/2d/textures-materials/water/stylize-water-texture-153577> (accedido 21 de junio de 2023).
- [32] «Yughues Free Metal Materials | 2D Metals | Unity Asset Store». <https://assetstore.unity.com/packages/2d/textures-materials/metals/yughues-free-metal-materials-12949> (accedido 21 de junio de 2023).
- [33] «50 Free PBR Materials | 2D Textures & Materials | Unity Asset Store». <https://assetstore.unity.com/packages/2d/textures-materials/50-free-pbr-materials-242760> (accedido 21 de junio de 2023).
- [34] «Destructive Force (Free Action Music) | Audio Music | Unity Asset Store». <https://assetstore.unity.com/packages/audio/music/destructive-force-free-action-music-205757> (accedido 21 de junio de 2023).
- [35] Boris, «Dungeon Generation in Binding of Isaac», *BorisTheBrave.Com*, 12 de septiembre de 2020. <https://www.boristhebrave.com/2020/09/12/dungeon-generation-in-binding-of-isaac/> (accedido 4 de junio de 2023).